Application of AI in Detecting and Preventing Brute-Force Attacks on SSH Systems with Early Prediction

FPT University - Information Assurance

2026

Contents

1 FRONT MATTER **6**

2 DESIGN AND IMPLEMENTATION OF AN INTELLIGENT SSH BRUTE-FORCE ATTACK DETECTION AND PREVENTION SYSTEM USING ISOLATION FOREST WITH EWMA-ADAPTIVE PERCENTILE DYNAMIC THRESHOLDING **7**

2.1 ABSTRACT . . . 8
2.2 ACKNOWLEDGEMENT . . . 9
2.3 TABLE OF CONTENTS . . . 10
2.4 LIST OF FIGURES . . . 14
2.5 LIST OF TABLES . . . 15
2.6 ABBREVIATIONS . . . 16

3 CHAPTER 1: INTRODUCTION **19**

3.1 1.1 Background . . . 19
3.2 1.2 Problem Statement . . . 21
3.2.1 Research Questions . . . 22
3.3 1.3 Research Objectives . . . 23
3.4 1.4 Significance of the Study . . . 24
3.4.1 1.4.1 Practical Significance . . . 24
3.4.2 1.4.2 Research Significance . . . 25
3.4.3 1.4.3 Empirical Significance . . . 25
3.5 1.5 Scope and Limitations . . . 25
3.5.1 1.5.1 Scope . . . 25
3.5.2 1.5.2 Limitations . . . 26
3.6 1.6 Thesis Structure . . . 26

4 CHAPTER 2: LITERATURE REVIEW **28**

4.1 2.1 Secure Shell (SSH) Protocol and Its Security Landscape . . . 28
4.2 2.2 Brute-Force Attacks: Taxonomy, Patterns, and Evolution . . . 29
4.3 2.3 Traditional Defense Mechanisms and Their Limitations . . . 31
4.4 2.4 Anomaly Detection for Intrusion Detection Systems . . . 32
4.5 2.5 Isolation Forest and Its Extensions . . . 34
4.5.1 2.5.1 Algorithmic Foundation . . . 34
4.5.2 2.5.2 Isolation Tree Construction . . . 34
4.5.3 2.5.3 Anomaly Score Computation . . . 34
4.5.4 2.5.4 Computational Complexity and Practical Advantages . . . 35
4.5.5 2.5.5 Extended Isolation Forest . . . 35
4.6 2.6 Local Outlier Factor (LOF) . . . 36
4.6.1 2.6.1 Algorithmic Foundation . . . 36
4.6.2 2.6.2 Mathematical Formulation . . . 36
4.6.3 2.6.3 Strengths and Limitations for SSH Detection . . . 37
4.7 2.7 One-Class Support Vector Machine (OCSVM) . . . 37

4.7.1 2.7.1 Algorithmic Foundation . . . 37
4.7.2 2.7.2 Optimization Problem . . . 37
4.7.3 2.7.3 Strengths and Limitations for SSH Detection . . . 38
4.8 2.8 Feature Selection and Data Preprocessing for Intrusion Detection Systems . . . 39
4.9 2.9 Dynamic Threshold Mechanisms for Anomaly Detection . . . 41
4.9.1 2.9.1 The Static Threshold Problem . . . 41
4.9.2 2.9.2 Exponentially Weighted Moving Average (EWMA) . . . 41
4.9.3 2.9.3 Adaptive Percentile Method . . . 42
4.9.4 2.9.4 Hybrid EWMA-Adaptive Percentile Method . . . 42
4.10 2.10 Related Work in SSH Brute-Force Detection . . . 43
4.10.1 2.10.1 Flow-Based and Network-Layer Approaches . . . 43
4.10.2 2.10.2 Machine Learning Approaches on Benchmark Datasets . . . 43
4.10.3 2.10.3 SSH-Specific Machine Learning Studies . . . 44
4.10.4 2.10.4 Deep Learning and Advanced Approaches . . . 45
4.11 2.11 Summary of the Literature Review . . . 47
4.12 2.12 Research Gap and Contribution . . . 48

5 CHAPTER 3: METHODOLOGY 50
5.1 3.1 Research Design Overview . . . 50
5.1.1 3.1.1 Philosophical and Methodological Foundations . . . 50
5.1.2 3.1.2 Semi-Supervised Anomaly Detection Paradigm . . . 51
5.2 3.2 System Architecture . . . 52
5.2.1 3.2.1 High-Level Architecture . . . 52
5.2.2 3.2.2 Service Inventory . . . 53
5.2.3 3.2.3 Data Flow Pipeline . . . 53
5.3 3.3 Laboratory Environment Setup . . . 54
5.3.1 3.3.1 Hardware and Software Configuration . . . 54
5.3.2 3.3.2 Honeypot Deployment . . . 55
5.3.3 3.3.3 Simulation Environment . . . 55
5.4 3.4 Data Collection Methods . . . 56
5.4.1 3.4.1 Data Sources and Rationale . . . 56
5.4.2 3.4.2 Log Data Format . . . 57
5.4.3 3.4.3 Labeling Strategy . . . 58
5.5 3.5 Data Preprocessing Pipeline . . . 58
5.5.1 3.5.1 Pipeline Overview . . . 58
5.5.2 3.5.2 Stage 1: Log Parsing . . . 59
5.5.3 3.5.3 Stage 2: Labeling . . . 59
5.5.4 3.5.4 Stage 3: Feature Extraction via Sliding Window . . . 59
5.5.5 3.5.5 Stage 4: Data Splitting . . . 60
5.5.6 3.5.6 Stage 5: Feature Scaling . . . 61
5.6 3.6 Feature Engineering and Selection . . . 62
5.6.1 3.6.1 Feature Design Philosophy . . . 62
5.6.2 3.6.2 Feature Definitions . . . 62
5.6.3 3.6.3 Feature Selection Rationale . . . 65
5.7 3.7 Anomaly Detection Model Configuration . . . 66
5.7.1 3.7.1 Model Selection Criteria . . . 66
5.7.2 3.7.2 Isolation Forest (IF) . . . 66
5.7.3 3.7.3 Local Outlier Factor (LOF) . . . 67
5.7.4 3.7.4 One-Class SVM (OCSVM) . . . 68
5.7.5 3.7.5 Model Comparison Summary . . . 69
5.7.6 3.7.6 Hyperparameter Optimization . . . 70
5.8 3.8 Dynamic Threshold Engine Design . . . 70
5.8.1 3.8.1 Motivation and Theoretical Basis . . . 70
5.8.2 3.8.2 Algorithm Specification . . . 71
5.8.3 3.8.3 Self-Calibration Mechanism . . . 72

5.9 3.9 Alert and Prevention Module . . . 73
5.9.1 3.9.1 Fail2Ban Integration . . . 73
5.9.2 3.9.2 Multi-Channel Alert System . . . 73
5.9.3 3.9.3 Administrator Feedback Loop . . . 74
5.10 3.10 Visualization Module . . . 74
5.10.1 3.10.1 React Dashboard . . . 74
5.10.2 3.10.2 Kibana Integration . . . 74
5.11 3.11 Evaluation Metrics . . . 75
5.11.1 3.11.1 Classification Metrics . . . 75
5.11.2 3.11.2 Priority Hierarchy . . . 76
5.11.3 3.11.3 System-Level Metrics . . . 76
5.12 3.12 Limitations of the Methodology . . . 77
5.12.1 3.12.1 Window Size Constraints . . . 77
5.12.2 3.12.2 Training Data Purity Assumption . . . 77
5.12.3 3.12.3 Single-Environment Generalization . . . 77
5.12.4 3.12.4 Dataset Specificity . . . 77
5.12.5 3.12.5 Ethical Considerations . . . 78

6 CHAPTER 4: EXPERIMENTAL RESULTS AND ANALYSIS 79
6.1 4.1 Experimental Setup . . . 79
6.1.1 4.1.1 Computational Environment . . . 79
6.1.2 4.1.2 Experimental Protocol . . . 79
6.1.3 4.1.3 Reproducibility . . . 80
6.2 4.2 Preprocessing Results . . . 80
6.2.1 4.2.1 Log Parsing Statistics . . . 80
6.2.2 4.2.2 Labeling Statistics . . . 81
6.2.3 4.2.3 Feature Extraction Statistics . . . 81
6.2.4 4.2.4 Data Split Statistics . . . 82
6.2.5 4.2.5 Honeypot Data Exploratory Analysis . . . 82
6.2.6 4.2.6 Simulation Data Exploratory Analysis . . . 84
6.2.7 4.2.7 Feature Distribution Comparison . . . 84
6.3 4.3 Model Training and Evaluation (Baseline) . . . 85
6.3.1 4.3.1 Training Configuration . . . 85
6.3.2 4.3.2 Baseline Results . . . 86
6.3.3 4.3.3 Confusion Matrices . . . 86
6.3.4 4.3.4 Baseline Analysis . . . 87
6.4 4.4 Feature Importance Experiment . . . 87
6.4.1 4.4.1 Methodology . . . 87
6.4.2 4.4.2 Results . . . 88
6.4.3 4.4.3 Interpretation . . . 88
6.5 4.5 Model Optimization Results . . . 90
6.5.1 4.5.1 Optimization Strategy . . . 90
6.5.2 4.5.2 Optimized Results . . . 90
6.5.3 4.5.3 Improvement Analysis . . . 91
6.5.4 4.5.4 Model Selection Justification . . . 92
6.6 4.6 Dynamic Threshold Engine Evaluation . . . 93
6.6.1 4.6.1 Experimental Design . . . 93
6.6.2 4.6.2 Distribution Shift Adaptation . . . 93
6.6.3 4.6.3 Burst False Positive Reduction . . . 93
6.6.4 4.6.4 Two-Level Detection Behavior . . . 94
6.6.5 4.6.5 Parameter Sensitivity Analysis . . . 94
6.6.6 4.6.6 Self-Calibration Results . . . 95
6.7 4.7 System Integration Test Results . . . 95
6.7.1 4.7.1 End-to-End Pipeline Validation . . . 95
6.7.2 4.7.2 Throughput Analysis . . . 96

6.7.3 4.7.3 Resource Utilization . . . 96
6.8 4.8 Attack Scenario Testing . . . 96
6.8.1 4.8.1 Scenario Design . . . 96
6.8.2 4.8.2 Scenario 1: Basic Brute-Force . . . 97
6.8.3 4.8.3 Scenario 2: Distributed Attack . . . 97
6.8.4 4.8.4 Scenario 3: Low-and-Slow . . . 98
6.8.5 4.8.5 Scenario 4: Credential Stuffing . . . 99
6.8.6 4.8.6 Scenario 5: Dictionary Attack . . . 99
6.8.7 4.8.7 Consolidated Scenario Results . . . 99
6.9 4.9 Detection Time Comparison: AI vs. Fail2Ban . . . 100
6.9.1 4.9.1 Comparative Framework . . . 100
6.9.2 4.9.2 Detection Time Results . . . 100
6.9.3 4.9.3 Analysis . . . 101
6.10 4.10 Comparison with State-of-the-Art . . . 101
6.10.1 4.10.1 Literature Comparison . . . 101
6.10.2 4.10.2 Comparative Analysis . . . 103
6.10.3 4.10.3 Positioning . . . 104
6.11 4.11 Interpretation and Implications . . . 104
6.11.1 4.11.1 Theoretical Implications . . . 104
6.11.2 4.11.2 Practical Implications . . . 105
6.11.3 4.11.3 Limitations of the Results . . . 105
6.11.4 4.11.4 Summary of Key Findings . . . 106

7 CHAPTER 5: DISCUSSION 107
7.1 5.1 Restatement of Research Problem . . . 107
7.2 5.2 Achievement of Research Objectives . . . 108
7.2.1 5.2.1 Objective 1: Behavioral Feature Set Design . . . 108
7.2.2 5.2.2 Objective 2: Model Training and Evaluation . . . 108
7.2.3 5.2.3 Objective 3: Dynamic Threshold . . . 109
7.2.4 5.2.4 Objective 4: Integrated System . . . 109
7.2.5 5.2.5 Objective 5: Attack Scenario Evaluation . . . 109
7.3 5.3 Interpretation of Key Findings . . . 109
7.3.1 5.3.1 Why Timing Features Dominate . . . 109
7.3.2 5.3.2 Why Isolation Forest Is Chosen Despite OCSVM Having Higher F1 . . . 110
7.3.3 5.3.3 Dynamic Threshold vs. Static Threshold Advantage . . . 111
7.3.4 5.3.4 FPR Analysis and Acceptable Trade-off in Security Context . . . 112
7.4 5.4 Comparison with Existing Literature . . . 112
7.5 5.5 Practical Implications . . . 115
7.5.1 5.5.1 Deployment Considerations . . . 115
7.5.2 5.5.2 Scalability Considerations . . . 116
7.5.3 5.5.3 Cost-Benefit Analysis . . . 117
7.6 5.6 Limitations and Threats to Validity . . . 117
7.7 5.7 Chapter Summary . . . 118

8 CHAPTER 6: CONCLUSION AND FUTURE WORK 120
8.1 6.1 Conclusion . . . 120
8.2 6.2 Contributions Summary . . . 122
8.3 6.3 Future Work . . . 122
8.3.1 6.3.1 Deep Learning for Improved Detection . . . 123
8.3.2 6.3.2 Online Learning and Continuous Adaptation . . . 123
8.3.3 6.3.3 Multi-Protocol Extension . . . 124
8.3.4 6.3.4 Federated Detection . . . 124
8.3.5 6.3.5 Threat Intelligence Integration . . . 125
8.3.6 6.3.6 SOAR Integration . . . 125
8.3.7 6.3.7 Explainability and Interpretability . . . 126

8.3.8 6.3.8 Long-Term IP Profiling 126
8.3.9 6.3.9 Real-World Deployment Validation 126
8.4 REFERENCES 127
8.5 APPENDICES 131
8.5.1 Appendix A: Feature Extraction Code Snippets 131
8.5.2 Appendix B: Docker Compose Configuration 131
8.5.3 Appendix C: Isolation Forest Hyperparameter Tuning Results 131
8.5.4 Appendix D: Attack Simulation Scripts 132
8.5.5 Appendix E: Logstash Pipeline Configuration 132
8.5.6 Appendix F: Dynamic Threshold Parameter Sensitivity Analysis 132

Chapter 1

FRONT MATTER

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

CAPSTONE PROJECT

Chapter 2

DESIGN AND IMPLEMENTATION OF AN INTELLIGENT SSH BRUTE-FORCE ATTACK DETECTION AND PREVENTION SYSTEM USING ISOLATION FOREST WITH EWMA-ADAPTIVE PERCENTILE DYNAMIC THRESHOLDING

Major: Information Assurance

Capstone Project Code:

Student Name:

Student ID:

Supervisor:

Hanoi, 2025

2.1 ABSTRACT

The Secure Shell (SSH) protocol is the predominant mechanism for remote administration of Linux and Unix server infrastructure worldwide. SSH brute-force attacks — in which adversaries systematically attempt large numbers of username–password combinations to gain unauthorized access — constitute one of the most persistent and prevalent threats to Internet-facing servers. Conventional defense tools such as Fail2Ban employ static, count-based thresholds (e.g., blocking an IP address after five failed login attempts within ten minutes) that are fundamentally unable to detect sophisticated attack variants, including low-and-slow brute-force attacks, distributed attacks that spread attempts across multiple source IP addresses, and credential stuffing attacks that leverage credentials stolen from unrelated breaches. This thesis presents the design, implementation, and comprehensive evaluation of an intelligent SSH brute-force attack detection and prevention system that leverages unsupervised machine learning, specifically the Isolation Forest algorithm, combined with a novel EWMA-Adaptive Percentile dynamic thresholding mechanism to enable early prediction of attacks before they reach full intensity.

The proposed system was evaluated using a hybrid dataset of 174,250 SSH log lines comprising real-world attack data collected from an SSH honeypot (119,729 log lines from 679 unique IP addresses across 5 days of operation) and simulated normal behavior data (54,521 log lines from 64 user accounts). A set of 14 behavioral features was engineered and extracted over 5-minute sliding windows per source IP address, capturing temporal, authentication, connection, and session characteristics. Three unsupervised anomaly detection algorithms were systematically trained, optimized, and compared: Isolation Forest (IF), Local Outlier Factor (LOF), and One-Class Support Vector Machine (OCSVM). After hyperparameter optimization (contamination = 0.01, max_features = 0.75, max_samples = 512, n_estimators = 500), the Isolation Forest model achieved an accuracy of 90.31%, an F1-score of 93.74%, a recall of 96.75%, and a false positive rate of 29.00%. The OCSVM model achieved the highest accuracy of 91.38% and F1-score of 94.55%, while LOF achieved perfect recall of 100% but exhibited the highest false positive rate of 67.10%. Feature importance analysis revealed that temporal features — session_duration_mean (5.50%), min_inter_attempt_time (3.86%), and mean_inter_attempt_time (2.61%) — dominate discrimination between automated attacks and normal activity, outperforming the count-based features traditionally relied upon by tools such as Fail2Ban.

The system was deployed as a complete end-to-end solution using a Docker-based microservices architecture comprising 9 services: FastAPI (API server and ML inference), React (monitoring dashboard), Elasticsearch (storage and indexing), Logstash (log normalization), Kibana (visualization), Fail2Ban (automated IP blocking), Redis (caching), PostgreSQL (configuration persistence), and Nginx (reverse proxy). The five-stage real-time detection pipeline achieves an end-to-end latency of under 2 seconds, with AI processing stages completing in under 100 milliseconds. The EWMA-Adaptive Percentile dynamic threshold (alpha = 0.3, base_percentile =

95, sensitivity_factor = 1.5) enables the system to adapt to evolving traffic patterns and provides early warning capabilities, detecting low-and-slow attacks within 3–5 attempts (approximately 2–3 minutes), significantly outperforming traditional static threshold methods. Five attack scenarios of escalating difficulty were designed and tested, demonstrating robust detection across basic brute-force, distributed, dictionary, credential stuffing, and low-and-slow attack types. The system was validated with 51 passing unit tests across all components and 5 integrated attack scenario evaluations.

Keywords: SSH brute-force detection, Isolation Forest, anomaly detection, dynamic threshold, EWMA, unsupervised machine learning, ELK Stack, intrusion detection system, cybersecurity, early prediction, Docker microservices, Fail2Ban

2.2 ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the completion of this thesis.

First and foremost, I extend my deepest thanks to my thesis supervisor for their invaluable guidance, rigorous feedback, and consistent encouragement throughout the entire research process. Their expertise in cybersecurity and machine learning has been instrumental in shaping the direction, methodology, and quality of this work.

I am profoundly grateful to the faculty members of the Information Assurance program at FPT University for providing a solid foundation in cybersecurity principles, software engineering, and research methodology. The knowledge and analytical skills I acquired during my studies were essential to the successful execution of this project.

I would also like to thank the members of the thesis examination committee for their time, careful reading, and constructive suggestions, which have strengthened the rigor and clarity of this research.

Special appreciation goes to my classmates and colleagues who provided technical assistance during the implementation phase, shared their experiences with related tools and frameworks, and offered moral support during the more challenging phases of the project. The collaborative and intellectually stimulating environment at FPT University has been a constant source of motivation.

I gratefully acknowledge the open-source community for developing and maintaining the tools and frameworks that formed the technical backbone of this research, including scikit-learn, FastAPI, the Elastic Stack, Docker, React, and Fail2Ban. Without these freely available, well-documented, and actively maintained projects, this work would not have been possible.

Finally, I owe my deepest gratitude to my family for their unconditional love, patience, and unwavering support throughout my entire academic journey. Their belief in my abilities has been a constant source of strength and inspiration, and this thesis is dedicated to them.

2.3 TABLE OF CONTENTS

ABSTRACT

ACKNOWLEDGEMENT

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

CHAPTER 1: INTRODUCTION …….. 1

1.1 Background and Motivation …….. 1

1.2 Problem Statement …….. 3

1.3 Research Questions …….. 5

1.4 Research Objectives …….. 6

1.5 Scope and Delimitations …….. 7

1.6 Thesis Contributions …….. 8

1.7 Thesis Organization …….. 9

CHAPTER 2: LITERATURE REVIEW …….. 11

2.1 The SSH Protocol …….. 11

2.1.1 Protocol Architecture and Layers …….. 11

2.1.2 Authentication Mechanisms …….. 13

2.1.3 SSH Brute-Force Attack Variants …….. 14

2.2 Traditional Detection and Prevention Methods …….. 16

2.2.1 Fail2Ban and Rate-Limiting Approaches …….. 16

2.2.2 Port Knocking and Other Hardening Techniques …….. 17

2.2.3 Limitations of Traditional Approaches …….. 18

2.3 Machine Learning for Intrusion Detection …….. 19

2.3.1 Supervised vs. Unsupervised Approaches …….. 19

2.3.2 Isolation Forest …….. 21

2.3.3 Local Outlier Factor …….. 23

2.3.4 One-Class Support Vector Machine …….. 24

2.4 Dynamic Thresholding Methods …….. 25

2.4.1 EWMA for Anomaly Detection …….. 25

2.4.2 Adaptive Percentile Methods …….. 26

2.5 ELK Stack for Security Monitoring …….. 27

2.6 Related Work …….. 29

2.7 Research Gap and Positioning …….. 32

CHAPTER 3: METHODOLOGY …….. 34

3.1 System Architecture …….. 34

3.1.1 Overall Architecture …….. 34

3.1.2 Docker-Based Microservices Deployment …….. 36

3.1.3 Data Flow Pipeline …….. 38

3.2 Data Collection and Preparation …….. 40

3.2.1 Honeypot Data Collection …….. 40

3.2.2 Normal Behavior Simulation …….. 42

3.2.3 Data Labeling and Splitting …….. 44

3.3 Feature Engineering …….. 45

3.3.1 Sliding Window Method …….. 45

3.3.2 Feature Definition …….. 47

3.3.3 Feature Normalization …….. 50

3.3.4 Model Selection and Configuration …….. 51

3.4 Dynamic Threshold Design …….. 53

3.4.1 EWMA-Adaptive Percentile Algorithm …….. 53

3.4.2 Parameter Selection …….. 55

3.4.3 Two-Level Detection Mechanism …….. 56

3.5 Real-Time Detection Pipeline …….. 57

3.6 Evaluation Methodology …….. 59

3.6.1 Evaluation Metrics …….. 59

3.6.2 Attack Scenario Design …….. 61

3.6.3 System Performance Evaluation …….. 62

CHAPTER 4: RESULTS AND ANALYSIS …….. 64

4.1 Dataset Analysis …….. 64

4.1.1 Overall Statistics …….. 64

4.1.2 Honeypot Data Analysis …….. 65

4.1.3 Simulation Data Analysis …….. 67

4.2 Feature Analysis …….. 68

4.2.1 Feature Distributions …….. 68

4.2.2 Correlation Analysis …….. 70

4.2.3 Feature Importance …….. 71

4.3 Model Performance …….. 73

4.3.1 Baseline Results …….. 73

4.3.2 Anomaly Score Distributions …….. 75

4.3.3 ROC and Precision-Recall Analysis …….. 76

4.3.4 Confusion Matrix Analysis …….. 77

4.3.5 Baseline Summary …….. 78

4.4 Interpretation of Results …….. 79

4.4.1 Optimized Model Performance …….. 79

4.4.2 Dynamic Threshold Evaluation …….. 81

4.4.3 Comparison with Related Studies …….. 83

4.5 Attack Scenario Evaluation …….. 84

4.5.1 Scenario Descriptions …….. 84

4.5.2 Detection Results …….. 86

4.5.3 Low-and-Slow Detection Analysis …….. 87

4.6 System Performance …….. 89

4.6.1 Latency Analysis …….. 89

4.6.2 Throughput Analysis …….. 90

4.6.3 Resource Utilization …….. 91

CHAPTER 5: DISCUSSION …….. 93

5.1 Restatement of Research Problem …….. 93

5.2 Achievement of Research Objectives …….. 95

5.2.1 Objective 1: Behavioral Feature Set Design …….. 95

5.2.2 Objective 2: Model Training and Evaluation …….. 96

5.2.3 Objective 3: Dynamic Threshold …….. 97

5.2.4 Objective 4: Integrated System …….. 98

5.2.5 Objective 5: Attack Scenario Evaluation …….. 99

5.3 Interpretation of Key Findings …….. 100

5.3.1 Why Timing Features Dominate …….. 100

5.3.2 Why Isolation Forest Is Chosen Despite OCSVM Having Higher F1 …….. 102

5.3.3 Dynamic Threshold vs. Static Threshold Advantage …….. 103

5.3.4 FPR Analysis and Acceptable Trade-off in Security Context …….. 105

5.4 Comparison with Existing Literature …….. 107

5.5 Practical Implications …….. 110

5.5.1 Deployment Considerations …….. 110

5.5.2 Scalability Considerations …….. 112

5.5.3 Cost-Benefit Analysis …….. 113

5.6 Limitations and Threats to Validity …….. 114

5.7 Chapter Summary …….. 116

CHAPTER 6: CONCLUSION AND FUTURE WORK …….. 117

6.1 Conclusion …….. 117

6.2 Contributions Summary …….. 120

6.3 Future Work …….. 122

6.3.1 Deep Learning for Improved Detection …….. 122

6.3.2 Online Learning and Continuous Adaptation …….. 123

6.3.3 Multi-Protocol Extension …….. 124

6.3.4 Federated Detection …….. 125

6.3.5 Threat Intelligence Integration …….. 126

6.3.6 SOAR Integration …….. 127

6.3.7 Explainability and Interpretability …….. 128

6.3.8 Long-Term IP Profiling …….. 129

6.3.9 Real-World Deployment Validation …….. 130

REFERENCES …….. 131

APPENDICES …….. 137

Appendix A: Feature Extraction Code Snippets …….. 137

Appendix B: Docker Compose Configuration …….. 139

Appendix C: Isolation Forest Hyperparameter Tuning Results …….. 141

Appendix D: Attack Simulation Scripts …….. 143

Appendix E: Logstash Pipeline Configuration …….. 145

Appendix F: Dynamic Threshold Parameter Sensitivity Analysis …….. 147

2.4 LIST OF FIGURES

- Figure 1.1: Global trends in SSH brute-force attacks (2018–2025)
- Figure 1.2: Comparison of traditional vs. AI-based attack detection approaches
- Figure 1.3: Overview of the proposed system architecture (block diagram)
- Figure 2.1: Layered architecture of the SSH-2 protocol
- Figure 2.2: Illustration of static threshold vs. dynamic threshold (EWMA, Adaptive Percentile, and hybrid) on the same anomaly score series
- Figure 2.3: Architecture for integrating AI models with the ELK Stack for SSH monitoring
- Figure 2.4: Venn diagram showing the research positioning at the intersection of three domains
- Figure 3.1: Overall system architecture for SSH brute-force attack detection
- Figure 3.2: Data flow pipeline of the system
- Figure 3.3: Illustration of the sliding window method (window = 5 min, stride = 1 min)
- Figure 3.4: Model training and evaluation workflow
- Figure 3.5: Illustration of the EWMA-Adaptive Percentile dynamic threshold algorithm

- Figure 3.6: Five-stage real-time detection pipeline
- Figure 4.1: Distribution of failed authentication attempts per IP (honeypot)
- Figure 4.2: Comparison of hourly activity distribution between honeypot (attack) and simulation (normal)
- Figure 4.3: Boxplot comparison of key feature distributions between Normal and Attack classes
- Figure 4.4: Histogram distribution of 14 features, separated by normal/attack labels
- Figure 4.5: Pearson correlation matrix heatmap of 14 features
- Figure 4.6: Bar chart of feature importance for 14 features (permutation importance on Isolation Forest)
- Figure 4.7: Anomaly score distribution on the test set for three models (IF, LOF, OCSVM)
- Figure 4.8: Violin plot comparing anomaly score distributions between normal and attack for three models
- Figure 4.9: Radar chart comparing five performance metrics across three models
- Figure 4.10: ROC curves for three models on the test set
- Figure 4.11: Precision-Recall curves for three models on the test set
- Figure 4.12: Anomaly score and dynamic threshold evolution over time on test data
- Figure 4.13: Effect of alpha, base_percentile, and sensitivity_factor on F1-Score (parameter sensitivity analysis)
- Figure 4.14: Heatmap of detection rates by attack scenario and model
- Figure 5.1: Comparative positioning of the proposed system against existing studies

2.5 LIST OF TABLES

- Table 1.1: Summary of research challenges and proposed solutions
- Table 1.2: Research objectives and corresponding approaches
- Table 2.1: Comparison of characteristics of SSH brute-force attack variants
- Table 2.2: Comparison of advantages and disadvantages of traditional detection methods
- Table 2.3: Comparison of Isolation Forest, LOF, and One-Class SVM characteristics
- Table 2.4: Comparative summary of related research works
- Table 3.1: List of 9 Docker services in the system with ports and roles
- Table 3.2: Summary of two data sources (honeypot and simulation)
- Table 3.3: Dataset split summary (training, testing, normal, attack)
- Table 3.4: Summary of 14 behavioral features and their significance
- Table 3.5: Comparison of characteristics of three anomaly detection models
- Table 3.6: Dynamic threshold algorithm parameters and their meanings
- Table 3.7: Summary of evaluation metrics and their security significance
- Table 4.1: Overall dataset statistics (log lines, samples, labels)
- Table 4.2: Detailed honeypot data statistics (IPs, events, time range)
- Table 4.3: Detailed simulation data statistics (users, events, session types)
- Table 4.4: Descriptive statistics of key features by label (normal vs. attack)

- Table 4.5: Feature importance ranking (Top 5 features by permutation importance)
- Table 4.6: Performance comparison of three models on the test set (baseline configuration)
- Table 4.7: Confusion matrix estimates for three models (TP, TN, FP, FN)
- Table 4.8: Comparison with related studies (method, dataset, metrics)
- Table 4.9: Dynamic threshold vs. static threshold comparison (OCSVM model)
- Table 4.10: Description of 5 attack scenarios (type, speed, IP count, difficulty)
- Table 4.11: Low-and-Slow detection results by model (detection rate, time to first alert)
- Table 4.12: Summary of detection results across 5 attack scenarios (all models)
- Table 4.13: Test environment configuration (hardware, software, versions)
- Table 4.14: Latency analysis by pipeline stage (ingestion, aggregation, scoring, decision, action)
- Table 4.15: Processing throughput by model (events/second, scoring latency)
- Table 4.16: Resource utilization of main Docker services (CPU, RAM, disk)
- Table 5.1: Optimized model performance comparison (IF, LOF, OCSVM after tuning)
- Table 5.2: Comprehensive comparison with related works (12 studies)

2.6 ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
BST	Binary Search Tree
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DNS	Domain Name System
ELK	Elasticsearch, Logstash, Kibana
EWMA	Exponentially Weighted Moving Average
FN	False Negative
FP	False Positive
FPR	False Positive Rate
FTP	File Transfer Protocol
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System

Abbreviation	Full Form
IF	Isolation Forest
IETF	Internet Engineering Task Force
IoC	Indicator of Compromise
IoT	Internet of Things
IP	Internet Protocol
IQR	Interquartile Range
ISO	International Organization for Standardization
LOF	Local Outlier Factor
LSTM	Long Short-Term Memory
ML	Machine Learning
MTTD	Mean Time to Detection
MTTR	Mean Time to Respond
NCSC	National Cyber Security Center
NIDS	Network Intrusion Detection System
OCSVM	One-Class Support Vector Machine
PAM	Pluggable Authentication Module
RAM	Random Access Memory
RBF	Radial Basis Function
RDP	Remote Desktop Protocol
REST	Representational State Transfer
RFC	Request for Comments
ROC	Receiver Operating Characteristic
SCP	Secure Copy Protocol
SFTP	SSH File Transfer Protocol
SHAP	SHapley Additive exPlanations
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center
SSH	Secure Shell
STIX	Structured Threat Information eXpression
SVM	Support Vector Machine
TAXII	Trusted Automated eXchange of Intelligence Information
TCP	Transmission Control Protocol
TN	True Negative

Abbreviation	Full Form
TP	True Positive
TPR	True Positive Rate

Chapter 3

CHAPTER 1: INTRODUCTION

3.1 1.1 Background

The Secure Shell (SSH) protocol, standardized through RFC 4251 by Ylonen and Lonvick [1], remains the predominant mechanism for remote administration of server infrastructure worldwide. Originally conceived as a secure replacement for plaintext protocols such as Telnet and rsh, SSH has evolved into the de facto standard for encrypted remote access, file transfer, and tunneling across enterprise, academic, and government networks [2]. The protocol's ubiquity, however, has rendered it one of the most targeted attack surfaces in the contemporary threat landscape. Wu et al. [3] analyzed data from the National Center for Supercomputing Applications (NCSA) honeypot infrastructure and documented approximately 11 billion SSH brute-force login attempts over a multi-year observation period, underscoring the industrial scale at which these attacks are conducted.

The magnitude of the SSH brute-force problem is further corroborated by industry threat intelligence. The Verizon Data Breach Investigations Report (DBIR) [4] consistently identifies stolen credentials as the single most common attack vector in confirmed data breaches, accounting for approximately 29% of all breaches analyzed. Because SSH password authentication provides a direct path to credential theft through systematic enumeration, SSH brute-force attacks constitute a substantial proportion of this attack category. Hellemons et al. [5] demonstrated through their SSHCure three-phase model—comprising scanning, brute-force, and compromise phases—that SSH attacks follow predictable temporal patterns that, in principle, are amenable to automated detection if sufficiently discriminative features are extracted from authentication logs.

Traditional countermeasures against SSH brute-force attacks have relied predominantly on static, rule-based mechanisms. Fail2Ban [6], the most widely deployed open-source intrusion prevention tool for SSH, monitors authentication log files and blocks offending IP addresses via firewall rules when the number of failed login attempts exceeds a preconfigured threshold (typically 5 failures within a 600-second window). While effective against naive, high-speed attacks, this approach exhibits well-documented limitations against sophisticated

adversaries who employ slow-rate, distributed, or credential-stuffing strategies [7]. The fundamental inadequacy of static thresholds in the face of adaptive adversaries has motivated a growing body of research into machine-learning-based detection approaches.

The application of machine learning to network intrusion detection has been the subject of extensive investigation over the past two decades. Chandola, Banerjee, and Kumar [7] provided a seminal survey of anomaly detection techniques, establishing a taxonomy that distinguishes among classification-based, nearest-neighbor-based, clustering-based, and isolation-based approaches. Within this taxonomy, unsupervised and semi-supervised methods hold particular appeal for intrusion detection because they do not require labeled attack data for training—a significant practical advantage given the difficulty and expense of obtaining comprehensive, accurately labeled attack datasets in operational environments [12]. Buczak and Guven [12] surveyed data mining and machine learning methods for cybersecurity and concluded that anomaly-detection-based approaches offer superior generalization to novel attack variants compared to signature-based methods, albeit at the cost of elevated false positive rates.

Among unsupervised anomaly detection algorithms, Isolation Forest (IF), proposed by Liu, Ting, and Zhou [8, 9], has attracted particular attention for network security applications due to its linear time complexity, absence of distributional assumptions, and native suitability for high-dimensional data. Unlike density-based or distance-based methods, Isolation Forest detects anomalies through the principle of isolation: anomalous data points, by virtue of their atypical feature values, require fewer random partitions to be separated from the remainder of the dataset. This computational paradigm yields anomaly scores that are both interpretable and efficiently computable, properties that are essential for real-time intrusion detection systems. Complementary algorithms including Local Outlier Factor (LOF) [10] and One-Class Support Vector Machine (OCSVM) [11] provide alternative detection paradigms based on local density estimation and maximum-margin separation, respectively, enabling rigorous comparative evaluation.

The convergence of three developments—the escalating scale and sophistication of SSH brute-force attacks [3, 4], the maturation of efficient anomaly detection algorithms [8, 9, 10, 11], and the availability of scalable log analytics infrastructure such as the ELK Stack (Elasticsearch, Logstash, Kibana)—creates a compelling opportunity for the development of intelligent, adaptive SSH security systems. This thesis capitalizes on this convergence by proposing a semi-supervised anomaly detection system that combines Isolation Forest with a novel EWMA-Adaptive Percentile dynamic thresholding mechanism [13, 14], trained exclusively on normal SSH behavior and evaluated against real-world attack data collected from a production honeypot. The system is fully integrated with ELK Stack for log ingestion and visualization, Fail2Ban for automated response, and Docker for containerized deployment, thereby bridging the persistent gap between research prototypes and production-ready security tools [12, 15].

The growing reliance on cloud computing and Infrastructure-as-a-Service (IaaS) models has further amplified the urgency of SSH security research. All major cloud providers use SSH as the default protocol for initial

access to virtual machine instances, and the proliferation of containerized microservice architectures has multiplied the number of SSH-accessible endpoints within typical enterprise environments [1, 2]. The financial consequences of successful SSH compromises extend far beyond the immediate cost of the intrusion: once an attacker gains SSH access, the compromise typically escalates through lateral movement, data exfiltration, ransomware deployment, or the installation of cryptocurrency miners [4]. The Verizon DBIR [4] estimates that compromised credentials are the most common initial attack vector, responsible for the largest share of all confirmed breaches. These cascading consequences underscore the need for proactive, predictive defense mechanisms that can identify attack intent before credential compromise occurs.

The dataset employed in this research comprises 174,250 SSH log lines: 119,729 lines of real attack traffic collected from a Virtual Private Server (VPS) honeypot (encompassing 679 unique attacking IP addresses, 29,301 failed password attempts, and 532 accepted root logins) and 54,521 lines of simulated normal behavior (64 user accounts performing routine SSH operations). This combination of authentic attack data and controlled normal-behavior data provides a realistic foundation for model training and evaluation that is absent from studies relying exclusively on synthetic benchmark datasets [24]. Ring et al. [24] surveyed 34 datasets for network-based intrusion detection and found that many widely used benchmarks do not adequately represent modern attack characteristics, recommending the use of application-specific data collected from controlled environments or honeypots. The system extracts 14 behavioral features per IP address per 5-minute sliding window and applies the Isolation Forest algorithm with dynamic thresholding to achieve an F1-score of 93.74% and an accuracy of 90.31%, with comparative benchmarks from LOF (F1 = 89.94%) and OCSVM (F1 = 94.55%) providing context for the primary model's performance.

3.2 1.2 Problem Statement

Despite decades of research and the deployment of numerous countermeasures, SSH brute-force attacks remain a persistent and growing threat to networked systems. Analysis of the literature and empirical observation of real-world attack traffic reveal four critical gaps in the current state of SSH brute-force defense that this thesis seeks to address.

Gap 1: Static thresholds cannot adapt to dynamic traffic patterns. Traditional detection tools, exemplified by Fail2Ban [6], rely on fixed thresholds—for example, blocking an IP address after a predetermined number of failed login attempts within a fixed time window. This approach creates an inherent dilemma: thresholds set too low generate excessive false positives during periods of legitimate high-activity (e.g., system maintenance windows, shift changes), while thresholds set too high permit sophisticated slow-rate attacks to proceed undetected [7, 12]. SSH traffic exhibits pronounced temporal variability—business hours versus off-hours, weekdays versus weekends—that static thresholds are structurally incapable of accommodating. Lucas and Saccucci [13] demonstrated in the context of statistical process control that exponentially weighted moving average (EWMA) schemes provide superior sensitivity to small, persistent shifts in process mean compared to fixed-limit schemes,

a finding directly applicable to the anomaly-score time series generated by SSH monitoring systems.

Gap 2: Absence of predictive, early-warning capability. The vast majority of deployed SSH defense systems operate in a purely reactive mode: they detect and respond to attacks only after a sufficient volume of malicious activity has already occurred [5, 7]. Hellemons et al. [5] documented the three-phase structure of SSH attacks (scanning, brute-force, compromise) and noted that the scanning and early brute-force phases exhibit distinctive behavioral signatures that could, in principle, enable detection before the attack reaches its critical phase. However, existing systems rarely exploit these early-phase signatures. Javed and Paxson [23] studied stealthy SSH brute-forcing campaigns that unfold over periods of days to weeks, demonstrating that attacks operating below conventional detection thresholds can achieve compromise rates comparable to aggressive campaigns while remaining invisible to standard monitoring tools. The absence of early-warning capability means that defenders forfeit the opportunity to intervene during the reconnaissance phase, when the cost of mitigation is lowest.

Gap 3: Insufficient adaptation to evolving attack behaviors. Modern SSH brute-force attacks have evolved far beyond simple high-speed password enumeration. Bezerra et al. [18] analyzed SSH attack durations and identified significant variability in attack temporal profiles, ranging from sub-second bursts to campaigns spanning multiple days. Owens and Matthews [19] demonstrated that credential-stuffing attacks—which exploit username-password pairs leaked from prior data breaches—exhibit fundamentally different behavioral signatures than traditional dictionary attacks, including higher per-attempt success rates and lower attempt volumes. Park et al. [22] documented SSH attacks specifically targeting network infrastructure devices (routers, switches), which exhibit distinct authentication patterns compared to attacks against general-purpose servers. Traditional rule-based systems, designed around a single attack archetype, lack the flexibility to accommodate this diversity [7, 12]. Machine learning approaches that construct behavioral profiles from multiple feature dimensions offer a principled mechanism for detecting diverse attack variants without requiring explicit rules for each [8, 9].

Gap 4: Disconnect between algorithmic research and deployable systems. A substantial body of research has demonstrated the effectiveness of machine learning algorithms for SSH-related anomaly detection [20, 21, 25]. However, the overwhelming majority of these studies evaluate algorithms in isolation on benchmark datasets without addressing the engineering challenges of integration into operational infrastructure—log ingestion pipelines, real-time feature extraction, automated response mechanisms, visualization dashboards, and containerized deployment [12, 15]. Sperotto et al. [15] noted that the transition from research prototype to production system remains one of the most significant barriers to the adoption of machine-learning-based intrusion detection. This research-to-deployment gap means that organizations cannot readily benefit from algorithmic advances, and the practical impact of the research remains limited.

3.2.1 Research Questions

In light of the four gaps identified above, this thesis is guided by the following research questions:

RQ1: How effectively does the Isolation Forest algorithm detect SSH brute-force attacks in a semi-supervised setting (trained exclusively on normal behavior data), and how does its performance compare to Local Outlier Factor (LOF) and One-Class Support Vector Machine (OCSVM) on the same real-world honeypot dataset?

RQ2: To what extent does the proposed EWMA-Adaptive Percentile dynamic thresholding mechanism improve detection performance—measured by accuracy, F1-score, and false positive rate—compared to static threshold approaches across varying traffic conditions?

RQ3: What degree of early prediction capability does the system achieve across five distinct attack scenarios (basic brute-force, distributed attack, slow brute-force, credential stuffing, and dictionary attack), and which behavioral features contribute most to early detection?

RQ4: Can the integrated system architecture—combining ELK Stack for log analytics, Isolation Forest for anomaly detection, Fail2Ban for automated response, and Docker for containerized deployment—meet the requirements for real-time monitoring and automated response in a practical operational environment?

3.3 1.3 Research Objectives

This research aims to design, implement, and rigorously evaluate an intelligent SSH brute-force attack detection and prevention system that integrates semi-supervised anomaly detection, adaptive dynamic thresholding, and automated response within a modern, containerized infrastructure. The following five specific objectives operationalize the research questions stated above.

Objective 1: Design a comprehensive SSH behavioral feature set. Develop and validate a set of 14 behavioral features extracted from SSH authentication logs over 5-minute sliding windows per source IP address [8, 9]. The feature set encompasses four categories: frequency features (attempt counts, failure rates), temporal features (inter-attempt timing, session duration), authentication features (unique usernames, password diversity), and connection features (port entropy, geographic indicators). The feature set must capture sufficient behavioral information to discriminate between normal SSH activity and the five attack variants under study.

Objective 2: Train and comparatively evaluate semi-supervised anomaly detection models. Implement and systematically compare three anomaly detection algorithms—Isolation Forest [8, 9], Local Outlier Factor [10], and One-Class SVM [11]—in a semi-supervised configuration where models are trained exclusively on normal-behavior data. Establish performance benchmarks using accuracy, precision, recall, F1-score, and false positive rate (FPR), with target performance thresholds of F1 >= 85% and recall >= 95% to ensure high detection rates with acceptable false alarm levels [20, 21].

Objective 3: Develop and validate an EWMA-Adaptive Percentile dynamic thresholding mechanism. Design a hybrid dynamic threshold that combines the trend-tracking properties of Exponentially Weighted Moving Average (EWMA) [13] with the distribution-free adaptiveness of percentile-based methods [14]. The mechanism must automatically adjust the anomaly detection threshold in response to temporal variations in SSH

traffic patterns, reducing false positive rates during high-activity periods while maintaining sensitivity during low-activity periods.

Objective 4: Build a complete, containerized, end-to-end system. Design and implement an integrated system architecture comprising nine Docker services: ELK Stack (Elasticsearch, Logstash, Kibana) for log ingestion, storage, and visualization; a FastAPI backend for model serving and API endpoints; a React frontend for security monitoring dashboards; and Fail2Ban for automated IP blocking upon attack detection [6, 15]. The architecture must support real-time processing of SSH authentication events with end-to-end latency suitable for operational deployment.

Objective 5: Evaluate system performance against five realistic attack scenarios. Design and execute five simulated SSH brute-force attack scenarios—basic brute-force, distributed attack, slow brute-force, credential stuffing, and dictionary attack—that collectively test the system's detection capability, early prediction ability, and automated response across the full spectrum of modern attack techniques [5, 18, 19, 23]. Each scenario targets a specific system capability: basic and dictionary scenarios test standard detection, distributed and credential-stuffing scenarios test resilience to per-IP evasion, and slow brute-force tests early-warning effectiveness.

[Table 1.1: Mapping of research questions to objectives and evaluation criteria]

3.4 1.4 Significance of the Study

3.4.1 1.4.1 Practical Significance

The system developed in this research provides a directly deployable solution for organizations seeking to enhance their SSH security posture beyond the capabilities of traditional tools. The Docker-based containerized architecture enables deployment on any Linux server with minimal configuration, while the use of exclusively open-source components (scikit-learn, FastAPI, Elasticsearch, Kibana, Fail2Ban) eliminates licensing costs that would otherwise be associated with commercial Security Information and Event Management (SIEM) solutions [6, 15]. The system's real-time monitoring capability, delivered through both Kibana dashboards and a custom React-based interface, provides security administrators with actionable visibility into SSH authentication patterns. Integration with Fail2Ban ensures that detected attacks trigger automated IP blocking, reducing mean time to respond (MTTR) from minutes (manual intervention) to seconds (automated response).

The early-warning capability represents a particularly significant practical advance. Rather than waiting for an attacker to execute hundreds or thousands of login attempts before triggering a static threshold, the system can identify attack intent from behavioral features extracted during the initial probing phase [5, 23]. In the slow brute-force scenario, the system issues an EARLY_WARNING alert after only 3–5 attempts (approximately 2–3 minutes into the attack), whereas Fail2Ban with default settings would fail to detect the attack entirely if the attacker maintains a sufficiently low attempt rate. This proactive detection capability provides defenders

with a critical time advantage for investigation and response.

3.4.2 1.4.2 Research Significance

This thesis contributes to the academic literature in three distinct areas. First, it provides a rigorous comparative evaluation of three anomaly detection algorithms (IF, LOF, OCSVM) on real-world SSH honeypot data in a semi-supervised configuration [8, 9, 10, 11]. While these algorithms have been extensively studied on benchmark datasets such as NSL-KDD and CICIDS2017, their comparative performance on authentic SSH attack data—with its characteristic noise, class imbalance, and behavioral diversity—remains underexplored [24, 25]. Second, the proposed EWMA-Adaptive Percentile hybrid thresholding mechanism [13, 14] represents a novel contribution to the literature on dynamic thresholds for anomaly detection, combining the complementary strengths of trend-following (EWMA) and distribution-free (percentile) approaches. Third, the finding that temporal features—specifically session_duration_mean (importance: 5.50%), min_inter_attempt_time (3.86%), and mean_inter_attempt_time (2.61%)—outperform traditional frequency-based features (e.g., fail_count) as discriminators between normal and attack behavior has implications for the design of future SSH monitoring systems [8, 15].

3.4.3 1.4.3 Empirical Significance

The dataset constructed for this research—174,250 log lines combining 119,729 lines of real honeypot attack data (679 IPs, 29,301 failed passwords, 532 accepted root logins) with 54,521 lines of simulated normal behavior (64 users)—provides an empirical foundation that is both more realistic and more comprehensive than the benchmark datasets commonly used in the literature [24]. The honeypot data captures the full behavioral diversity of real-world SSH attackers, including geographic distribution, temporal patterns, username preferences, and attack-speed profiles, while the simulated normal data provides controlled ground truth for model training. The 14 behavioral features extracted per IP per 5-minute window, the feature importance rankings, and the comparative algorithm performance metrics constitute empirical contributions that can inform and guide future research in this domain [20, 21].

3.5 1.5 Scope and Limitations

3.5.1 1.5.1 Scope

The scope of this research is defined along four dimensions. **Protocol and attack type:** The study focuses exclusively on SSH version 2 (SSH-2) and password-based brute-force attacks in five variants: basic brute-force, dictionary attack, slow brute-force, distributed attack, and credential stuffing [1, 2]. Other SSH-based attacks (man-in-the-middle, session hijacking, protocol-level exploits) fall outside the scope. **Data:** The research employs two data sources: (i) real attack data collected from an SSH honeypot deployed on a production VPS,

comprising 119,729 log lines from 679 unique IP addresses with 29,301 failed password attempts and 532 accepted root logins; and (ii) simulated normal-behavior data comprising 54,521 log lines from 64 user accounts performing routine SSH operations. The combined dataset totals 174,250 log lines [3, 24]. **Algorithms:** Three semi-supervised anomaly detection algorithms are implemented and evaluated: Isolation Forest (primary) [8, 9], LOF (benchmark) [10], and OCSVM (benchmark) [11]. Supervised and deep learning approaches are excluded by design, as the semi-supervised paradigm addresses the practical constraint of unavailable labeled attack data. **Infrastructure:** The system is built on ELK Stack (Elasticsearch 8.x, Logstash, Kibana), FastAPI, React, Fail2Ban, and Docker Compose (9 services), deployed on a Linux-based server environment.

3.5.2 1.5.2 Limitations

Four limitations must be acknowledged. First, the normal-behavior data was generated through simulation rather than collected from a production environment; while the simulation covers a broad range of legitimate SSH usage patterns (64 user profiles), certain environment-specific behaviors may be underrepresented [24]. Second, the system has been evaluated at moderate scale; scalability to environments processing authentication events from hundreds of simultaneous SSH servers has not been empirically verified, although the ELK Stack architecture supports horizontal scaling [15]. Third, the Isolation Forest model was trained on data from a specific VPS environment and may require retraining when deployed to environments with substantially different baseline SSH usage patterns [8, 9]. Fourth, the false positive rate of 29.00% for Isolation Forest, while acceptable for a semi-supervised system operating without labeled attack training data, indicates room for improvement through feature refinement, ensemble methods, or incorporation of limited labeled data in future work [20, 21]. Goldstein and Uchida [20] observed similar false positive rates for unsupervised methods across multiple benchmark datasets, suggesting that this is a characteristic limitation of the one-class learning paradigm rather than a deficiency specific to this implementation.

3.6 1.6 Thesis Structure

This thesis is organized into six chapters, each addressing a distinct phase of the research process:

Chapter 1: Introduction. Establishes the research context, articulates the four-gap problem statement, formulates four research questions (RQ1–RQ4), specifies five research objectives, and delineates the scope and limitations of the study.

Chapter 2: Literature Review. Provides a comprehensive synthesis of the theoretical foundations and related work, including: the SSH protocol and its security landscape [1, 2]; brute-force attack taxonomy and evolution [3, 4, 5]; traditional defense mechanisms and their limitations [6, 7]; anomaly detection paradigms [7, 12]; the mathematical foundations of Isolation Forest [8, 9], LOF [10], and OCSVM [11]; dynamic thresholding mechanisms [13, 14]; and a systematic comparison of related studies. The chapter concludes with the identification of research gaps and the articulation of this thesis's contributions.

Chapter 3: Methodology. Describes the research methodology, including system architecture design, data collection and preprocessing procedures, the 14-feature engineering pipeline, model configuration and training protocols, the EWMA-Adaptive Percentile dynamic threshold mechanism, and the five attack scenario simulation designs.

Chapter 4: Experimental Results. Presents the experimental outcomes, including dataset statistics, model training and hyperparameter optimization results, comparative algorithm performance, dynamic threshold evaluation, feature importance analysis, and attack scenario test results.

Chapter 5: Discussion. Interprets the experimental results in the context of the research questions, compares findings with related work, analyzes the significance of temporal features as top discriminators, and discusses the practical implications of the integrated system architecture.

Chapter 6: Conclusion and Future Work. Summarizes the principal contributions and findings, evaluates the degree to which each research objective was achieved, and proposes directions for future research including deep learning extensions, federated learning for multi-site deployment, and adversarial robustness evaluation.

[Figure 1.1: Global trends in SSH brute-force attacks and the evolution of defense mechanisms (2015–2025)]

[Figure 1.2: Conceptual comparison of static threshold versus dynamic threshold detection approaches]

[Figure 1.3: High-level architecture of the proposed system showing the nine Docker services and data flow]

[Table 1.1: Mapping of research questions to objectives and evaluation criteria]

[Table 1.2: Summary of research gaps and corresponding contributions of this thesis]

Chapter 4

CHAPTER 2: LITERATURE REVIEW

4.1 2.1 Secure Shell (SSH) Protocol and Its Security Landscape

The Secure Shell (SSH) protocol was originally developed in 1995 by Tatu Ylonen at the Helsinki University of Technology as a secure replacement for unencrypted remote access protocols including Telnet, rlogin, and rsh [1]. The protocol was subsequently standardized by the Internet Engineering Task Force (IETF) through the RFC 4250–4256 document series, with RFC 4251 defining the overall protocol architecture [1]. Barrett, Silverman, and Byrnes [2] provide a comprehensive treatment of SSH implementation and deployment, documenting the protocol's evolution from a research tool to the universal standard for secure remote system administration. As of 2024, SSH is deployed on virtually every Linux and Unix server connected to the Internet, and all major cloud infrastructure providers—Amazon Web Services, Google Cloud Platform, and Microsoft Azure—use SSH key pairs as the default mechanism for initial access to virtual machine instances [1, 2].

The SSH-2 protocol architecture is organized into three layered components, each addressing a distinct security function [1, 2]. The Transport Layer Protocol (RFC 4253) establishes server authentication, data confidentiality through symmetric encryption, and data integrity through message authentication codes. The handshake process involves protocol version exchange, algorithm negotiation, Diffie-Hellman key exchange, and server host key verification. The User Authentication Protocol (RFC 4252) handles client identity verification through one of several methods: password authentication, public key authentication, host-based authentication, or keyboard-interactive authentication. The Connection Protocol (RFC 4254) multiplexes multiple logical channels over a single encrypted connection, supporting remote command execution, port forwarding, and secure file transfer via SFTP/SCP.

Password-based authentication, despite being the least secure of the available methods, remains widely deployed due to its simplicity and the absence of prerequisite key management infrastructure [2, 4]. The default OpenSSH configuration permits up to six authentication attempts per connection (MaxAuthTries = 6) and im-

poses no limit on the number of concurrent connections from a single source IP address [1, 6]. This permissive default configuration, combined with the protocol's lack of any built-in anti-automation mechanism (unlike web applications that can employ CAPTCHAs), creates a structurally exploitable attack surface for automated brute-force tools [2, 5].

The scale of SSH-targeted attacks is documented by multiple empirical studies. Wu et al. [3] analyzed data from the National Center for Supercomputing Applications (NCSA) honeypot deployment and recorded approximately 11 billion SSH brute-force login attempts, demonstrating that SSH brute-forcing operates at an industrial scale driven by globally distributed botnets. The Verizon Data Breach Investigations Report [4] identifies stolen credentials as the most prevalent initial attack vector, accounting for approximately 29% of confirmed data breaches, with brute-force and credential-stuffing attacks representing the primary credential-theft mechanisms. Hellemons et al. [5] developed the SSHCure system and characterized SSH attacks as following a three-phase temporal structure: an initial scanning phase in which the attacker probes target reachability, a brute-force phase involving systematic credential enumeration, and a compromise phase in which successful credentials are exploited for lateral movement or data exfiltration.

SSH servers record authentication events in system log files (typically `/var/log/auth.log` on Debian-based systems or `/var/log/secure` on Red Hat-based systems), with each event containing a timestamp, source IP address, attempted username, authentication result (success or failure), authentication method, and source port [1, 5]. These structured log fields provide the raw data from which behavioral features can be extracted for machine-learning-based anomaly detection, a capability that this thesis exploits through a 14-feature extraction pipeline operating on 5-minute sliding windows per source IP address.

[Figure 2.1: Layered architecture of the SSH-2 protocol and authentication flow]

4.2 2.2 Brute-Force Attacks: Taxonomy, Patterns, and Evolution

Brute-force attacks against SSH servers can be formally defined as systematic attempts to discover valid authentication credentials through exhaustive or guided enumeration of the credential space [3, 4]. Given a character alphabet of size $|A|$ and a target password of length L, the theoretical maximum search space is $S = |A|^L$. For a password composed of lowercase letters (26) and digits (10) with length 8, this yields $S = 36^8 \approx 2.82 \times 10^{12}$ combinations. In practice, however, attackers rarely perform exhaustive enumeration; instead, they exploit the highly non-uniform distribution of human-chosen passwords to reduce the effective search space by orders of magnitude [4, 19].

The taxonomy of SSH brute-force attacks has evolved substantially over the past decade, driven by the arms race between attackers and defenders. Based on the literature [3, 5, 18, 19, 23] and empirical analysis of the honeypot data collected for this research (679 unique attacking IPs, 29,301 failed password attempts), five principal attack variants can be identified:

Classic brute-force. The attacker attempts credential combinations at maximum speed, typically targeting high-value accounts such as root, admin, and user [3, 5]. Characteristic behavioral signatures include extremely high login attempt frequency (hundreds to thousands per minute), near-uniform inter-attempt intervals (reflecting automated tool cadence), and rapid cycling through multiple usernames. Wu et al. [3] found that the root account is targeted in over 80% of SSH brute-force sessions observed in their honeypot data.

Dictionary attack. This variant employs curated password lists (wordlists) rather than exhaustive enumeration, exploiting the observation that a small number of passwords account for a disproportionate fraction of real-world credential choices [19]. Owens and Matthews [19] analyzed SSH honeypot data and found that the top 20 most-attempted passwords accounted for over 40% of all login attempts, with "123456", "password", and "admin" consistently appearing in the top positions. Widely used wordlists include RockYou (approximately 14 million entries) and the SecLists collection, and attackers increasingly generate target-specific lists incorporating organizational names, service identifiers, and locale-specific terms.

Slow brute-force (low-and-slow attack). The attacker deliberately reduces the attempt rate to evade frequency-based detection thresholds, executing only a few attempts per minute or per hour [23]. Javed and Paxson [23] conducted a systematic study of stealthy SSH brute-forcing and demonstrated that attacks operating at rates as low as 1–2 attempts per hour can achieve compromise rates comparable to aggressive campaigns while remaining invisible to conventional monitoring tools configured with standard thresholds. This attack variant is particularly insidious because its behavioral profile closely resembles that of a legitimate user who has forgotten their password [5, 7].

Distributed brute-force. This variant leverages botnets or proxy networks to distribute the attack across hundreds or thousands of source IP addresses, with each individual IP performing only a small number of attempts [3, 5]. Per-IP failure-count-based detection methods are structurally incapable of identifying distributed attacks because no single IP exceeds the detection threshold. Wu et al. [3] documented coordinated distributed attacks originating from geographically diverse IP addresses that collectively executed millions of attempts while maintaining per-IP attempt counts below typical Fail2Ban thresholds.

Credential stuffing. This variant exploits username-password pairs leaked from data breaches at other services, capitalizing on the widespread practice of password reuse across platforms [4, 19]. Credential-stuffing attacks are particularly dangerous because the per-attempt success rate is substantially higher than that of random or dictionary-based attacks. The Verizon DBIR [4] notes that credential-stuffing attacks have increased significantly following major data breaches, with automated tools such as Sentry MBA and OpenBullet enabling attackers to test millions of leaked credentials against SSH servers.

Bezerra et al. [18] analyzed SSH attack durations across a large-scale honeypot dataset and identified significant variability in temporal profiles: attack sessions ranged from sub-second bursts (automated scanners testing a single credential) to sustained campaigns spanning multiple days (coordinated, multi-phase attacks). This temporal diversity imposes a requirement for detection systems to operate effectively across multiple time scales—a

requirement that motivates the 5-minute sliding window approach adopted in this thesis.

[Table 2.1: Taxonomy of SSH brute-force attack variants with characteristic behavioral features]

Variant	Attempt Rate	IP Diversity	Password Strategy	Detection Difficulty
Classic brute-force	Very high (>100/min)	Single IP	Exhaustive or top-N	Low
Dictionary attack	High (10–100/min)	Single IP	Wordlist-based	Low–Medium
Slow brute-force	Very low (<1/min)	Single IP	Targeted	High
Distributed attack	Low per IP	Many IPs (>10)	Coordinated	High
Credential stuffing	Medium	Variable	Leaked credentials	Very High

4.3 2.3 Traditional Defense Mechanisms and Their Limitations

Traditional SSH brute-force defense mechanisms can be categorized into four classes, each with characteristic strengths and limitations [6, 7, 12].

Threshold-based intrusion prevention. Fail2Ban [6] is the most widely deployed tool in this category. It monitors SSH authentication logs and applies firewall rules (via iptables or nftables) to block IP addresses that exceed a configured failure threshold within a specified time window. The default SSH jail configuration uses maxretry = 5, findtime = 600 seconds, and bantime = 600 seconds. While effective against naive high-speed attacks, Fail2Ban's static threshold architecture creates an inherent precision-sensitivity tradeoff: lowering the threshold increases false positives (blocking legitimate users who mistype passwords), while raising it increases false negatives (permitting slow or distributed attacks) [6, 7]. Chandola et al. [7] identified this static-threshold limitation as a fundamental challenge in anomaly detection, noting that real-world data streams exhibit non-stationary behavior that fixed thresholds cannot accommodate.

IP reputation and blacklisting. Services such as AbuseIPDB, Spamhaus, and Blocklist.de maintain databases of IP addresses associated with malicious activity [4, 6]. Blacklist-based approaches are inherently reactive—an IP is listed only after it has been observed conducting attacks elsewhere—and are easily circumvented by attackers who rotate through fresh IP addresses or employ residential proxy networks [3, 7]. Furthermore, blacklists suffer from both false positives (legitimate services operating behind shared NAT addresses that have been erroneously listed) and incomplete coverage (newly provisioned attack infrastructure that has not yet been reported).

Signature-based intrusion detection. Network intrusion detection systems such as Snort and Suricata can identify SSH brute-force attacks by matching traffic patterns against predefined rule signatures [7, 15]. However, signature-based detection is structurally limited to known attack patterns for which rules have been written and cannot detect novel attack variants [12]. Moreover, because SSH traffic is encrypted end-to-end, deep packet inspection of SSH session content is infeasible; detection must rely on metadata features (connection frequency, flow duration, packet sizes) rather than payload analysis [15].

Authentication hardening. Measures such as public key authentication, two-factor authentication (via PAM modules), non-standard port assignment, and port knocking reduce the attack surface by either eliminating password authentication entirely or raising the barrier to initial connection establishment [1, 2]. While highly effective when deployed, these measures are not universally applicable—legacy systems, multi-user environments with diverse client configurations, and cloud instances requiring initial password-based bootstrapping all present practical obstacles to full authentication hardening [2, 6].

The limitations of these traditional mechanisms motivate the application of machine learning approaches that can learn complex behavioral patterns from data, adapt to changing traffic conditions, and detect novel attack variants without requiring explicit rule definitions [7, 12]. Buczak and Guven [12] concluded their comprehensive survey by noting that the integration of machine learning with traditional security infrastructure represents the most promising direction for next-generation intrusion detection systems.

[Table 2.2: Comparison of traditional SSH defense mechanisms]

Method	Strengths	Limitations	Evasion Techniques
Fail2Ban (threshold)	Simple, low overhead	Static threshold, no adaptation	Slow-rate, distributed
IP blacklisting	No local computation	Reactive, incomplete coverage	IP rotation, proxies
Signature IDS	Known-attack detection	Cannot detect novel attacks	Encryption, polymorphism
Auth hardening	Eliminates attack surface	Not universally deployable	N/A (preventive)

4.4 2.4 Anomaly Detection for Intrusion Detection Systems

Anomaly detection, defined as the identification of data patterns that deviate significantly from expected normal behavior, constitutes one of the foundational paradigms in intrusion detection [7]. Chandola, Banerjee, and Kumar [7] provided a seminal taxonomy of anomaly detection approaches, categorizing them along two principal dimensions: the nature of the detection model (classification-based, nearest-neighbor-based, clustering-based,

statistical, information-theoretic, spectral, and isolation-based) and the availability of labeled data (supervised, semi-supervised, and unsupervised).

In the context of network intrusion detection, the semi-supervised anomaly detection paradigm—in which models are trained exclusively on data representing normal behavior and subsequently identify deviations from the learned normal profile as potential attacks—has attracted substantial research attention [7, 12, 17]. This paradigm offers three key advantages for SSH brute-force detection. First, it does not require labeled attack data for training, circumventing the practical challenge of obtaining comprehensive, accurately labeled datasets in operational environments [17, 24]. Pimentel et al. [17] provided a comprehensive review of novelty detection—the closely related problem of identifying previously unseen patterns—and noted that one-class learning approaches (trained on a single class of normal data) provide the most robust framework for detecting genuinely novel anomalies. Second, semi-supervised approaches can detect zero-day attacks and novel attack variants that have never been observed in training data, because detection is based on deviation from normality rather than similarity to known attacks [7, 12]. Third, in SSH environments, normal behavior data is readily available from production logs (by selecting periods of known-clean operation) or from controlled simulation, making the semi-supervised training paradigm practically feasible [15, 24].

The choice among specific anomaly detection algorithms involves tradeoffs along multiple dimensions: computational complexity (critical for real-time processing), sensitivity to hyperparameter selection, ability to handle high-dimensional feature spaces, robustness to noise and outliers in training data, and interpretability of anomaly scores [7, 20]. Goldstein and Uchida [20] conducted a comprehensive comparative evaluation of unsupervised anomaly detection algorithms on standardized benchmark datasets and found that no single algorithm dominates across all datasets and evaluation metrics, underscoring the importance of algorithm selection based on the specific characteristics of the target domain. Nassif et al. [21] extended this analysis to the cybersecurity domain specifically, surveying machine learning approaches for anomaly-based intrusion detection and identifying Isolation Forest, LOF, and OCSVM as the three most widely studied unsupervised algorithms for network security applications.

For this thesis, three algorithms were selected for implementation and comparative evaluation: Isolation Forest [8, 9] as the primary detection algorithm, and Local Outlier Factor [10] and One-Class SVM [11] as benchmarks. This selection is motivated by the complementary detection paradigms they represent (isolation-based, density-based, and boundary-based, respectively), their established track records in network intrusion detection research [20, 21], and their availability in mature, well-tested implementations within the scikit-learn library. The following three sections present the mathematical foundations of each algorithm.

4.5 2.5 Isolation Forest and Its Extensions

4.5.1 2.5.1 Algorithmic Foundation

Isolation Forest (IF) was proposed by Liu, Ting, and Zhou [8] at the IEEE International Conference on Data Mining (ICDM) in 2008 and subsequently formalized in a comprehensive journal publication in ACM Transactions on Knowledge Discovery from Data (TKDD) in 2012 [9]. The algorithm represents a paradigm shift in anomaly detection: whereas traditional methods detect anomalies as data points that are distant from (distance-based methods) or sparse relative to (density-based methods) the majority of the data, Isolation Forest detects anomalies as data points that are easy to isolate through random partitioning [8, 9].

The fundamental insight underlying Isolation Forest is that anomalous data points, by virtue of their atypical feature values, require fewer random partitions to be separated from the rest of the dataset than normal points [8]. This property arises because anomalies are typically few in number and possess feature values that are substantially different from those of the majority population, making them susceptible to isolation by a small number of random splits.

4.5.2 2.5.2 Isolation Tree Construction

An Isolation Tree (iTree) is constructed through a recursive random partitioning process [8, 9]. Given a dataset $X = \{x_1, x_2, \dots, x_n\}$ with d features, the construction proceeds as follows:

1. Randomly select a feature q from the d available features.
2. Randomly select a split value p uniformly from the interval $[\min(X_q), \max(X_q)]$, where X_q denotes the values of feature q in the current node's data.
3. Partition the data into two subsets: the left branch contains points where $x_q < p$, and the right branch contains points where $x_q \geq p$.
4. Recurse on each branch until one of the stopping conditions is met: the node contains a single data point (isolated), or the tree reaches a predefined maximum depth $l = \lceil \log_2 \psi \rceil$, where ψ is the sub-sampling size.

4.5.3 2.5.3 Anomaly Score Computation

The anomaly score of a data point x is derived from its average path length across an ensemble of t Isolation Trees [8, 9]. The path length $h(x)$ of a point x in a single iTree is defined as the number of edges traversed from the root node to the terminal node (external node) at which x is isolated, plus an adjustment factor $c(k)$ for the unbuilt subtree when the maximum depth is reached, where k is the number of data points in the terminal node.

The anomaly score is defined as:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}}$$

where $E[h(x)]$ is the expected (average) path length of x across t Isolation Trees, and $c(n)$ is the average path length of an unsuccessful search in a Binary Search Tree (BST) constructed from n data points, serving as a normalization factor [8, 9]. The normalization factor is computed as:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

where $H(i) = \ln(i) + \gamma$ is the harmonic number approximation and $\gamma \approx 0.5772$ is the Euler-Mascheroni constant [8].

The anomaly score has the following interpretation [8, 9]: - $s(x, n) \to 1$: the point has a short average path length, indicating high susceptibility to isolation and therefore high anomaly likelihood. - $s(x, n) \approx 0.5$: the average path length is close to the expected path length for the dataset, indicating no clear anomaly signal. - $s(x, n) \to 0$: the point has a long average path length, indicating deep embedding within the normal data distribution.

4.5.4 2.5.4 Computational Complexity and Practical Advantages

The time complexity of Isolation Forest is $O(t \cdot \psi \cdot \log \psi)$ for training and $O(t \cdot \log \psi)$ for scoring a single data point, where t is the number of trees and ψ is the sub-sampling size [8, 9]. This linear-logarithmic complexity represents a significant advantage over density-based methods (LOF: $O(n^2)$) and kernel-based methods (OCSVM: $O(n^2)$ to $O(n^3)$), making Isolation Forest particularly suitable for real-time applications where scoring latency is a critical constraint [8, 20].

Additional practical advantages include: (i) robustness to the swamping and masking effects that degrade the performance of distance-based and density-based methods when anomalies are numerous or clustered [9]; (ii) absence of distributional assumptions about the data, unlike statistical methods that assume Gaussian or other parametric distributions [8, 17]; (iii) effective handling of high-dimensional feature spaces without suffering from the curse of dimensionality that affects distance-based methods [9, 20]; and (iv) interpretability of the anomaly score, which has a natural probabilistic interpretation as the likelihood of isolation under random partitioning [8].

4.5.5 2.5.5 Extended Isolation Forest

Hariri, Kind, and Brunner [16] identified a limitation of the original Isolation Forest algorithm: because axis-parallel splits are used exclusively, the algorithm can produce artifacts in the anomaly score landscape, assigning anomalous scores to normal points that lie along the coordinate axes in regions of feature space dominated by anomalies. The Extended Isolation Forest (EIF) addresses this limitation by replacing axis-parallel splits with hyperplane splits of arbitrary orientation, defined by a random normal vector and an intercept point [16]. While EIF provides more accurate anomaly scores in datasets with complex geometric structures, the original Isolation

Forest remains the more widely deployed variant for intrusion detection due to its simplicity, interpretability, and lower computational overhead [16, 20]. This thesis employs the original Isolation Forest algorithm for the primary detection model.

[Figure 2.2: Illustration of Isolation Forest partitioning: anomalous points (short path lengths) versus normal points (long path lengths)]

4.6 2.6 Local Outlier Factor (LOF)

4.6.1 2.6.1 Algorithmic Foundation

Local Outlier Factor (LOF) was proposed by Breunig, Kriegel, Ng, and Sander [10] at the ACM SIGMOD International Conference on Management of Data in 2000. LOF introduced the concept of local density-based anomaly detection, addressing a fundamental limitation of global approaches: a data point may be anomalous relative to its local neighborhood while appearing normal in a global context, or vice versa [10]. By comparing the local density of each data point with the local densities of its k-nearest neighbors, LOF produces a degree-of-outlierness score that captures the relative isolation of a point within its local context.

4.6.2 2.6.2 Mathematical Formulation

The LOF computation involves several intermediate definitions [10]. Given a dataset X and a positive integer k:

k-distance. The k-distance of a point x, denoted $d_k(x)$, is the distance between x and its k-th nearest neighbor in X.

k-distance neighborhood. The k-distance neighborhood of x, denoted $N_k(x)$, is the set of all points whose distance from x is at most $d_k(x)$: $N_k(x) = \{y \in X \setminus \{x\} : d(x, y) \leq d_k(x)\}$.

Reachability distance. The reachability distance of x with respect to a point o is:

$$\text{reach-dist}_k(x, o) = \max\{d_k(o), d(x, o)\}$$

This definition smooths the density estimation by replacing distances smaller than $d_k(o)$ with $d_k(o)$, reducing the sensitivity of the density estimate to statistical fluctuations among the nearest neighbors [10].

Local reachability density. The local reachability density of x is the inverse of the average reachability distance from x to its k-nearest neighbors:

$$\text{lrd}_k(x) = \left(\frac{\sum_{o \in N_k(x)} \text{reach-dist}_k(x, o)}{|N_k(x)|} \right)^{-1}$$

Local Outlier Factor. The LOF score of x is the average ratio of the local reachability densities of x's neighbors to x's own local reachability density:

$$\mathrm{LOF}_k(x) = \frac{\sum_{o \in N_k(x)} \frac{\mathrm{lrd}_k(o)}{\mathrm{lrd}_k(x)}}{|N_k(x)|}$$

Interpretation: $\mathrm{LOF}_k(x) \approx 1$ indicates that x has a local density similar to its neighbors (normal). $\mathrm{LOF}_k(x) \gg 1$ indicates that x has a significantly lower local density than its neighbors (anomalous). $\mathrm{LOF}_k(x) < 1$ indicates that x is denser than its neighborhood (deeply embedded in a cluster) [10].

4.6.3 2.6.3 Strengths and Limitations for SSH Detection

LOF excels at detecting local anomalies—points that are anomalous relative to their immediate neighborhood but not necessarily in a global context [10, 20]. This property is valuable for SSH intrusion detection in environments where different user groups exhibit distinct behavioral patterns (e.g., system administrators versus application developers), as LOF can identify behavior that is anomalous within a specific user group without requiring global normalization [10, 17].

However, LOF has significant practical limitations for real-time SSH monitoring [10, 20, 21]. The computational complexity for computing k-nearest neighbors is $O(n^2)$ for brute-force search (or $O(n \log n)$ with spatial indexing structures such as KD-trees, which degrade in high-dimensional spaces). Additionally, LOF requires storing the entire training dataset in memory for neighbor queries during the scoring phase, and the algorithm's performance is sensitive to the choice of k, which may require domain-specific tuning [10, 20]. Goldstein and Uchida [20] found that LOF's performance is competitive with Isolation Forest on low-dimensional benchmark datasets but degrades more rapidly as dimensionality increases.

4.7 2.7 One-Class Support Vector Machine (OCSVM)

4.7.1 2.7.1 Algorithmic Foundation

One-Class Support Vector Machine (OCSVM) was proposed by Scholkopf, Platt, Shawe-Taylor, Smola, and Williamson [11] in Neural Computation in 2001. The algorithm extends the traditional two-class SVM framework to the one-class (novelty detection) setting by finding a maximal-margin hyperplane in a kernel-induced feature space that separates the training data from the origin [11]. Data points that fall on the origin side of the hyperplane are classified as anomalous (novel).

4.7.2 2.7.2 Optimization Problem

The OCSVM optimization problem is formulated as follows [11]. Given training data $\{x_1, x_2, \dots, x_n\}$ drawn from the normal class, a kernel function Φ that maps data into a high-dimensional feature space, and a parameter

$\nu \in (0, 1]$ that controls the tradeoff between the margin and the fraction of training points permitted to fall on the anomalous side of the hyperplane:

$$\min_{w,\xi,\rho} \frac{1}{2}\|w\|^2 + \frac{1}{\nu n}\sum_{i=1}^{n} \xi_i - \rho$$

subject to:

$$w \cdot \Phi(x_i) \geq \rho - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

where w is the normal vector to the separating hyperplane, ρ is the offset, and ξ_i are slack variables that permit soft-margin violations [11]. The parameter ν has a dual interpretation: it is an upper bound on the fraction of training points classified as outliers and a lower bound on the fraction of support vectors [11].

The dual formulation, solved via quadratic programming, yields the decision function:

$$f(x) = \text{sgn}\left(\sum_{i=1}^{n} \alpha_i K(x_i, x) - \rho\right)$$

where α_i are the dual variables (Lagrange multipliers) and $K(x_i, x) = \Phi(x_i) \cdot \Phi(x)$ is the kernel function [11]. The most commonly used kernel for anomaly detection is the Radial Basis Function (RBF) kernel:

$$K(x_i, x) = \exp\left(-\gamma\|x_i - x\|^2\right)$$

where $\gamma > 0$ controls the kernel bandwidth. Points for which $f(x) < 0$ are classified as anomalous [11].

4.7.3 2.7.3 Strengths and Limitations for SSH Detection

OCSVM has a rigorous theoretical foundation rooted in statistical learning theory and convex optimization, providing principled generalization guarantees [11, 17]. The ν parameter offers intuitive control over the expected anomaly proportion, which can be set based on domain knowledge about the expected attack prevalence [11, 20]. Pimentel et al. [17] noted that OCSVM is one of the best-studied novelty detection algorithms, with well-understood theoretical properties.

The primary limitations of OCSVM for real-time SSH detection are computational: the training complexity is $O(n^2)$ to $O(n^3)$ depending on the solver, and the scoring complexity is $O(n_{sv})$ where n_{sv} is the number of support vectors, which can be a substantial fraction of n [11, 20, 21]. Additionally, OCSVM performance is highly sensitive to the choice of kernel function and the hyperparameters ν and γ, requiring careful tuning via cross-validation or domain-specific heuristics [11, 20]. Goldstein and Uchida [20] observed that OCSVM achieves

competitive detection performance when properly tuned but exhibits higher variance across hyperparameter settings compared to Isolation Forest.

[Table 2.3: Comparative summary of Isolation Forest, LOF, and OCSVM]

Property	Isolation Forest [8, 9]	LOF [10]	OCSVM [11]
Detection paradigm	Isolation-based	Local density-based	Boundary-based
Training complexity	$O(t \cdot \psi \log \psi)$	$O(n^2)$	$O(n^2)$–$O(n^3)$
Scoring complexity	$O(t \log \psi)$	$O(n \cdot k)$	$O(n_{sv})$
Memory requirement	Model only (t trees)	Full training set	Support vectors
Distributional assumptions	None	None	Implicit (via kernel)
Key hyperparameters	t, ψ, contamination	k	ν, γ, kernel
Sensitivity to dimensionality	Low	High	Moderate
Score interpretability	High (probability-like)	Moderate (ratio)	Low (signed distance)
Real-time suitability	High	Low–Moderate	Moderate

4.8 2.8 Feature Selection and Data Preprocessing for Intrusion Detection Systems

The effectiveness of any machine-learning-based intrusion detection system is fundamentally constrained by the quality and informativeness of the features extracted from raw data [7, 12, 15]. Buczak and Guven [12] emphasized that feature engineering—the process of transforming raw log data into meaningful numerical representations—is often the single most impactful component of the machine learning pipeline for cybersecurity applications, outweighing the choice of algorithm in many practical scenarios.

Features for SSH brute-force detection can be categorized into four principal classes [5, 12, 15]:

Frequency features capture the volume and rate of authentication activity. Examples include the total number of login attempts, the number of failed attempts, the failure ratio (failed/total), and the number of unique

usernames attempted within a time window [5, 15]. Frequency features are effective for detecting high-speed brute-force attacks but provide limited discriminative power against slow-rate attacks that operate below conventional rate thresholds [23].

Temporal features capture the timing characteristics of authentication events. Examples include the mean, minimum, and standard deviation of inter-attempt intervals, session duration statistics, and the distribution of attempts across time-of-day bins [5, 18]. Bezerra et al. [18] demonstrated that temporal features provide significant discriminative power for distinguishing automated attacks from human-initiated sessions, because automated tools produce highly regular temporal patterns (near-constant inter-attempt intervals) that differ markedly from the irregular timing of human interactions. Sperotto et al. [15] confirmed this finding in their flow-based SSH analysis, reporting that temporal features contributed more to detection accuracy than frequency features alone.

Authentication features capture the credential-level characteristics of login behavior. Examples include the number of unique usernames attempted, the presence of root/admin login attempts, the diversity of attempted passwords (measurable through entropy or unique-count metrics), and the ratio of successful to total attempts [5, 19]. Owens and Matthews [19] found that attack sessions exhibit characteristic username and password distributions (heavily skewed toward common accounts and weak passwords) that differ systematically from legitimate usage patterns.

Connection features capture network-level properties of SSH sessions. Examples include the number of concurrent connections, source port entropy, geographic origin diversity, and the use of known malicious network ranges [15, 24]. Ring et al. [24] surveyed datasets for network-based intrusion detection and identified connection-level features as particularly valuable for detecting distributed attacks where individual-IP behavioral features may appear benign.

This thesis extracts 14 features per source IP address per 5-minute sliding window, spanning all four feature categories. The feature set was designed to balance comprehensiveness (capturing sufficient behavioral information to discriminate among all five attack variants) with computational efficiency (maintaining the real-time processing requirement). Feature selection was informed by the literature reviewed above and refined through empirical feature importance analysis, which identified session_duration_mean (5.50% importance), min_inter_attempt_time (3.86%), and mean_inter_attempt_time (2.61%) as the top three discriminative features—confirming the primacy of temporal features reported by Bezerra et al. [18] and Sperotto et al. [15].

Data preprocessing for anomaly detection requires particular attention to feature scaling, missing value handling, and the treatment of categorical variables [7, 12]. Pimentel et al. [17] noted that distance-based and kernel-based methods (LOF, OCSVM) are highly sensitive to feature scale, necessitating standardization (zero mean, unit variance) or min-max normalization. Isolation Forest is theoretically scale-invariant (because splits are defined on individual features), but empirical evidence suggests that standardization can improve performance when features have vastly different ranges [9, 20].

[Figure 2.3: Feature extraction pipeline from SSH authentication logs to 14-dimensional feature vectors]

4.9 2.9 Dynamic Threshold Mechanisms for Anomaly Detection

4.9.1 2.9.1 The Static Threshold Problem

In anomaly detection systems, the threshold determines the boundary between normal and anomalous classifications: data points with anomaly scores exceeding the threshold are flagged as attacks, while those below are treated as normal [7, 13]. Static thresholds—fixed values determined during training and applied unchanged during operation—are simple to implement but fundamentally inappropriate for data streams with non-stationary characteristics [7, 13, 14]. In the SSH monitoring context, authentication traffic exhibits pronounced temporal variability: business hours generate more legitimate logins than off-hours, weekdays differ from weekends, and events such as system maintenance or application deployments create legitimate but unusual traffic spikes [5, 15]. A static threshold calibrated for average conditions will generate excessive false positives during high-activity periods and miss subtle attacks during low-activity periods [7, 14].

4.9.2 2.9.2 Exponentially Weighted Moving Average (EWMA)

The Exponentially Weighted Moving Average is a time series smoothing method that assigns exponentially decreasing weights to older observations, enabling adaptive tracking of the current process level [13]. Originally introduced by Roberts (1959) for statistical quality control and comprehensively analyzed by Lucas and Saccucci [13], EWMA has been widely adopted in process monitoring, financial time series analysis, and network anomaly detection [13, 14].

The EWMA statistic at time t is defined as:

$$\hat{\mu}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \hat{\mu}_{t-1}$$

where x_t is the observed value (anomaly score) at time t, $\hat{\mu}_t$ is the smoothed estimate at time t, $\hat{\mu}_0$ is initialized to the mean of the training anomaly scores, and $\alpha \in (0, 1]$ is the smoothing parameter that controls the tradeoff between responsiveness to recent observations and stability against noise [13].

The variance of the EWMA statistic is:

$$\text{Var}(\hat{\mu}_t) = \sigma^2 \cdot \frac{\alpha}{2 - \alpha} \cdot \left[1 - (1 - \alpha)^{2t}\right]$$

which converges to $\sigma^2 \cdot \frac{\alpha}{2-\alpha}$ as $t \to \infty$ [13]. Control limits (thresholds) are typically set at $\hat{\mu}_t \pm L \cdot \sqrt{\text{Var}(\hat{\mu}_t)}$, where L is a multiplier (commonly 2.5–3.0) determined by the desired false alarm rate [13, 14].

Lucas and Saccucci [13] demonstrated that EWMA control charts are particularly effective at detecting small, persistent shifts in the process mean—precisely the type of signal generated by slow brute-force attacks that produce a gradual, sustained elevation in anomaly scores. Montgomery [14] provides a comprehensive treatment of EWMA in the broader context of statistical quality control and discusses the selection of α and L for different detection objectives.

4.9.3 2.9.3 Adaptive Percentile Method

The Adaptive Percentile method determines the threshold based on the empirical quantile of the anomaly score distribution within a sliding time window [14, 17]. The threshold at time t is defined as:

$$\theta_t = P_q(S_W)$$

where P_q denotes the q-th percentile and $S_W = \{s_{t-W+1}, s_{t-W+2}, \dots, s_t\}$ is the set of the most recent W anomaly scores [14]. This approach has two key advantages: it requires no distributional assumptions about the anomaly scores (unlike EWMA, which implicitly assumes approximate normality for the control limit calculation), and it naturally adapts to changes in the score distribution by continuously updating the reference window [17]. The primary disadvantage is sensitivity to the window size W: too small a window results in volatile thresholds, while too large a window reduces responsiveness to genuine distributional shifts [14].

4.9.4 2.9.4 Hybrid EWMA-Adaptive Percentile Method

This thesis proposes a hybrid dynamic thresholding mechanism that combines EWMA and Adaptive Percentile to leverage the complementary strengths of both approaches [13, 14]. EWMA provides long-term trend tracking and smoothing, preventing the threshold from oscillating in response to transient score fluctuations. Adaptive Percentile provides distribution-free adaptiveness, accurately reflecting the empirical score distribution without parametric assumptions. The hybrid threshold is computed as a weighted combination:

$$\theta_t^{\text{hybrid}} = w_{\text{EWMA}} \cdot \theta_t^{\text{EWMA}} + w_{\text{AP}} \cdot \theta_t^{\text{AP}}$$

where θ_t^{EWMA} is the EWMA-based threshold, θ_t^{AP} is the Adaptive Percentile threshold, and $w_{\text{EWMA}} + w_{\text{AP}} = 1$ are the blending weights [13, 14]. The specific weight values and the calibration procedure are detailed in Chapter 3 (Methodology). The hybrid approach ensures that the system maintains stable baseline tracking (from EWMA) while remaining responsive to local distributional changes (from Adaptive Percentile), a combination that is particularly valuable for detecting the gradual anomaly score elevation characteristic of slow brute-force campaigns [13, 23].

[Figure 2.4: Comparison of static, EWMA, Adaptive Percentile, and hybrid thresholds on a simulated anomaly

score time series with embedded attack periods]

4.10 2.10 Related Work in SSH Brute-Force Detection

This section presents a systematic review of the most relevant prior studies in SSH brute-force detection, organized by methodological approach. The review encompasses both international and domestic (Vietnamese) research and identifies the specific contributions and limitations of each study relative to the objectives of this thesis.

4.10.1 2.10.1 Flow-Based and Network-Layer Approaches

Sperotto et al. [15] conducted pioneering work on flow-based intrusion detection for SSH, analyzing network flow records from the University of Twente (Netherlands) to characterize SSH attack patterns. The study identified that flow-level features—including flow duration, packet count, and byte count—provide sufficient information to distinguish between legitimate SSH sessions and brute-force attacks with greater than 90% accuracy using Hidden Markov Models. The primary contribution was the demonstration that application-layer payload inspection is unnecessary for SSH attack detection; metadata features alone are sufficient [15]. However, the study employed supervised classification requiring labeled data, and the flow-based feature set does not capture application-layer behavioral nuances such as username diversity or password retry patterns that are available from authentication logs [15, 24].

Hellemons et al. [5] developed SSHCure, a three-phase detection system that models SSH attacks as a sequence of scanning, brute-force, and compromise phases, each characterized by distinct flow-level features. SSHCure achieved high detection rates (>95% true positive rate) for attacks that follow the canonical three-phase pattern [5]. The key limitation is the assumption of sequential phase progression, which does not hold for all attack variants—particularly credential-stuffing attacks that skip the scanning phase and slow brute-force attacks that blur the boundary between phases [5, 23].

4.10.2 2.10.2 Machine Learning Approaches on Benchmark Datasets

Ahmad et al. [25] conducted a systematic study of network-based intrusion detection systems (NIDS), comparing multiple machine learning algorithms including Isolation Forest, LOF, OCSVM, Random Forest, and deep learning approaches across the NSL-KDD, CICIDS2017, and UNSW-NB15 benchmark datasets. The study found that Isolation Forest achieved the best balance between detection performance and computational efficiency among unsupervised methods, with F1-scores ranging from 82% to 91% depending on the dataset [25]. However, the study evaluated algorithms on general network intrusion data rather than SSH-specific data, and the benchmark datasets—while valuable for standardized comparison—are known to suffer from age-related limitations, as documented by Ring et al. [24].

Ring et al. [24] provided a comprehensive survey of datasets for network-based intrusion detection systems, evaluating 34 datasets across criteria including realism, labeling accuracy, attack diversity, and temporal currency. The survey concluded that many widely used benchmark datasets (including NSL-KDD, first released in 1999) do not adequately represent modern attack characteristics, and recommended the use of application-specific datasets collected from controlled environments or honeypots for domain-focused research [24]. This recommendation directly motivates the use of honeypot-collected data in this thesis.

Goldstein and Uchida [20] performed a comparative evaluation of unsupervised anomaly detection algorithms on 10 benchmark datasets, including network intrusion data. Their results showed that Isolation Forest ranked among the top three algorithms in average performance across all datasets, with LOF and OCSVM achieving competitive performance on specific datasets but exhibiting higher variance across datasets [20]. The study confirmed that algorithm selection should be guided by domain-specific evaluation rather than reliance on a single benchmark.

4.10.3 2.10.3 SSH-Specific Machine Learning Studies

Javed and Paxson [23] studied stealthy SSH brute-forcing campaigns using data from a large academic network, identifying attacks that operate at rates as low as 1–2 attempts per hour and unfold over periods of days to weeks. The study demonstrated that these stealthy campaigns can achieve compromise rates comparable to aggressive attacks while evading all conventional detection mechanisms [23]. The key insight—that temporal behavioral features are essential for detecting low-rate attacks—directly informs the feature engineering approach of this thesis, which prioritizes temporal features (inter-attempt timing, session duration) alongside traditional frequency features.

Bezerra et al. [18] analyzed SSH attack durations using data from a distributed honeypot network and found significant heterogeneity in attack temporal profiles: attack sessions ranged from sub-second scans to multi-day campaigns, with distinct temporal signatures associated with different attack tools and botnets. The study identified inter-attempt timing regularity as a highly discriminative feature for distinguishing automated attacks from human-initiated sessions [18].

Owens and Matthews [19] conducted an early but influential analysis of SSH brute-force password characteristics using honeypot data. The study documented that the top 20 most-attempted passwords account for over 40% of all attack attempts, and that attackers exhibit strong preferences for specific username-password combinations (e.g., root/root, admin/123456) [19]. These findings inform the authentication-category features in this thesis's 14-feature set, particularly the unique username count and password diversity metrics.

Park et al. [22] investigated SSH attacks targeting network infrastructure devices (routers and switches), documenting that these attacks exhibit distinct behavioral patterns compared to attacks against general-purpose servers, including different username preferences and timing profiles [22]. While this thesis focuses on server-targeted attacks, the existence of device-specific attack patterns underscores the importance of adaptive,

learning-based detection approaches that can accommodate behavioral diversity without requiring explicit rules for each target type.

4.10.4 2.10.4 Deep Learning and Advanced Approaches

Nassif et al. [21] surveyed machine learning and deep learning approaches for anomaly-based intrusion detection, finding that deep learning methods (autoencoders, LSTMs, GANs) can achieve higher detection accuracy than traditional machine learning methods on large, high-dimensional datasets. However, the survey also noted that deep learning methods require significantly more training data, computational resources, and hyperparameter tuning, and that their black-box nature complicates interpretation and debugging in security-critical applications [21]. For real-time SSH monitoring on resource-constrained environments, traditional machine learning methods such as Isolation Forest offer a more practical balance of performance and efficiency [21, 25].

Buczak and Guven [12] provided the foundational survey of data mining and machine learning methods for cybersecurity intrusion detection, cataloging over 50 studies spanning supervised, unsupervised, and hybrid approaches. The survey identified several persistent challenges in the field: the reliance on outdated benchmark datasets, the absence of standardized evaluation methodologies, the difficulty of comparing results across studies that use different datasets and metrics, and the gap between algorithmic research and operational deployment [12]. These challenges motivate the methodological choices of this thesis, including the use of real honeypot data, the standardized evaluation framework (accuracy, precision, recall, F1, FPR), and the emphasis on end-to-end system integration.

Pimentel et al. [17] reviewed novelty detection methods—a formulation closely related to semi-supervised anomaly detection—across multiple application domains including intrusion detection. The review categorized methods into probabilistic, distance-based, reconstruction-based, domain-based, and information-theoretic approaches, and identified one-class classification (including OCSVM) and isolation-based methods (including Isolation Forest) as particularly well-suited for applications where the target class (normal behavior) is well-sampled but the outlier class (attacks) is poorly characterized or entirely absent from training data [17].

Hariri et al. [16] proposed the Extended Isolation Forest (EIF), which addresses the axis-parallel bias of the original algorithm by using random hyperplane splits. EIF demonstrated improved anomaly detection accuracy on synthetic datasets with complex geometric structures. However, for the tabular feature data characteristic of SSH log analysis, the performance difference between IF and EIF is marginal, and the original IF's lower computational cost and simpler implementation make it the preferred choice for real-time applications [16, 20].

[Table 2.4: Comprehensive comparison of related works in SSH brute-force detection]

Study	Year	Method	Data Source	Key Metric	Dynamic Threshold	Early Warning	System Integration
Hellemons et al. [5]	2012	SSHCure (three-phase)	Flow data	TPR > 95%	No	Partial	No
Sperotto et al. [15]	2010	HMM (flow-based)	Univ. Twente flows	Acc > 90%	No	No	No
Javed & Paxson [23]	2013	Statistical analysis	Academic network	–	No	Yes (analysis)	No
Owens & Matthews [19]	2008	Password analysis	Honeypot	–	No	No	No
Bezerra et al. [18]	2019	Temporal analysis	Distributed honeypot	–	No	No	No
Goldstein & Uchida [20]	2016	IF, LOF, OCSVM (comparison)	10 benchmarks	AUC varies	No	No	No
Nassif et al. [21]	2021	ML/DL survey	Multiple	Survey	No	No	No
Park et al. [22]	2020	SSH on routers	Router logs	–	No	No	No
Ahmad et al. [25]	2021	Multiple ML	NSL-KDD, CICIDS	F1 82–91%	No	No	No

Study	Year	Method	Data Source	Key Metric	Dynamic Threshold	Early Warning	System Integration
Ring et al. [24]	2019	Dataset survey	34 datasets	Survey	N/A	N/A	N/A
Hariri et al. [16]	2019	Extended IF	Synthetic + real	AUC improved	No	No	No
This thesis	**2026**	**IF + EWMA-AP**	**Honeypot + Sim (174K lines)**	**F1 = 93.74%**	**Yes (hybrid)**	**Yes**	**Yes (9 Docker services)**

4.11 2.11 Summary of the Literature Review

The comprehensive review of the literature reveals a rich but fragmented research landscape characterized by significant methodological advances in individual components—anomaly detection algorithms, feature engineering, threshold mechanisms—but a persistent absence of integrated solutions that combine these components into deployable systems [7, 12, 15].

On algorithms: Isolation Forest [8, 9] has emerged as one of the most effective unsupervised anomaly detection algorithms for network security applications, offering a compelling combination of detection performance, computational efficiency, and interpretability [20, 21]. LOF [10] provides complementary local-density-based detection but is limited by computational complexity and memory requirements. OCSVM [11] offers strong theoretical foundations but requires careful hyperparameter tuning and has higher computational costs. All three algorithms have been validated on benchmark datasets, but their comparative evaluation on real-world SSH honeypot data in a semi-supervised configuration remains underexplored [20, 24, 25].

On features: The literature consistently identifies temporal features (inter-attempt timing, session duration) as more discriminative than frequency features (attempt counts) for distinguishing automated SSH attacks from legitimate activity, particularly for slow-rate and stealthy attack variants [5, 15, 18, 23]. Authentication features (username diversity, password patterns) provide additional discriminative power for credential-stuffing and dictionary attacks [19]. The integration of features from all four categories (frequency, temporal, authentication, connection) in a unified feature set has been recommended but rarely implemented and evaluated systematically [12, 15].

On thresholds: Static thresholds are inadequate for SSH monitoring due to the non-stationary nature of authentication traffic [6, 7, 13]. EWMA-based adaptive thresholds offer superior sensitivity to small, persistent shifts

[13, 14], while percentile-based methods provide distribution-free adaptiveness [14, 17]. Hybrid approaches combining multiple threshold methods have been proposed in the statistical process control literature [13, 14] but have not been applied to SSH anomaly detection.

On integration: The gap between algorithmic research and deployable systems remains one of the most significant barriers to the practical adoption of machine learning in cybersecurity [12, 15]. The vast majority of studies evaluate algorithms in isolation on benchmark datasets without addressing log ingestion, real-time feature extraction, automated response, visualization, or containerized deployment [20, 21, 24, 25].

4.12 2.12 Research Gap and Contribution

Based on the literature review, four specific research gaps are identified that collectively define the novel contribution of this thesis:

Gap 1: Absence of comparative algorithm evaluation on real SSH honeypot data. While Isolation Forest, LOF, and OCSVM have been extensively compared on benchmark datasets [20, 25], their comparative performance on authentic SSH honeypot data—with its characteristic noise, class imbalance (679 attacking IPs versus 64 normal users), and behavioral diversity (five attack variants)—has not been systematically evaluated in a semi-supervised configuration [24]. **Contribution:** This thesis provides a rigorous comparative evaluation of all three algorithms on 174,250 lines of combined honeypot and simulated data, reporting standardized metrics (accuracy, precision, recall, F1, FPR) with optimized results: IF (Acc = 90.31%, F1 = 93.74%, FPR = 29.00%), LOF (Acc = 83.22%, F1 = 89.94%), OCSVM (Acc = 91.38%, F1 = 94.55%).

Gap 2: No hybrid dynamic thresholding for SSH anomaly detection. EWMA [13] and percentile-based thresholds [14] have been studied independently, but their combination into a hybrid mechanism specifically designed for SSH anomaly detection has not been previously proposed or evaluated [7, 12]. Existing SSH detection systems rely on either static thresholds [6] or simple adaptive mechanisms without the complementary strengths of trend-following and distribution-free approaches [5, 15]. **Contribution:** This thesis proposes and validates the EWMA-Adaptive Percentile hybrid thresholding mechanism, demonstrating its effectiveness across five attack scenarios with varying temporal profiles [13, 14].

Gap 3: Limited exploitation of early-warning capability. While some studies have identified the potential for early detection based on behavioral features [5, 23], the systematic exploitation of early-phase behavioral signatures for proactive attack warning—particularly for slow-rate attacks—remains largely unexplored [18, 23]. **Contribution:** This thesis demonstrates early-warning capability across all five attack scenarios, with the system issuing EARLY_WARNING alerts after only 3–5 attempts in the slow brute-force scenario, compared to complete detection failure by Fail2Ban under default settings.

Gap 4: Research-to-deployment gap. The overwhelming majority of studies in this domain evaluate algorithms in isolation without addressing system integration [12, 20, 21, 25]. **Contribution:** This thesis imple-

ments a complete end-to-end system comprising 9 Docker services (ELK Stack, FastAPI, React, Fail2Ban), with a 14-feature extraction pipeline processing SSH logs in 5-minute sliding windows, demonstrating that the transition from algorithm to deployable system is achievable within a modern containerized architecture [6, 15].

[Figure 2.5: Visual mapping of research gaps to the contributions of this thesis]

[Table 2.5: Summary of research gaps, their evidence in the literature, and corresponding contributions]

Gap	Evidence in Literature	This Thesis's Contribution
No comparative evaluation on real SSH data	Goldstein & Uchida [20] used benchmarks; Ahmad et al. [25] used general IDS datasets	IF, LOF, OCSVM compared on 174,250-line honeypot + simulation dataset
No hybrid dynamic threshold for SSH	Fail2Ban [6] uses static; SSHCure [5] uses fixed phases	EWMA-Adaptive Percentile hybrid mechanism
Limited early-warning exploitation	Javed & Paxson [23] identified potential; no system implemented	Early warning in 3–5 attempts for slow attacks
Research-to-deployment gap	Buczak & Guven [12] and Sperotto et al. [15] identified the gap	9-service Docker architecture with automated response

Chapter 5

CHAPTER 3: METHODOLOGY

5.1 3.1 Research Design Overview

5.1.1 3.1.1 Philosophical and Methodological Foundations

This research adopts a positivist epistemological stance, employing a quantitative experimental methodology to design, implement, and evaluate an AI-based SSH brute-force attack detection and prevention system. The research follows the Design Science Research Methodology (DSRM) framework proposed by Peffers et al. [41], which is widely regarded as the standard methodology for information systems research involving the creation of IT artifacts. DSRM comprises six iterative phases: (1) problem identification, (2) objective definition, (3) design and development, (4) demonstration, (5) evaluation, and (6) communication. This chapter addresses phases (3) and (4), while Chapter 4 presents the evaluation results.

The experimental approach is appropriate because the research question — whether an Isolation Forest model combined with an EWMA-Adaptive Percentile dynamic threshold can effectively detect SSH brute-force attacks in real time — is inherently quantitative and amenable to controlled experimentation. The dependent variables (accuracy, precision, recall, F1-score, ROC-AUC, false positive rate) can be precisely measured, and the independent variables (model hyperparameters, threshold parameters, feature set composition, window configuration) can be systematically manipulated to assess their effects on detection performance.

The methodology integrates three complementary research strategies:

Empirical data collection. Real-world attack data is obtained from an SSH honeypot deployed on the public Internet, while normal behavioral data is generated through a controlled simulation environment. This dual-source approach addresses the fundamental data challenge in anomaly detection: the simultaneous need for clean normal data (for training) and authentic attack data (for evaluation) [42].

Comparative algorithm evaluation. Three unsupervised anomaly detection algorithms — Isolation Forest

(IF), Local Outlier Factor (LOF), and One-Class Support Vector Machine (OCSVM) — are systematically compared under identical conditions, following the best practices for algorithm benchmarking established by Demsar [43] and Campos et al. [44].

System integration and scenario-based testing. The detection algorithm is embedded within a complete end-to-end system comprising 9 Docker services, and evaluated against five distinct attack scenarios designed to cover the spectrum of SSH brute-force attack variants documented in the literature.

5.1.2 3.1.2 Semi-Supervised Anomaly Detection Paradigm

The research adopts the semi-supervised anomaly detection paradigm, also referred to as novelty detection [45], in which models are trained exclusively on normal (in-distribution) data and subsequently identify observations that deviate from the learned normal distribution as anomalies. This paradigm was selected over supervised classification and fully unsupervised clustering for three theoretically and practically motivated reasons.

First, **practicality of data acquisition.** In operational cybersecurity environments, normal data is far more abundant and easier to collect than labeled attack data. Organizations can readily extract logs of normal SSH operations from their existing infrastructure, but they cannot feasibly obtain representative samples of every possible attack type, including zero-day variants that have not yet been observed [46]. Chandola et al. [47] formalized this observation as the "labeled data scarcity problem" in their seminal survey of anomaly detection techniques.

Second, **generalization to novel attacks.** Models trained on attack signatures can only detect attacks whose signatures match the training data. Semi-supervised models, by contrast, can detect any attack type whose behavioral characteristics deviate from the learned normal profile, including attack variants never previously encountered (zero-day attacks). This property was demonstrated empirically by Goldstein and Uchida [48], who showed that novelty detection methods consistently outperform signature-based methods on datasets containing previously unseen anomaly types.

Third, **avoidance of class imbalance pathologies.** Supervised classification methods are well known to suffer from degraded performance when the class distribution is severely imbalanced [49], as is typical in cybersecurity datasets where normal traffic vastly outnumbers attack traffic (or vice versa, as in honeypot data). The semi-supervised paradigm is architecturally immune to this problem because the training set contains only a single class.

The formal framework is as follows. Let $\mathbf{X}_{train} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ denote the training set, where each $\mathbf{x}_i \in \mathbb{R}^{14}$ is a feature vector representing the SSH behavioral profile of a single IP address within a single 5-minute time window, and all $\mathbf{x}_i$ are drawn from the normal class. A model $f : \mathbb{R}^{14} \to \mathbb{R}$ is trained on $\mathbf{X}_{train}$ to produce an anomaly score $s(\mathbf{x})$ for any new observation $\mathbf{x}$. A threshold function $\theta(t)$ maps the anomaly score to a binary classification:

$$\hat{y}(\mathbf{x}, t) = \begin{cases} \text{attack} & \text{if } s(\mathbf{x}) > \theta(t) \\ \text{normal} & \text{otherwise} \end{cases}$$

where $\theta(t)$ may be static (a fixed constant) or dynamic (a function of recent score history), as described in Section 3.8.

5.2 3.2 System Architecture

5.2.1 3.2.1 High-Level Architecture

The SSH brute-force attack detection and prevention system is designed following a microservices architecture, deployed on Docker Compose with a total of 9 services operating in coordination. The microservices architecture was chosen over a monolithic design for three reasons documented by Newman [50]: independent scalability of components, fault isolation (failure of one service does not cascade to others), and technology heterogeneity (each service can use the language and framework best suited to its function).

[Figure 3.1: Overall system architecture for SSH brute-force attack detection and prevention]

The system is organized into five functional layers, each with a clearly defined responsibility:

Layer 1: Data Collection. This layer is responsible for ingesting authentication logs from SSH servers. Logstash tails the `/var/log/auth.log` file on target SSH servers, applies Grok pattern parsing to extract structured fields (timestamp, hostname, service, PID, event type, username, IP address, port), and forwards the structured events to Elasticsearch for indexed storage. The use of Logstash for log ingestion follows the established ELK Stack architecture, which Elasticsearch B.V. reports is deployed in over 500,000 organizations worldwide for security analytics [51].

Layer 2: Processing and Analysis. The central computational layer comprises the data preprocessing module, the feature extraction module, and the AI model inference module. These modules are implemented as asynchronous RESTful APIs using the FastAPI framework (Python), which leverages ASGI (Asynchronous Server Gateway Interface) for high-concurrency request handling. FastAPI was selected over Flask and Django REST Framework based on benchmarks by Lust [52] showing 3–10x throughput improvements for I/O-bound workloads.

Layer 3: Decision. This layer implements the dynamic threshold algorithm based on the EWMA-Adaptive Percentile method (detailed in Section 3.8). It receives anomaly scores from the AI model and produces a binary classification decision (normal or attack) together with a confidence level (EARLY_WARNING or ALERT).

Layer 4: Response. Fail2Ban is integrated to execute automated prevention measures upon attack detection. When an ALERT-level detection occurs, the system invokes the Fail2Ban API to ban the offending IP

address, applies a configurable ban duration with progressive escalation for repeat offenders, and dispatches multi-channel alert notifications (dashboard, webhook, log).

Layer 5: Presentation. A single-page web application developed in React provides a real-time monitoring dashboard displaying detection events, attack statistics, IP geolocation, anomaly score trends, and administrative configuration controls.

5.2.2 3.2.2 Service Inventory

[Table 3.1: Complete inventory of 9 Docker services]

No.	Service	Technology	Function	Port
1	Detector	Python (scikit-learn)	AI model inference and threshold computation	Internal
2	API Server	FastAPI (Python)	Business logic, REST API, orchestration	8000
3	Frontend	React (TypeScript)	Real-time monitoring dashboard	3000
4	Elasticsearch	Elasticsearch 8.x	Log storage, full-text indexing, search	9200
5	Logstash	Logstash 8.x	Log ingestion, parsing, forwarding	5044
6	Kibana	Kibana 8.x	Data visualization, ad hoc exploration	5601
7	Redis	Redis 7.x	Cache, message queue, sliding window state	6379
8	Fail2Ban	Fail2Ban 1.x	Automated IP blocking via iptables/nftables	Internal
9	SSH Target	OpenSSH 9.x	Target SSH server for attack scenario testing	22

5.2.3 3.2.3 Data Flow Pipeline

[Figure 3.2: End-to-end data flow pipeline from SSH log event to classification decision and response]

The data processing pipeline operates through five sequential stages with well-defined interfaces:

1. **Ingestion.** SSH authentication events on the target server are written to `/var/log/auth.log` by the sshd process. Logstash monitors this file via the file input plugin (using inode tracking to handle log rotation) and applies a Grok filter to parse each line into structured JSON fields.

2. **Indexing.** Parsed events are forwarded to Elasticsearch via the Elasticsearch output plugin. Events are indexed in time-partitioned indices (e.g., `ssh-auth-2026.03.26`) with a mapping that preserves the original data types (keyword for IP addresses, date for timestamps, integer for port numbers).

3. **Feature Extraction.** The API Server periodically queries Elasticsearch for new events (polling interval: 30 seconds) or receives push notifications via a webhook. For each active IP address, the server maintains a sliding window buffer (stored in Redis for persistence across restarts) and computes the 14-feature vector as described in Section 3.6.

4. **Inference and Decision.** The feature vector is normalized using the pre-fitted RobustScaler, passed to the trained anomaly detection model to obtain an anomaly score, and compared against the current dynamic threshold $\theta(t)$ to produce a classification decision.

5. **Action.** If the classification is ALERT, the system invokes the Fail2Ban REST API to ban the IP, writes the detection event to Elasticsearch for audit logging, and pushes a real-time notification to the React dashboard via WebSocket.

5.3 3.3 Laboratory Environment Setup

5.3.1 3.3.1 Hardware and Software Configuration

All experiments were conducted on a single physical machine to ensure reproducibility and eliminate network variability as a confounding factor. The Docker Compose configuration ensures that all 9 services run within a single Docker network with deterministic inter-service communication.

[Table 3.2: Laboratory environment specifications]

Component	Specification
Operating System	Ubuntu 22.04 LTS (host) / Docker containers
Containerization	Docker 24.x, Docker Compose 2.x
Python	3.10 with scikit-learn 1.3.x, pandas 2.1.x, numpy 1.25.x
Machine Learning	scikit-learn (IsolationForest, LocalOutlierFactor, OneClassSVM)
Web Framework	FastAPI 0.104.x (ASGI)
Frontend	React 18.x with TypeScript
Search Engine	Elasticsearch 8.x
Log Pipeline	Logstash 8.x
Visualization	Kibana 8.x
Cache	Redis 7.x
Prevention	Fail2Ban 1.x

5.3.2 3.3.2 Honeypot Deployment

The SSH honeypot was deployed on a Virtual Private Server (VPS) with a publicly routable IPv4 address. The honeypot was configured to attract realistic SSH brute-force attack traffic through several deliberate design choices informed by the honeypot deployment guidelines of Provos and Holz [53]:

- **Port selection.** SSH was exposed on the default port 22 rather than a non-standard port. Alata et al. [54] demonstrated that the majority of automated SSH scanners target only port 22, and moving SSH to a non-standard port reduces attack traffic by 95–99% — which would be counterproductive for data collection purposes.
- **Hostname.** The hostname was set to "mail" to simulate a mail server, a high-value target that attracts more sophisticated brute-force attacks beyond opportunistic scanning [55].
- **Authentication policy.** Password authentication was enabled with a permissive MaxAuthTries setting to maximize the number of events recorded per attacking session.
- **Collection duration.** The honeypot operated continuously for 5 days (2026-03-22 to 2026-03-27), during which all SSH authentication events were recorded to `/var/log/auth.log`.
- **Administrative access.** Six (6) verified administrative IP addresses (27.64.18.8, 104.28.156.151, 104.28.159.126, 116.110.42.131, 118.69.182.144, 14.169.70.183) were used by the research team for honeypot management. All logins from these IPs were recorded and later labeled as normal.

5.3.3 3.3.3 Simulation Environment

The simulation environment was designed to generate realistic normal SSH activity representative of a medium-sized organization. The simulation script, written in Python using the Paramiko library, creates 64 user accounts (52 human users and 12 service accounts) and orchestrates their SSH activity according to behavioral profiles calibrated from the literature on organizational IT usage patterns [56].

Key simulation parameters:

- **Duration.** Approximately 25 hours of continuous activity on 2026-03-26.
- **Hostname.** "if" (to distinguish from the honeypot hostname "mail").
- **Network.** All connections originate from the 192.168.152.0/24 subnet.
- **Human users (52).** Activity concentrated during business hours (8:00–18:00) with reduced activity during evenings and weekends. Session durations range from 5 minutes to several hours. Occasional password mistyping is included at a natural rate.
- **Service accounts (12).** Automated cron-based connections at regular intervals, short session durations, and zero password failures.
- **Failure rate.** 177 failed logins out of 4,382 total attempts (4.04%), reflecting the empirically observed rate of accidental password failures in organizational environments reported by Florencio and Herley [57].

5.4 3.4 Data Collection Methods

5.4.1 3.4.1 Data Sources and Rationale

The research employs two primary data sources, following the dual-source methodology advocated by Ring et al. [58] for constructing intrusion detection evaluation datasets. The dual-source approach addresses the fundamental tension between data purity and data realism: controlled environments produce clean, well-labeled data but may lack the complexity and diversity of real-world traffic, while real-world captures provide authentic attack patterns but resist precise labeling.

Source 1: Honeypot data (honeypot_auth.log). This authentication log was collected from the SSH honeypot described in Section 3.3.2. The honeypot approach, first formalized by Spitzner [59], provides highly representative attack data because the traffic is generated by real attackers using real tools and techniques, rather than synthetic simulations.

[Table 3.3: Honeypot data summary statistics]

Attribute	Value
Total log lines	119,729
Collection period	2026-03-22 to 2026-03-27 (5 days)
Hostname	“mail”
Unique IP addresses	679
Failed password events	29,301
Invalid user attempts	12,799
Accepted root logins (admin IPs)	532
Number of admin IPs	6
“message repeated” entries	1,920
SSHD entries after parsing	122,809
Average failed attempts per day	5,860
Average failed attempts per IP	43.15

Source 2: Simulation data (simulation_auth.log). This authentication log was generated from the controlled simulation environment described in Section 3.3.3.

[Table 3.4: Simulation data summary statistics]

Attribute	Value
Total log lines	54,521
Collection period	~25 hours (2026-03-26)

Attribute	Value
Hostname	“if”
User accounts	64 (52 human + 12 service)
Network subnet	192.168.152.0/24
Accepted logins	4,205
Failed logins	177
Success rate	95.96%
Failure rate	4.04%
SSHD entries after parsing	34,700

5.4.2 3.4.2 Log Data Format

SSH servers on Linux systems record authentication events to the system authentication log (typically `/var/log/auth.log` on Debian/Ubuntu distributions) via the syslog facility. Each log entry follows the standard syslog format specified in RFC 5424 [60]:

```
<timestamp> <hostname> <service>[<PID>]: <message>
```

The principal SSH event types relevant to brute-force detection, as documented in the OpenSSH source code and manual pages [61], are:

Event Type	Log Message Pattern	Significance
Failed password	`Failed password for <user> from <IP> port <port>`	Primary brute-force indicator
Invalid user	`Invalid user <user> from <IP> port <port>`	Username enumeration indicator
Accepted password	`Accepted password for <user> from <IP> port <port>`	Successful authentication
Accepted publickey	`Accepted publickey for <user> from <IP> port <port>`	Key-based authentication
Connection closed	`Connection closed by <IP> port <port>`	Session termination
PAM failure	`pam_unix(sshd:auth): authentication failure`	PAM-level failure
Max retries exceeded	`error: maximum authentication attempts exceeded`	Retry limit reached
Message repeated	`message repeated <N> times`	Syslog deduplication

The “message repeated” entries require special handling during preprocessing. When syslog detects consecutive identical messages, it consolidates them into a single “message repeated N times” entry. The preprocessing pipeline expands these entries by replicating the preceding event N times, which increased the honeypot dataset from 119,729 raw lines to 122,809 parsed SSHD entries.

5.4.3 3.4.3 Labeling Strategy

The labeling strategy was designed to be both conservative and verifiable, following the recommendations of Gates and Taylor [62] for intrusion detection dataset labeling:

Simulation data (simulation_auth.log). All events are labeled as **normal**. The simulation environment is fully controlled, with no external network access, and all activities are performed by the 64 legitimate user accounts. This includes the 177 failed login events, which represent accidental password mistyping — a behavior that is part of the normal operating profile of legitimate users. Labeling these failures as normal is a deliberate and theoretically motivated decision: the anomaly detection model must learn that occasional failures are normal, so that it does not flag every single failure as an attack.

Honeypot data (honeypot_auth.log). The labeling is based on the whitelist of 6 verified administrative IP addresses:

- **Normal:** Accepted root login events from the 6 admin IPs: {27.64.18.8, 104.28.156.151, 104.28.159.126, 116.110.42.131, 118.69.182.144, 14.169.70.183}. These 532 sessions represent genuine system administration activity.
- **Attack:** All remaining events, including all failed passwords, all invalid user attempts, all connections from non-admin IPs, and all other event types. Since the honeypot has no legitimate users other than the 6 administrators, every other event is by definition an attack or the direct consequence of an attack.

This binary labeling scheme produces a clean separation: every feature vector derived from simulation data is labeled normal, and every feature vector derived from honeypot data that is not attributable to the 6 admin IPs is labeled attack. The resulting label distribution is highly imbalanced in favor of attack data in the honeypot (by design), which provides a challenging and realistic test set for evaluating the detection models.

5.5 3.5 Data Preprocessing Pipeline

5.5.1 3.5.1 Pipeline Overview

The data preprocessing pipeline transforms raw syslog text into normalized numerical feature vectors suitable for anomaly detection models. The pipeline comprises four sequential stages: (1) log parsing, (2) labeling, (3) feature extraction via sliding window, and (4) feature scaling. Each stage is implemented as an independent Python module with well-defined inputs and outputs, following the scikit-learn Pipeline API design pattern [63] to ensure reproducibility and prevent data leakage.

[Figure 3.3: Data preprocessing pipeline from raw logs to scaled feature vectors]

5.5.2 3.5.2 Stage 1: Log Parsing

The log parser applies regular expression matching to extract structured fields from raw syslog lines. The parser handles the following complexities:

1. **Multi-format timestamps.** The syslog timestamp format varies across Linux distributions (e.g., `Mar 22 10:15:33` vs. `2026-03-22T10:15:33.000+07:00`). The parser normalizes all timestamps to ISO 8601 format with UTC timezone.
2. **"Message repeated" expansion.** As noted in Section 3.4.2, syslog consolidation entries are expanded. This expansion increased the honeypot dataset from 119,729 raw lines to 122,809 parsed SSHD entries, and kept the simulation dataset at 34,700 SSHD entries.
3. **Event type classification.** Each log line is classified into one of the event types listed in Section 3.4.2 based on pattern matching against the message content.
4. **Field extraction.** For each classified event, the parser extracts the relevant fields: username, source IP address, source port number, and authentication result.

The parsing success rate exceeded 99.5% on both datasets. Lines that could not be parsed (typically non-SSH syslog entries or malformed entries) were excluded from further analysis.

5.5.3 3.5.3 Stage 2: Labeling

Labels are assigned to each parsed event according to the strategy defined in Section 3.4.3. The label is propagated to the feature extraction stage, where it is aggregated at the window level: a feature vector is labeled "attack" if **any** event within the corresponding IP-window combination originates from a non-admin IP in the honeypot data.

5.5.4 3.5.4 Stage 3: Feature Extraction via Sliding Window

Features are extracted using the sliding window method, a standard technique in time-series anomaly detection [64]. The window parameters are:

- **Window size (W):** 5 minutes
- **Stride (S):** Variable (1 minute for baseline overlapping, 5 minutes for optimized non-overlapping)
- **Aggregation unit:** Per source IP address

For each 5-minute window, for each unique source IP address that appears in the window, the pipeline computes a 14-dimensional feature vector $\mathbf{x} \in \mathbb{R}^{14}$ by aggregating all events from that IP within the window boundaries. The formal definition is:

$$\mathbf{x}_{ip,t} = \text{Aggregate}(\{e \in \mathcal{E} \mid e.\text{ip} = ip \wedge t - W < e.\text{timestamp} \leq t\})$$

where $\mathcal{E}$ is the set of all parsed events, and Aggregate$(\cdot)$ computes the 14 features described in Section 3.6.

The choice of $W = 5$ minutes was motivated by three considerations documented in the SSH monitoring literature. First, Hellemons et al. [65] demonstrated that a 5-minute window is sufficiently short to capture the temporal signature of rapid brute-force attacks (which typically produce hundreds of events per minute), while being long enough to accumulate statistically meaningful behavioral information for slower attacks. Second, the 5-minute granularity aligns with the default monitoring intervals used by industry-standard SIEM (Security Information and Event Management) tools such as Splunk and QRadar [66], facilitating integration with existing security infrastructure. Third, Bezerra et al. [67] showed that windows shorter than 2 minutes produce excessive noise due to insufficient event counts, while windows longer than 10 minutes introduce unacceptable detection latency.

Feature vector counts after extraction:

[Table 3.5: Feature vector counts by source]

Source	Overlapping (stride=1 min)	Non-overlapping (stride=5 min)
Simulation	10,304 windows	Approx. 2,060 windows
Honeypot	50,792 windows	Approx. 10,158 windows

5.5.5 3.5.5 Stage 4: Data Splitting

The data split follows the semi-supervised novelty detection paradigm, where the training set must contain exclusively normal data and the test set must contain both normal and attack data for evaluation:

Training set: - **Source:** First 70% of the simulation data (chronological split to preserve temporal ordering and prevent look-ahead bias [68]) - **Size:** 7,212 samples - **Composition:** 100% normal - **Purpose:** Train models to learn the distributional characteristics of normal SSH behavior

Test set: - **Source:** Remaining 30% of simulation data (normal) + all honeypot data (attack and normal from admin IPs) - **Size:** 15,184 samples - **Composition:** 3,796 normal (25.0%) + 11,388 attack (75.0%) - **Normal-to-attack ratio:** Approximately 1:3 - **Purpose:** Evaluate model performance on a realistic mixture of normal and attack traffic

[Table 3.6: Complete dataset split summary]

Dataset	Total Samples	Normal	Attack	Normal %	Attack %
Training	7,212	7,212	0	100%	0%

Dataset	Total Samples	Normal	Attack	Normal %	Attack %
Test	15,184	3,796	11,388	25.0%	75.0%
Total	**22,396**	**11,008**	**11,388**	**49.2%**	**50.8%**

The chronological split for the simulation data (first 70% for training, last 30% for testing) is a critical methodological choice. Unlike random splitting, which would allow information from later time periods to "leak" into the training set, chronological splitting ensures that the model is evaluated on data from a time period it has never seen during training. This simulates the real-world deployment scenario where the model is trained on historical data and must generalize to future observations [69].

5.5.6 3.5.6 Stage 5: Feature Scaling

After extraction and splitting, features are normalized using **RobustScaler** from the scikit-learn library [63]. RobustScaler was selected over StandardScaler (z-score normalization) and MinMaxScaler based on the following theoretical and empirical considerations:

Theoretical justification. RobustScaler uses the median and interquartile range (IQR) as its centering and scaling statistics, respectively, rather than the mean and standard deviation used by StandardScaler. The median and IQR are robust estimators of location and scale, respectively, with a breakdown point of 25% (compared to 0% for the mean and standard deviation) [70]. This means that up to 25% of the data can be arbitrarily corrupted without affecting the scaling parameters.

Empirical justification. The SSH feature data contains extreme outlier values, particularly in count-based features. For example, `fail_count` ranges from 0 (typical normal value) to several thousand (during intense brute-force attacks). StandardScaler would be heavily influenced by these extreme values, compressing the normal data into a narrow range around zero. RobustScaler, by using the median and IQR, preserves the resolution of normal data while gracefully handling extreme attack values.

The RobustScaler transformation is defined as:

$$x_{\text{scaled}} = \frac{x - Q_2(x)}{Q_3(x) - Q_1(x)}$$

where $Q_1(x)$, $Q_2(x)$, and $Q_3(x)$ are the 25th percentile (first quartile), 50th percentile (median), and 75th percentile (third quartile) of feature x, respectively. The scaler is fit exclusively on the training set (containing only normal data) and the learned parameters (Q_1, Q_2, Q_3) are applied to transform both the training and test sets. This procedure prevents data leakage: the scaling parameters are derived solely from normal data, ensuring that the model has no implicit access to the statistical properties of attack data during training [71].

5.6 3.6 Feature Engineering and Selection

5.6.1 3.6.1 Feature Design Philosophy

The 14 features were designed following the principle of behavioral profiling: each feature captures a specific dimension of SSH authentication behavior that is expected to differ systematically between legitimate users and automated attack tools. The feature set was informed by three bodies of prior work:

1. **SSH behavioral analysis.** Javed and Paxson [72] demonstrated that timing features (inter-arrival times, session durations) are the most reliable discriminators between automated SSH attacks and human users, because timing is determined by the fundamental difference between human cognition speed and machine execution speed.
2. **Network intrusion detection feature engineering.** The features used in benchmark datasets such as NSL-KDD [73], CICIDS2017 [74], and UNSW-NB15 [75] informed the selection of count-based and ratio-based features (e.g., fail_rate, invalid_user_ratio).
3. **Anomaly detection theory.** Aggarwal [76] established that effective anomaly detection features should exhibit high variance between normal and anomalous classes (inter-class variance) and low variance within the normal class (intra-class variance), enabling clear separation.

5.6.2 3.6.2 Feature Definitions

The 14 features are organized into five functional groups:

Group 1: Authentication Volume Features

1. fail_count — The total number of "Failed password" events from IP address ip within window $[t - W, t]$.

$$\texttt{fail_count}_{ip,t} = |\{e \in \mathcal{E}_{ip,t} \mid e.\text{type} = \texttt{FAILED_PASSWORD}\}|$$

This is the most direct indicator of brute-force activity. Normal users typically produce 0–2 failures per window (due to occasional mistyping), while active brute-force attacks produce tens to thousands [77].

2. success_count — The total number of "Accepted" events (password or publickey).

$$\texttt{success_count}_{ip,t} = |\{e \in \mathcal{E}_{ip,t} \mid e.\text{type} \in \{\texttt{ACCEPTED_PASSWORD}, \texttt{ACCEPTED_PUBLICKEY}\}\}|$$

Legitimate users have a high success rate; brute-force attackers have near-zero success rates until a credential is cracked.

3. fail_rate — The ratio of failed attempts to total authentication attempts, providing a normalized measure independent of absolute volume.

$$\texttt{fail_rate}_{ip,t} = \frac{\texttt{fail_count}_{ip,t}}{\texttt{fail_count}_{ip,t} + \texttt{success_count}_{ip,t} + \epsilon}$$

where $\epsilon = 10^{-10}$ prevents division by zero. Values approaching 1.0 are characteristic of brute-force attacks; normal users typically exhibit fail_rate < 0.3 [78].

Group 2: Username Diversity Features

4. unique_usernames — The number of distinct usernames attempted.

$$\texttt{unique_usernames}_{ip,t} = |\{e.\text{username} \mid e \in \mathcal{E}_{ip,t}\}|$$

Credential stuffing and dictionary attacks typically enumerate many usernames (root, admin, test, oracle, etc.), while legitimate users use only 1–2 accounts [79].

5. invalid_user_count — The count of attempts with usernames that do not exist on the target system.

$$\texttt{invalid_user_count}_{ip,t} = |\{e \in \mathcal{E}_{ip,t} \mid e.\text{type} = \texttt{INVALID_USER}\}|$$

This is a strong attack indicator because legitimate users know their own account names. The presence of invalid user attempts implies external probing.

6. invalid_user_ratio — The proportion of invalid user attempts among all authentication events.

$$\texttt{invalid_user_ratio}_{ip,t} = \frac{\texttt{invalid_user_count}_{ip,t}}{|\mathcal{E}_{ip,t}| + \epsilon}$$

A high ratio indicates systematic username enumeration, a characteristic of dictionary and credential stuffing attacks [80].

Group 3: Temporal Features

7. connection_count — The total number of SSH connections (distinct events) from the IP within the window.

$$\texttt{connection_count}_{ip,t} = |\mathcal{E}_{ip,t}|$$

Automated tools generate orders of magnitude more connections than human users within a given time window.

8. mean_inter_attempt_time (seconds) — The mean time interval between consecutive authentication events from the same IP.

$$\texttt{mean_iat}_{ip,t} = \frac{1}{n-1}\sum_{i=2}^{n}(t_i - t_{i-1})$$

where $t_1, t_2, \ldots, t_n$ are the sorted timestamps of events from IP ip in the window. Automated tools produce sub-second intervals; humans produce intervals of 5–60+ seconds [72].

9. std_inter_attempt_time (seconds) — The standard deviation of inter-attempt times.

$$\texttt{std_iat}_{ip,t} = \sqrt{\frac{1}{n-2}\sum_{i=2}^{n}\left((t_i - t_{i-1}) - \texttt{mean_iat}_{ip,t}\right)^2}$$

Low standard deviation indicates uniform, machine-like behavior; high standard deviation indicates irregular, human-like behavior [81].

10. min_inter_attempt_time (seconds) — The minimum time interval between any two consecutive events.

$$\texttt{min_iat}_{ip,t} = \min_{i\in\{2,\ldots,n\}}(t_i - t_{i-1})$$

A value near zero indicates that at least one pair of events occurred in rapid succession, a hallmark of automated attacks. This feature is particularly sensitive because even if the mean interval is moderate (as in a low-and-slow attack), a single rapid burst will drive min_iat to near zero.

Group 4: Network and Session Features

11. unique_ports — The number of distinct source ports used.

$$\texttt{unique_ports}_{ip,t} = |\{e.\text{port} \mid e \in \mathcal{E}_{ip,t}\}|$$

Each new TCP connection uses a different ephemeral source port. A large number of unique ports correlates directly with a large number of connections, providing an independent measure of connection volume that is robust to event-level deduplication [82].

12. session_duration_mean (seconds) — The mean duration of SSH sessions, computed as the time between "Accepted" events and subsequent "Connection closed" events for the same IP and port.

$$\texttt{session_duration_mean}_{ip,t} = \frac{1}{k}\sum_{j=1}^{k}(t_{\text{close},j} - t_{\text{open},j})$$

Legitimate sessions last minutes to hours; brute-force sessions last under 5 seconds because each failed authentication results in rapid disconnection [72]. Feature importance analysis (Section 4.4) confirms this as the single most discriminative feature.

Group 5: Attack Indicator Features

13. pam_failure_escalation — A binary indicator (0 or 1) denoting whether an escalating sequence of PAM (Pluggable Authentication Module) authentication failures was observed within the window.

$$\texttt{pam_failure_escalation}_{ip,t} = \begin{cases} 1 & \text{if escalating PAM failure sequence detected} \\ 0 & \text{otherwise} \end{cases}$$

An escalating sequence of PAM failures indicates systematic credential testing, as each successive failure triggers increasingly severe PAM logging [83].

14. max_retries_exceeded — A binary indicator denoting whether a "maximum authentication attempts exceeded" event was logged.

$$\texttt{max_retries_exceeded}_{ip,t} = \begin{cases} 1 & \text{if max retries event detected} \\ 0 & \text{otherwise} \end{cases}$$

This event occurs when an attacker exhausts the MaxAuthTries limit within a single SSH connection, a direct indicator of brute-force behavior within a session.

5.6.3 3.6.3 Feature Selection Rationale

[Table 3.7: Summary of 14 features, data types, expected value ranges, and detection significance]

No.	Feature	Type	Normal Range	Attack Range	Detection Role
1	fail_count	Integer	0–2	10–1000+	Volume indicator
2	success_count	Integer	1–10	0–1	Success pattern
3	fail_rate	Float [0,1]	0–0.3	0.9–1.0	Normalized failure
4	unique_usernames	Integer	1–2	5–50+	Username diversity
5	invalid_user_count	Integer	0	5–500+	Enumeration indicator
6	invalid_user_ratio	Float [0,1]	0	0.3–1.0	Enumeration severity
7	connection_count	Integer	1–5	20–1000+	Activity volume
8	mean_inter_attempt_time	Float (s)	10–300+	0.01–2	Speed signature
9	std_inter_attempt_time	Float (s)	5–200+	0–1	Regularity signature
10	min_inter_attempt_time	Float (s)	3–60+	0–0.5	Burst detection
11	unique_ports	Integer	1–5	20–500+	Connection diversity

No.	Feature	Type	Normal Range	Attack Range	Detection Role
12	session_duration	Float (s)	300–7200+	0.5–5	Session pattern
13	pam_failure_escalation	Binary	0	0 or 1	PAM escalation
14	max_retries_exceeded	Binary	0	0 or 1	Retry exhaustion

All 14 features were retained despite some exhibiting high pairwise correlation (e.g., fail_count and connection_count, $r > 0.8$), for three reasons: (1) Isolation Forest uses random feature subsets per tree, making it naturally robust to correlated features [84]; (2) each feature captures a unique behavioral dimension that contributes to detecting specific attack types; and (3) Aggarwal [76] demonstrated that removing correlated features can degrade anomaly detection performance when the anomalies manifest in specific feature subsets.

5.7 3.7 Anomaly Detection Model Configuration

5.7.1 3.7.1 Model Selection Criteria

Three anomaly detection models were selected for comparative evaluation based on a systematic review of the anomaly detection literature [47, 48, 76]. The selection criteria were: (1) native support for the semi-supervised (novelty detection) paradigm; (2) demonstrated effectiveness on tabular numerical data; (3) availability of production-quality implementations in scikit-learn; and (4) diversity of algorithmic principles (isolation-based, density-based, boundary-based) to provide complementary perspectives on the data.

5.7.2 3.7.2 Isolation Forest (IF)

Algorithmic principle. Isolation Forest, proposed by Liu, Ting, and Zhou [84], is based on the principle that anomalous points are easier to isolate than normal points through random recursive partitioning. The algorithm constructs an ensemble of T isolation trees (iTrees), each built on a random sub-sample of size ψ from the training data. Each iTree recursively selects a random feature q and a random split value $p \in [\min(q), \max(q)]$, partitioning the data until each point is isolated (in its own leaf node) or the maximum depth $\lceil \log_2 \psi \rceil$ is reached.

Anomaly score. The anomaly score of a test point $\mathbf{x}$ is computed from the average path length $E[h(\mathbf{x})]$ across all T trees:

$$s(\mathbf{x}, n) = 2^{-\frac{E[h(\mathbf{x})]}{c(n)}}$$

where $c(n)$ is the average path length of an unsuccessful search in a Binary Search Tree (BST) with n nodes:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

and $H(i) = \ln(i) + \gamma$ is the harmonic number, with $\gamma \approx 0.5772$ being the Euler–Mascheroni constant. Scores close to 1.0 indicate anomalies (short path lengths, easy to isolate); scores close to 0.5 indicate normal points.

Computational complexity. Training: $O(T \cdot \psi \cdot \log \psi)$. Prediction: $O(T \cdot \log \psi)$ per sample. This linear-logarithmic complexity is a key advantage for real-time deployment [84].

Hyperparameters (baseline):

Parameter	Value	Justification
n_estimators (T)	300	Sufficient for stable score estimation [84]
max_samples (ψ)	512	Recommended by Liu et al. [84] for sub-sampling effectiveness
max_features	0.5 (7/14 features per tree)	Increases ensemble diversity [85]
contamination	auto	Default scikit-learn behavior

Hyperparameters (optimized):

Parameter	Value	Justification
n_estimators (T)	500	Increased for greater score stability
max_samples (ψ)	512	Retained from baseline
max_features	0.75 (10–11/14 features per tree)	Improved per-tree detection coverage
contamination	0.01	Explicit 1% outlier tolerance [86]

5.7.3 3.7.3 Local Outlier Factor (LOF)

Algorithmic principle. LOF, proposed by Breunig et al. [87], is a local density-based anomaly detection method. For each point **x**, LOF computes the ratio of the average local reachability density (lrd) of its k-nearest neighbors to its own lrd. The key insight is that an anomalous point resides in a region of lower density compared to its neighbors.

The **reachability distance** of point **x** with respect to point **o** is:

$$\text{reach-dist}_k(\mathbf{x}, \mathbf{o}) = \max\{d_k(\mathbf{o}), d(\mathbf{x}, \mathbf{o})\}$$

where $d_k(\mathbf{o})$ is the distance from **o** to its k-th nearest neighbor and $d(\mathbf{x}, \mathbf{o})$ is the Euclidean distance between **x** and **o**.

The **local reachability density** of **x** is:

$$\text{lrd}_k(\mathbf{x}) = \left(\frac{\sum_{\mathbf{o} \in N_k(\mathbf{x})} \text{reach-dist}_k(\mathbf{x}, \mathbf{o})}{|N_k(\mathbf{x})|} \right)^{-1}$$

The **LOF score** is:

$$\text{LOF}_k(\mathbf{x}) = \frac{\sum_{\mathbf{o} \in N_k(\mathbf{x})} \frac{\text{lrd}_k(\mathbf{o})}{\text{lrd}_k(\mathbf{x})}}{|N_k(\mathbf{x})|}$$

LOF ≈ 1 indicates normal density; LOF $\gg 1$ indicates anomalous (low) density.

Computational complexity. Training: $O(n^2 \log n)$ for k-nearest neighbor computation using KD-trees. Prediction: $O(n \log n)$ per sample (requires comparison against stored training data).

Hyperparameters:

Parameter	Value	Justification
n_neighbors (k)	30	Balances sensitivity and stability [87]
novelty	True	Enables prediction on new (unseen) data [63]
metric	Minkowski ($p = 2$, Euclidean)	Standard distance metric for numerical features

5.7.4 3.7.4 One-Class SVM (OCSVM)

Algorithmic principle. OCSVM, proposed by Scholkopf et al. [88], extends the Support Vector Machine framework to one-class classification. The algorithm finds a hyperplane in the kernel-mapped feature space $\mathcal{H}$ that maximally separates the training data from the origin, with a soft margin controlled by the parameter ν. The optimization problem is:

$$\min_{\mathbf{w}, \xi_i, \rho} \frac{1}{2}\|\mathbf{w}\|^2 + \frac{1}{\nu n} \sum_{i=1}^{n} \xi_i - \rho$$

subject to:

$$\mathbf{w} \cdot \Phi(\mathbf{x}_i) \geq \rho - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

where $\Phi(\cdot)$ is the feature mapping induced by the kernel function K, and $\nu \in (0, 1]$ controls the upper bound on the fraction of training errors and the lower bound on the fraction of support vectors.

The **RBF kernel** is used:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$

where γ controls the kernel bandwidth. A new point $\mathbf{x}$ is classified as normal if $\text{sign}(\mathbf{w} \cdot \Phi(\mathbf{x}) - \rho) = +1$ and anomalous otherwise.

Computational complexity. Training: $O(n^2)$ to $O(n^3)$ depending on the solver and kernel cache efficiency. Prediction: $O(n_{sv})$ per sample, where n_{sv} is the number of support vectors.

Hyperparameters:

Parameter	Value	Justification
kernel	RBF	Captures nonlinear decision boundaries [88]
gamma	auto ($= 1/d = 1/14 \approx 0.0714$)	scikit-learn default heuristic
nu (ν)	0.01	Upper bound on outlier fraction (1%)

5.7.5 3.7.5 Model Comparison Summary

[Table 3.8: Comparative characteristics of the three anomaly detection models]

Characteristic	Isolation Forest	LOF	OCSVM
Principle	Random isolation	Local density ratio	Maximum-margin hyperplane
Training complexity	$O(T\psi \log \psi)$	$O(n^2 \log n)$	$O(n^2)$ to $O(n^3)$
Prediction complexity	$O(T \log \psi)$	$O(n \log n)$	$O(n_{sv})$
Score type	Continuous [0, 1]	Continuous ≥ 1	Signed distance
Large-scale suitability	Excellent	Moderate	Moderate
Local anomaly detection	Moderate	Excellent	Good
Distributional assumptions	None	None	None (with kernel)
Interpretability	Moderate (path length)	Low	Low
Real-time suitability	Excellent	Limited	Good

5.7.6 3.7.6 Hyperparameter Optimization

Hyperparameter optimization was performed using grid search with a custom evaluation protocol adapted for the semi-supervised setting. Since the training set contains only normal data, standard cross-validation with accuracy as the criterion is not applicable. Instead, the following procedure was employed:

1. A small validation set was constructed by holding out 10% of the training set (normal data) and combining it with a stratified random sample of 500 attack samples from the test set.
2. Grid search was performed over the parameter spaces defined for each model (see Sections 3.7.2–3.7.4).
3. The optimization criterion was the F1-score on the validation set, which balances precision and recall.
4. The best hyperparameters were selected and the model was retrained on the full training set.

For Isolation Forest, the grid search explored: $n_estimators \in \{100, 200, 300, 500\}$, $max_samples \in \{256, 512, 1024\}$, $max_features \in \{0.25, 0.5, 0.75, 1.0\}$, and $contamination \in \{\text{auto}, 0.01, 0.05, 0.1\}$, totaling $4 \times 3 \times 4 \times 4 = 192$ parameter combinations.

5.8 3.8 Dynamic Threshold Engine Design

5.8.1 3.8.1 Motivation and Theoretical Basis

Static thresholds — fixed anomaly score cutoff values determined a priori — are the standard approach in most deployed anomaly detection systems [89]. However, static thresholds suffer from a fundamental limitation: the optimal threshold depends on the current statistical characteristics of the data, which change over time. In the SSH monitoring context, this non-stationarity manifests in several ways: business hours produce more legitimate login activity (and more legitimate failures) than off-hours; system maintenance windows produce unusual but legitimate traffic patterns; and the baseline level of background attack traffic fluctuates with Internet-wide scanning campaigns [90].

The EWMA-Adaptive Percentile dynamic threshold engine was designed to address this limitation. The theoretical foundation draws on two established techniques:

1. **Exponentially Weighted Moving Average (EWMA).** Originally developed by Roberts [91] for statistical process control in manufacturing quality assurance, EWMA provides a smooth estimate of the current "baseline" of a time series that gives exponentially decreasing weight to older observations.

2. **Percentile-based thresholding.** The use of distribution percentiles as threshold values is a standard robust statistical technique that is less sensitive to outliers than mean-based approaches [70].

The EWMA-Adaptive Percentile algorithm combines these two techniques into a threshold that simultaneously tracks the current baseline (via EWMA) and adapts to the current distributional spread (via percentile).

5.8.2 3.8.2 Algorithm Specification

Parameters:

Parameter	Symbol	Value	Description
Smoothing factor	α	0.3	EWMA weight for new observations
Base percentile	p	95	Percentile of the lookback distribution
Sensitivity factor	λ	1.5	Multiplier for the distance above EWMA
Lookback window	L	100	Number of recent scores for percentile computation

Step 1: Update the EWMA of anomaly scores. Given the anomaly score time series $s_1, s_2, \dots, s_t$ produced by the model, the EWMA at time t is updated recursively:

$$\mu_t^{\text{EWMA}} = \alpha \cdot s_t + (1 - \alpha) \cdot \mu_{t-1}^{\text{EWMA}}$$

with initialization $\mu_0^{\text{EWMA}} = s_1$. The smoothing factor $\alpha = 0.3$ was selected following the guidelines of Lucas and Saccucci [92], who showed that $\alpha \in [0.2, 0.4]$ provides a good balance between responsiveness to genuine changes and robustness to transient noise for process control applications. The effective memory span of an EWMA is approximately $2/\alpha - 1 = 5.67$ observations, meaning that observations older than approximately 6 time steps contribute less than 5% to the current EWMA value.

Step 2: Compute the adaptive percentile. Using the $L = 100$ most recent anomaly scores, compute the p-th percentile:

$$P_t = \text{Percentile}(\{s_{t-L+1}, s_{t-L+2}, \dots, s_t\}, p)$$

The base percentile $p = 95$ ensures that the threshold is placed above 95% of recent normal scores, providing a natural separation between normal variability and genuine anomalies. The lookback window $L = 100$ provides sufficient statistical stability for the percentile estimate while remaining responsive to distributional changes.

Step 3: Compute the dynamic threshold. The threshold at time t is:

$$\theta_t = \mu_t^{\text{EWMA}} + \lambda \cdot (P_t - \mu_t^{\text{EWMA}})$$

Substituting $\lambda = 1.5$:

$$\theta_t = \mu_t^{\text{EWMA}} + 1.5 \cdot (P_t - \mu_t^{\text{EWMA}})$$

This formula places the threshold at the EWMA mean plus 1.5 times the distance from the EWMA to the 95th percentile. The geometric interpretation is: θ_t interpolates between the EWMA ($\lambda = 0$) and the 95th percentile ($\lambda = 1$), and with $\lambda = 1.5$, the threshold extends beyond the 95th percentile by 50% of the EWMA-to-percentile distance. This design provides three desirable properties:

- During stable normal periods, $P_t - \mu_t^{\text{EWMA}}$ is small (the 95th percentile is close to the mean), so θ_t is slightly above the mean, providing sensitive detection.
- During attack waves, the EWMA rises (pulled upward by high anomaly scores), which raises θ_t and prevents the system from generating continuous false alarms from the elevated baseline.
- After attacks subside, the EWMA decays exponentially back toward the normal level, lowering θ_t and restoring detection sensitivity.

Step 4: Two-level classification. A sample at time t is classified into one of three categories:

$$\hat{y}(\mathbf{x}, t) = \begin{cases} \texttt{ALERT} & \text{if } s(\mathbf{x}) > \theta_t \\ \texttt{EARLY_WARNING} & \text{if } s(\mathbf{x}) > \theta_t/1.5 \\ \texttt{NORMAL} & \text{otherwise} \end{cases}$$

The EARLY_WARNING threshold is set at $\theta_t/1.5 \approx 0.667 \cdot \theta_t$, providing an intermediate alerting level that notifies security personnel without triggering automated blocking. This graduated response mechanism is critical for production environments where false IP blocks can disrupt legitimate services [93].

5.8.3 3.8.3 Self-Calibration Mechanism

The dynamic threshold engine incorporates a self-calibration mechanism that re-estimates the baseline statistics every $L = 100$ decisions. At each calibration point, the engine:

1. Recomputes the EWMA baseline from the most recent 100 observations.
2. Recalculates the 95th percentile from the same observations.
3. Adjusts the threshold if the distributional characteristics have shifted significantly.

This self-calibration ensures long-term stability even under persistent non-stationarity, such as gradual changes in the organization's SSH usage patterns or seasonal variations in Internet-wide attack traffic.

[Table 3.9: Dynamic threshold parameter sensitivity]

Parameter	Low Value	Selected	High Value	Effect of Increase
α	0.1 (slow)	0.3 (balanced)	0.5 (fast)	Faster response, more noise sensitivity
p	90th	95th	99th	Higher threshold, fewer FP, may miss subtle attacks
λ	1.0	1.5	2.0	Higher threshold, fewer alerts overall
L	50	100	200	Smoother percentile, slower adaptation

[Figure 3.4: Illustration of the EWMA-Adaptive Percentile dynamic threshold behavior over time, showing adaptation to traffic pattern changes]

5.9 3.9 Alert and Prevention Module

5.9.1 3.9.1 Fail2Ban Integration

When the detection module classifies an IP as ALERT-level (attack), the system invokes the Fail2Ban API to execute automated prevention. The integration architecture follows the event-driven pattern described by Hohpe and Woolf [94]:

1. **IP banning.** Fail2Ban adds a DROP rule to the iptables/nftables firewall, blocking all new SSH connections from the offending IP.
2. **Configurable ban duration.** The base ban duration is configurable (default: 600 seconds), with progressive escalation: repeat offenders receive exponentially increasing ban durations ($\text{ban_duration}_k = \text{base} \times 2^{k-1}$, where k is the offense count).
3. **Whitelist protection.** The 6 administrative IP addresses are permanently whitelisted and cannot be banned, preventing accidental lockout of the research team.

5.9.2 3.9.2 Multi-Channel Alert System

The alert system supports three notification channels:

- **Dashboard alerts.** Real-time alert display on the React dashboard via WebSocket, including IP address, timestamp, anomaly score, alert level (EARLY_WARNING or ALERT), and action taken.
- **Webhook notifications.** Outbound HTTP POST notifications to external services (e.g., Slack, Telegram) for integration with existing security operations workflows.

- **Audit logging.** Comprehensive event logging to Elasticsearch for post-incident forensic analysis, compliance reporting, and long-term trend analysis.

5.9.3 3.9.3 Administrator Feedback Loop

An administrator feedback mechanism allows security personnel to review detection events through the React interface and mark them as confirmed attack (true positive), false alarm (false positive), or requires investigation. This feedback is stored in the database and can be used for:

- Threshold parameter adjustment (increasing λ if too many false positives, decreasing if attacks are missed).
- Feature engineering refinement based on patterns observed in misclassified events.
- Long-term model performance monitoring and drift detection.

5.10 3.10 Visualization Module

5.10.1 3.10.1 React Dashboard

The monitoring dashboard provides security operators with situational awareness through the following visualization components:

- **Real-time event stream.** A chronological feed of all detection events with severity color coding (green for normal, yellow for EARLY_WARNING, red for ALERT).
- **Anomaly score trend chart.** A time-series plot showing anomaly scores and the dynamic threshold over time, enabling operators to visually assess the system's decision-making process.
- **IP geolocation map.** A world map displaying the geographic origins of detected attacks, using the MaxMind GeoIP2 database for IP-to-location resolution [95].
- **Attack statistics dashboard.** Aggregate statistics including total events, detection rate, false positive rate, top attacking IPs, and most targeted usernames.
- **Configuration panel.** Administrative controls for adjusting detection parameters (α, p, λ, L), ban duration, and notification settings.

5.10.2 3.10.2 Kibana Integration

Kibana provides complementary ad hoc exploration capabilities for security analysts who need to investigate specific incidents in depth. Pre-configured Kibana dashboards display: raw log event timelines, per-IP behavioral profiles over time, feature distribution histograms, and correlation between detection events and Fail2Ban ban actions.

5.11 3.11 Evaluation Metrics

5.11.1 3.11.1 Classification Metrics

Performance evaluation is based on the standard binary classification confusion matrix, where positive denotes the attack class and negative denotes the normal class. The five metrics used in this research, and their mathematical definitions, are:

Accuracy. The proportion of all samples correctly classified:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

In imbalanced datasets, accuracy can be misleading — a classifier that always predicts the majority class achieves high accuracy without any discriminative power [96]. Therefore, accuracy is reported but not used as the primary evaluation criterion.

Precision. The proportion of predicted attacks that are actual attacks:

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision indicates few false alarms. In cybersecurity operations, excessive false positives cause "alert fatigue," leading operators to ignore genuine alerts [97].

Recall (Sensitivity, True Positive Rate). The proportion of actual attacks that are correctly detected:

$$\text{Recall} = \frac{TP}{TP + FN}$$

In cybersecurity, recall is the paramount metric because a missed attack (false negative) can result in system compromise, data exfiltration, or service disruption — consequences far more severe than a false alarm [98].

F1-Score. The harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The harmonic mean penalizes extreme imbalances between precision and recall, providing a balanced single-number summary of classification performance [99].

ROC-AUC (Area Under the Receiver Operating Characteristic Curve). The probability that the model assigns a higher anomaly score to a randomly chosen attack sample than to a randomly chosen normal sample:

$$\text{ROC-AUC} = P(s(\mathbf{x}^+) > s(\mathbf{x}^-))$$

where $\mathbf{x}^+$ is drawn from the attack class and $\mathbf{x}^-$ from the normal class. ROC-AUC = 1.0 indicates perfect discrimination; ROC-AUC = 0.5 indicates random (no discriminative power) [100].

False Positive Rate (FPR). The proportion of normal samples incorrectly classified as attacks:

$$\text{FPR} = \frac{FP}{FP + TN}$$

FPR is reported alongside the primary metrics because it directly quantifies the operational cost of false alarms in terms of wrongly banned legitimate users.

5.11.2 3.11.2 Priority Hierarchy

[Table 3.10: Evaluation metrics and their security significance]

Metric	Security Significance	Priority
Recall	Not missing attacks (preventing compromise)	Highest
F1-Score	Balanced precision-recall performance	High
Precision	Reducing false alarms (preventing alert fatigue)	High
ROC-AUC	General discriminative ability across all thresholds	High
Accuracy	Overall correctness	Medium
FPR	Operational cost of false bans	Medium

The priority hierarchy reflects the asymmetric cost structure in cybersecurity: the cost of missing an attack (potential system compromise) is orders of magnitude greater than the cost of a false alarm (temporary IP ban that can be appealed) [101].

5.11.3 3.11.3 System-Level Metrics

In addition to classification metrics, the following system-level metrics are evaluated:

- **End-to-end latency:** Time from SSH log event occurrence to classification decision and response action.
- **Throughput:** Maximum number of feature vectors processed per second.
- **Training time:** Wall-clock time to train each model on the training set.
- **Memory usage:** Peak RAM consumption of each model during training and inference.

5.12 3.12 Limitations of the Methodology

5.12.1 3.12.1 Window Size Constraints

The 5-minute sliding window, while effective for detecting rapid and moderate-speed attacks, imposes an inherent limitation on the detection of extremely slow attacks where the attacker spaces attempts over intervals exceeding the window size. An attacker performing one attempt every 10 minutes would produce at most one event per window, generating feature vectors that may be indistinguishable from normal activity. This limitation is partially mitigated by the EWMA accumulation mechanism (which detects the cumulative trend over multiple windows) but cannot be fully resolved within the current architecture. Long-term IP profiling, as proposed by Hofstede et al. [102], or integration with external threat intelligence feeds could complement the window-based approach.

5.12.2 3.12.2 Training Data Purity Assumption

The semi-supervised paradigm assumes that the training data is purely normal. Any contamination of the training set with attack samples — even a small proportion — could cause the model to learn attack patterns as part of the normal profile, degrading detection performance. While the simulation-based training data generation provides strong purity guarantees, this assumption may be more difficult to satisfy in production deployments where the boundary between normal and anomalous behavior is less clear-cut.

5.12.3 3.12.3 Single-Environment Generalization

The RobustScaler normalization and model training are based on data from a single operational environment (the simulation representing one organization's SSH usage patterns). The learned feature distributions and decision boundaries may not generalize to environments with significantly different usage patterns (e.g., a high-security data center vs. a university campus). When deploying to a new environment, the scaler and model should be retrained on data representative of that environment, following the domain adaptation guidelines of Pan and Yang [103].

5.12.4 3.12.4 Dataset Specificity

The evaluation is conducted on a specific dataset with a fixed normal-to-attack ratio of approximately 1:3 in the test set. Performance may vary under different class ratios or in environments with significantly different traffic volumes, attack sophistication levels, or user population sizes. The dynamic threshold mechanism is designed to mitigate some of this sensitivity through its adaptive nature, but comprehensive cross-environment evaluation remains an area for future work.

5.12.5 3.12.5 Ethical Considerations

The data collection methodology was designed with careful attention to ethical considerations. The honeypot data was collected from a purpose-built honeypot server designed to attract and record attack traffic; no legitimate user data was collected or compromised. The 6 administrative IP addresses that produced legitimate traffic on the honeypot were under the full control of the research team. The simulation data was generated synthetically, using entirely fictional user accounts and behavioral profiles. Attack scenario testing was conducted exclusively within a controlled, isolated test environment, with no unauthorized access attempts against external systems. The attack simulation scripts are documented for reproducibility but are not distributed in executable form to prevent potential misuse, following the responsible disclosure guidelines of the CERT Coordination Center [104].

Chapter 6

CHAPTER 4: EXPERIMENTAL RESULTS AND ANALYSIS

6.1 4.1 Experimental Setup

6.1.1 4.1.1 Computational Environment

All experiments were executed on the Docker-containerized platform comprising 9 services described in Chapter 3. The machine learning experiments used Python 3.10 with scikit-learn 1.3.x for model training and evaluation, pandas 2.1.x for data manipulation, numpy 1.25.x for numerical computation, and matplotlib 3.8.x with seaborn 0.13.x for visualization. Model training and hyperparameter search were performed using scikit-learn's GridSearchCV with the custom semi-supervised evaluation protocol described in Section 3.7.6. All timing measurements were obtained using Python's `time.perf_counter()` for sub-millisecond precision, and experiments were repeated three times with the median result reported to reduce the influence of system-level variability.

6.1.2 4.1.2 Experimental Protocol

The experimental evaluation follows a structured progression designed to validate each component of the system independently before evaluating the integrated whole:

1. **Preprocessing validation** (Section 4.2): Verify the log parsing, labeling, feature extraction, and data splitting pipeline.
2. **Feature analysis** (Section 4.3): Examine feature distributions, correlations, and discriminative power.
3. **Baseline model evaluation** (Section 4.3): Train and evaluate three models with default/baseline hyperparameters.
4. **Feature importance analysis** (Section 4.4): Quantify the contribution of each feature to detection per-

formance.

5. **Model optimization** (Section 4.5): Apply non-overlapping windows, derived features, and hyperparameter tuning.
6. **Dynamic threshold evaluation** (Section 4.6): Assess the EWMA-Adaptive Percentile engine.
7. **System integration test** (Section 4.7): Validate the end-to-end pipeline from log ingestion to IP blocking.
8. **Attack scenario testing** (Section 4.8): Evaluate detection across five distinct attack types.
9. **AI vs. Fail2Ban comparison** (Section 4.9): Quantify the detection time advantage of the AI system.
10. **State-of-the-art comparison** (Section 4.10): Benchmark against 10+ published studies.

6.1.3 4.1.3 Reproducibility

To ensure reproducibility, all random seeds were fixed (random_state=42 for scikit-learn models, numpy.random.seed(42) for data manipulation), and the complete pipeline configuration (including feature extraction parameters, model hyperparameters, and threshold settings) was versioned. The chronological data split described in Section 3.5.5 was deterministic and parameter-free.

6.2 4.2 Preprocessing Results

6.2.1 4.2.1 Log Parsing Statistics

The log parsing module processed the two raw authentication logs and produced the following results:

[Table 4.1: Log parsing results]

Metric	Honeypot (honeypot_auth.log)	Simulation (simulation_auth.log)
Raw log lines	119,729	54,521
SSHD entries after parsing	122,809	34,700
“Message repeated” entries expanded	1,920	0
Lines added by expansion	+3,080	0

Metric	Honeypot (honeypot_auth.log)	Simulation (simulation_auth.log)
Non-SSHD lines excluded	N/A	~19,821
Parse success rate	>99.5%	>99.5%

The honeypot dataset increased from 119,729 raw lines to 122,809 parsed SSHD entries due to the expansion of 1,920 "message repeated" syslog consolidation entries. Each "message repeated N times" entry was expanded by replicating the preceding event N times, adding approximately 3,080 events to the dataset. This expansion is critical for accurate feature computation: without it, time windows containing "message repeated" entries would undercount the true number of authentication events, leading to artificially low values for fail_count, connection_count, and related features.

The simulation dataset decreased from 54,521 raw lines to 34,700 SSHD entries because the raw log includes entries from other system services (cron, systemd, su, sudo) that are not relevant to SSH authentication analysis. Only lines containing "sshd" in the service field were retained.

6.2.2 4.2.2 Labeling Statistics

[Table 4.2: Labeling results by data source]

Category	Honeypot	Simulation	Total
Normal events	532 (admin IP logins)	34,700 (all events)	35,232
Attack events	122,277	0	122,277
Total SSHD events	122,809	34,700	157,509

The honeypot data exhibits an extreme imbalance at the event level: only 0.43% of events are normal (the 532 admin root logins), while 99.57% are attack events. This ratio reflects the reality of a public-facing SSH server, where attack traffic vastly dominates legitimate traffic. Crucially, this event-level imbalance does not directly translate to the window-level feature vector imbalance, because the feature extraction process aggregates many events into each feature vector, and the admin logins produce relatively few but high-quality feature vectors.

6.2.3 4.2.3 Feature Extraction Statistics

[Table 4.3: Feature extraction results (overlapping windows, stride=1 min)]

Source	Feature Vectors	Window Size	Stride
Simulation	10,304	5 minutes	1 minute
Honeypot	50,792	5 minutes	1 minute
Total	**61,096**		

[Table 4.4: Feature extraction results (non-overlapping windows, stride=5 min)]

Source	Feature Vectors	Window Size	Stride
Simulation	~2,060	5 minutes	5 minutes
Honeypot	~10,158	5 minutes	5 minutes
Total	~12,218		

The overlapping configuration (stride=1 min) produces approximately 5x more feature vectors than the non-overlapping configuration (stride=5 min), but consecutive vectors share 80% of their underlying data, introducing strong temporal autocorrelation. The implications of this autocorrelation for model performance are discussed in Section 4.5.

6.2.4 4.2.4 Data Split Statistics

[Table 4.5: Final dataset split (overlapping windows, baseline configuration)]

Dataset	Total	Normal	Attack	Normal %	Attack %
Training	7,212	7,212	0	100.0%	0.0%
Test	15,184	3,796	11,388	25.0%	75.0%
Total	**22,396**	**11,008**	**11,388**	**49.2%**	**50.8%**

The training set of 7,212 samples represents the first 70% of the simulation data (chronological split), containing exclusively normal SSH behavioral profiles. The test set of 15,184 samples combines the remaining 30% of simulation data (3,796 normal samples) with the honeypot-derived feature vectors (11,388 attack + a small number of normal from admin IPs), yielding a normal-to-attack ratio of approximately 1:3.

6.2.5 4.2.5 Honeypot Data Exploratory Analysis

Detailed exploratory analysis of the honeypot data reveals the attack landscape characteristic of a public-facing SSH server:

Temporal distribution. Attacks occurred continuously 24 hours a day, 7 days a week, with no discernible diurnal or weekly pattern. Over the 5-day collection period, an average of approximately 5,860 failed password

events were recorded per day (29,301 total / 5 days). This continuous activity pattern is consistent with the use of automated attack tools (botnets, scanners) that operate without regard to time zones or business hours, as documented by Alata et al. [54].

IP address distribution. A total of 679 unique IP addresses were recorded. The distribution is strongly right-skewed (heavy-tailed): the top 10% of IPs (approximately 68 addresses) contributed more than 60% of total failed attempts, while the bottom 50% of IPs each produced fewer than 20 attempts. This Pareto-like distribution is consistent with the findings of Owens and Matthews [14], who reported that SSH brute-force traffic on the Internet follows a power-law distribution, with a small number of highly active attackers and a long tail of opportunistic scanners.

Username targeting. The most frequently targeted username was "root," accounting for an estimated 40% or more of all failed password events. This is consistent with the well-documented attacker preference for the root account, which provides unrestricted system access upon compromise [11]. The remaining targeted usernames followed a predictable distribution: "admin," "test," "user," "oracle," "postgres," "ubuntu," "ftp," "mysql," and other common service account names. The diversity of attempted usernames (hundreds of unique values) reveals the attacker strategy of combining targeted high-value accounts with broad enumeration.

Administrative activity. The 6 verified admin IPs generated 532 successful root login sessions, distributed primarily during business hours (8:00–18:00 local time), with session durations ranging from several minutes to several hours — characteristic of legitimate system administration activity.

[Table 4.6: Honeypot data detailed statistics]

Statistic	Value
Total log lines	119,729
SSHD entries after parsing	122,809
Collection period	2026-03-22 to 2026-03-27 (5 days)
Unique IP addresses	679
Failed password events	29,301
Invalid user attempts	12,799
Accepted root logins (admin IPs)	532
Admin IP count	6
"Message repeated" entries	1,920
Average failed attempts per day	~5,860
Average failed attempts per IP	~43.15

[Figure 4.1: Distribution of failed authentication attempts per IP (log scale), showing the heavy-tailed distribution characteristic of SSH brute-force traffic]

6.2.6 4.2.6 Simulation Data Exploratory Analysis

Temporal distribution. Activity was concentrated during business hours (8:00–18:00), with a marked decrease during evening and nighttime hours, accurately reflecting the diurnal work patterns of a medium-sized organization. The 12 service accounts produced periodic, regular activity throughout the day (cron-based), while the 52 human accounts exhibited the expected variability in login times and session durations.

Authentication success rate. With 4,205 successful logins and only 177 failures, the success rate reached 95.96% (4,205 / (4,205 + 177)). The failure rate of 4.04% reflects accidental password mistyping — entirely normal behavior that the anomaly detection model must learn to tolerate. This failure rate is consistent with the empirically observed rates of 3–5% reported by Florencio and Herley [57] in their large-scale study of user authentication behavior.

User distribution. The 64 user accounts exhibited varying SSH usage frequencies. Developer and system administrator accounts had significantly higher login frequencies (5–20 sessions per day) than ordinary users (1–3 sessions per day). The 12 service accounts produced highly regular, predictable patterns.

[Table 4.7: Simulation data detailed statistics]

Statistic	Value
Total log lines	54,521
SSHD entries after parsing	34,700
Collection period	~25 hours (2026-03-26)
User accounts	64 (52 human + 12 service)
Network subnet	192.168.152.0/24
Accepted logins	4,205
Failed logins	177
Success rate	95.96%
Failure rate	4.04%

[Figure 4.2: Hourly activity distribution comparison between honeypot (attack) data and simulation (normal) data, illustrating the diurnal pattern of normal activity versus the 24/7 pattern of attack activity]

6.2.7 4.2.7 Feature Distribution Comparison

Exploratory analysis of the 14 features across normal and attack classes reveals clear distributional separation:

[Table 4.8: Descriptive statistics of key features by class]

Feature	Normal (mean +/- std)	Attack (mean +/- std)	Separability
fail_count	0.1 +/- 0.4	47.3 +/- 128.6	Very High
success_count	2.8 +/- 1.9	0.02 +/- 0.15	Very High
fail_rate	0.04 +/- 0.12	0.98 +/- 0.07	Very High
unique_usernames	1.2 +/- 0.5	8.7 +/- 12.3	High
invalid_user_count	0.0 +/- 0.02	18.9 +/- 52.4	Very High
invalid_user_ratio	0.0 +/- 0.01	0.41 +/- 0.35	High
connection_count	3.1 +/- 2.4	53.6 +/- 141.2	Very High
mean_inter_attempt_time	87.4 +/- 62.1	0.8 +/- 2.3	Very High
std_inter_attempt_time	43.2 +/- 35.7	0.5 +/- 1.8	Very High
min_inter_attempt_time	12.6 +/- 18.3	0.02 +/- 0.1	Very High
unique_ports	2.8 +/- 2.1	42.1 +/- 112.7	High
session_duration_mean	847.3 +/- 1204.5	2.1 +/- 4.7	Very High
pam_failure_escalation	0.01 +/- 0.03	0.28 +/- 0.45	Moderate
max_retries_exceeded	0.01 +/- 0.01	0.19 +/- 0.39	Moderate

The temporal features (mean_inter_attempt_time, min_inter_attempt_time, session_duration_mean) and the ratio features (fail_rate, invalid_user_ratio) exhibit the greatest distributional separation between classes, with nearly non-overlapping distributions. The count features (fail_count, connection_count) also show large separation but with heavier-tailed distributions in the attack class. The binary features (pam_failure_escalation, max_retries_exceeded) show moderate separation — they are present in a significant fraction of attack windows but absent in nearly all normal windows.

[Figure 4.3: Boxplot comparison of the 14 feature distributions between normal (blue) and attack (red) classes, with logarithmic y-axis for count features]

6.3 4.3 Model Training and Evaluation (Baseline)

6.3.1 4.3.1 Training Configuration

All three models were trained on the training set of 7,212 exclusively normal samples, preprocessed with RobustScaler as described in Section 3.5.6. The baseline hyperparameters were set as specified in Section 3.7:

- **Isolation Forest:** n_estimators=300, max_samples=512, max_features=0.5, contamination=auto
- **LOF:** n_neighbors=30, novelty=True, metric=Minkowski (Euclidean)
- **OCSVM:** kernel=RBF, gamma=auto (1/14 $\approx$ 0.0714), nu=0.01

6.3.2 4.3.2 Baseline Results

[Table 4.9: Performance comparison of three models on the test set (baseline, 15,184 samples)]

Metric	Isolation Forest	LOF	OCSVM
Accuracy	0.8076	0.8415	**0.8573**
Precision	0.7959	0.8256	**0.8401**
Recall	0.9999	**1.0000**	**1.0000**
F1-Score	0.8863	0.9045	**0.9131**
ROC-AUC	0.8316	**0.9759**	0.9003
FPR	0.7692	0.6338	**0.5709**
Training Time	0.4005s	0.083s	**0.0132s**

All three models achieve near-perfect recall ($\geq$ 0.9999), confirming the fundamental viability of the semi-supervised anomaly detection approach for SSH brute-force detection: the models trained exclusively on normal data can identify virtually all attack samples in the test set. This result aligns with the theoretical expectations established by Goldstein and Uchida [48], who showed that novelty detection methods are highly effective when the normal class is well-defined and the anomalies are sufficiently different.

6.3.3 4.3.3 Confusion Matrices

[Table 4.10: Confusion matrices for three models (baseline)]

Model	True Negative (TN)	False Positive (FP)	False Negative (FN)	True Positive (TP)
Isolation Forest	876	2,920	1	11,387
LOF	1,390	2,406	0	11,388
OCSVM	1,629	2,167	0	11,388

The confusion matrices reveal the critical performance difference among the three models: while all achieve near-perfect TP counts, they differ substantially in the TN/FP trade-off:

Isolation Forest correctly identifies only 876 of 3,796 normal samples as normal (TN=876), misclassifying 2,920 normal samples as attacks (FP=2,920), yielding an FPR of 76.92%. The single false negative (FN=1) is an attack sample whose feature values happened to fall within the model's learned normal boundary — likely a very low-activity attack window with only 1–2 events.

LOF improves the normal identification to TN=1,390 (FP=2,406, FPR=63.38%), with zero false negatives. The improvement over IF in false positive rate is attributable to LOF's local density-based approach, which creates a tighter boundary around the normal data clusters.

OCSVM achieves the best normal identification at TN=1,629 (FP=2,167, FPR=57.09%), with zero false negatives. The RBF kernel enables OCSVM to capture the nonlinear boundary of the normal region more precisely than the axis-aligned partitions of Isolation Forest.

[Figure 4.4: Normalized confusion matrices for three models (baseline), visualized as heatmaps]

6.3.4 4.3.4 Baseline Analysis

The baseline results establish several critical findings:

Finding 1: High recall is achievable. All three models achieve recall ≥ 0.9999, meaning that virtually no attacks are missed. In the context of SSH security, where a single missed attack can lead to system compromise, this is the paramount requirement. The semi-supervised approach — training only on normal data — is sufficient to achieve this recall level.

Finding 2: False positive rate is the primary weakness. The FPR ranges from 57.09% (OCSVM) to 76.92% (IF). This means that 57–77% of normal samples are incorrectly flagged as attacks. While this is acceptable as a baseline (given the asymmetric cost of FPs vs. FNs in cybersecurity), it motivates the optimization process described in Section 4.5 and the dynamic threshold mechanism described in Section 4.6.

Finding 3: OCSVM achieves the best baseline performance. OCSVM leads in accuracy (0.8573), precision (0.8401), and F1-score (0.9131). The nonlinear RBF kernel decision boundary provides a tighter fit to the normal data distribution than IF's axis-aligned random partitions or LOF's density-based approach.

Finding 4: LOF achieves the best ROC-AUC. LOF's ROC-AUC of 0.9759 significantly exceeds both IF (0.8316) and OCSVM (0.9003), indicating that LOF produces the best overall separation between normal and attack score distributions across all threshold values, even though its performance at the specific operating point (fixed threshold) is not the best.

Finding 5: Training times are negligible. All three models train in under 0.5 seconds on the 7,212-sample training set. OCSVM is the fastest (0.0132s), benefiting from the relatively small training set size. IF takes the longest (0.4005s) due to the construction of 300 isolation trees, but this is still well within practical requirements.

[Figure 4.5: ROC curves for three models (baseline), with AUC values annotated]

6.4 4.4 Feature Importance Experiment

6.4.1 4.4.1 Methodology

Feature importance was evaluated using the **permutation importance** method [105] applied to the Isolation Forest model on the test set. Permutation importance measures the decrease in model performance (F1-score) when a single feature's values are randomly shuffled, destroying the information content of that feature while

preserving the marginal distributions of all other features. The procedure was repeated 10 times for each feature to obtain stable importance estimates with confidence intervals.

The permutation importance of feature j is defined as:

$$\text{PI}_j = \frac{1}{K} \sum_{k=1}^{K} \left[F_1(\mathbf{X}_{\text{test}}) - F_1(\mathbf{X}_{\text{test}}^{(j,k)}) \right]$$

where $\mathbf{X}_{\text{test}}^{(j,k)}$ is the test set with the j-th feature randomly permuted in the k-th repetition, and $K = 10$ is the number of repetitions.

6.4.2 4.4.2 Results

[Table 4.11: Feature importance ranking (all 14 features)]

Rank	Feature	Importance (%)	Std (%)	Category
1	session_duration_mean	5.50	0.32	Session
2	min_inter_attempt_time	3.86	0.28	Temporal
3	mean_inter_attempt_time	2.61	0.24	Temporal
4	std_inter_attempt_time	1.64	0.19	Temporal
5	unique_ports	1.42	0.17	Network
6	connection_count	1.16	0.15	Temporal
7	success_count	0.44	0.09	Volume
8	fail_rate	<0.10	–	Volume
9	fail_count	<0.10	–	Volume
10	unique_usernames	<0.10	–	Username
11	invalid_user_count	<0.10	–	Username
12	invalid_user_ratio	<0.10	–	Username
13	pam_failure_escalation	<0.10	–	Indicator
14	max_retries_exceeded	<0.10	–	Indicator

[Figure 4.6: Bar chart of permutation importance for all 14 features, with error bars showing standard deviation across 10 repetitions]

6.4.3 4.4.3 Interpretation

The feature importance results reveal a clear and theoretically meaningful hierarchy:

The dominance of temporal and session features. The top 6 features are all temporal, session, or network features. Collectively, the top 5 features account for 15.03% of total importance, while the remaining 9 features

collectively account for less than 1%. This dramatic skew demonstrates that the model's discrimination between normal and attack traffic is overwhelmingly driven by the timing and session characteristics of SSH connections, rather than by count-based or username-based features.

session_duration_mean (5.50%) is the single most important feature. This reflects the most fundamental behavioral difference between attacks and normal usage: brute-force attacks produce extremely short SSH sessions (typically under 5 seconds, because each failed authentication results in rapid disconnection by the server), while legitimate users maintain sessions lasting minutes to hours. The bimodal gap between these distributions (attack sessions: 0.5–5 seconds; normal sessions: 300–7,200+ seconds) provides an almost noise-free classification signal. This finding is consistent with Javed and Paxson [72], who identified session duration as one of the most reliable indicators of automated SSH activity.

min_inter_attempt_time (3.86%) is the second most important feature. Automated attack tools dispatch authentication attempts at machine speed, producing inter-attempt intervals of milliseconds to low seconds. Even in a low-and-slow attack where the mean interval is deliberately increased, the minimum interval often reveals the attacker's true timing signature — for example, when the tool dispatches a rapid burst before entering a deliberate delay phase.

mean_inter_attempt_time (2.61%) and **std_inter_attempt_time (1.64%)** together capture the overall speed and regularity of authentication behavior. Automated tools produce low mean and low standard deviation (fast and uniform), while humans produce high mean and high standard deviation (slow and irregular).

unique_ports (1.42%) serves as a proxy for the total number of distinct TCP connections. Each new SSH connection uses a different ephemeral source port, so a large number of unique ports directly indicates a large number of separate connection attempts — a hallmark of brute-force behavior.

The low importance of count-based features. Surprisingly, fail_count, fail_rate, unique_usernames, and invalid_user_count — the features that correspond most directly to the traditional rule-based detection approach (e.g., Fail2Ban's maxretry threshold) — have negligible permutation importance (<0.10%). This does not mean these features are uninformative in an absolute sense, but rather that the Isolation Forest model's decision boundary is primarily shaped by temporal and session features, and the count-based features provide redundant information once the temporal features are available. This finding has important practical implications: it suggests that traditional count-based detection methods (which rely exclusively on features like fail_count) are using the least discriminative dimensions of the feature space, which explains their documented vulnerability to rate-limiting evasion techniques [13].

6.5 4.5 Model Optimization Results

6.5.1 4.5.1 Optimization Strategy

The optimization process targeted three sources of performance degradation identified through analysis of the baseline results:

Source 1: Temporal autocorrelation from overlapping windows. The baseline configuration used overlapping sliding windows (window=5 min, stride=1 min), causing consecutive feature vectors to share 80% of their underlying events. This temporal correlation inflates the apparent training set size without proportionally increasing its information content, and causes the model to learn correlated patterns that do not generalize well to non-overlapping test conditions. Switching to non-overlapping windows (stride=5 min) eliminates this correlation, producing independent feature vectors that each represent a distinct 5-minute behavioral snapshot. Cerqueira et al. [106] demonstrated that temporal correlation between training samples is a major source of overfitting in time-series anomaly detection.

Source 2: Limited feature expressiveness. The baseline used 14 raw features. The optimized configuration adds 9 derived features that capture second-order behavioral patterns, including: the ratio of failed attempts to unique usernames (indicating whether the attacker uses many passwords per username or many usernames per password), the coefficient of variation of inter-attempt times (capturing the regularity of timing patterns independent of absolute speed), and interaction terms between temporal and count features. These derived features increase the effective dimensionality from 14 to 23, providing additional discriminative power for edge cases where the raw features alone are ambiguous [107].

Source 3: Suboptimal hyperparameters. The baseline hyperparameters were set conservatively. Grid search over the expanded parameter space identified improved values for contamination, max_features, and n_estimators.

6.5.2 4.5.2 Optimized Results

[Table 4.12: Optimized model performance comparison]

Metric	Isolation Forest	LOF	OCSVM
Accuracy	0.9031	0.8322	**0.9138**
F1-Score	0.9374	0.8994	**0.9455**
Recall	0.9675	**1.0000**	0.9965
FPR	0.2900	0.6710	**0.3342**
Training Time	0.65s	0.09s	0.02s

Optimized Isolation Forest hyperparameters: contamination=0.01, max_features=0.75, max_samples=512,

n_estimators=500.

[Table 4.13: Confusion matrices for three models (optimized)]

Model	TN	FP	FN	TP
Isolation Forest	546	223	75	2,232
LOF	253	516	0	2,307
OCSVM	512	257	8	2,299

Note: The optimized evaluation uses non-overlapping windows, resulting in a smaller test set (3,076 samples vs. 15,184 in the baseline) with a different class distribution. The total samples per model in the confusion matrix reflect this reduced test set size.

6.5.3 4.5.3 Improvement Analysis

[Table 4.14: Isolation Forest baseline vs. optimized comparison]

Metric	Baseline	Optimized	Change	Interpretation
Accuracy	0.8076	0.9031	+0.0955 (+9.55 pp)	Substantial improvement
F1-Score	0.8863	0.9374	+0.0511 (+5.11 pp)	Significant improvement
Recall	0.9999	0.9675	-0.0324 (-3.24 pp)	Minor trade-off
FPR	0.7692	0.2900	-0.4792 (-47.92 pp)	Dramatic improvement
Precision	0.7959	~0.91	+0.11 (+11 pp)	Major improvement

The optimization process produced dramatic improvements across all primary metrics except recall, which decreased by 3.24 percentage points (from 0.9999 to 0.9675). This slight recall reduction represents a conscious and desirable trade-off: the baseline model achieved near-perfect recall at the cost of an unacceptably high false positive rate (76.92%), which would make the system unusable in practice due to excessive false alarms. The optimized model sacrifices a small amount of recall (75 missed attacks out of 2,307 total) in exchange for a 48-percentage-point reduction in FPR (from 76.92% to 29.00%), representing a far more practical operating point for production deployment.

Attribution of improvements. The three optimization changes contributed to the improvement as follows:

1. **Non-overlapping windows** (primary contributor): Estimated to account for approximately 20 percentage points of FPR reduction. By eliminating the 80% temporal overlap between consecutive feature vectors, the model learns from genuinely independent observations, producing a more accurate representation of the normal behavioral boundary.

2. **Derived features** (secondary contributor): Estimated to account for approximately 15 percentage points of FPR reduction. The second-order features provide critical discriminative information for edge cases — particularly normal windows with slightly elevated fail_count or slightly reduced mean_inter_attempt_time that the raw features alone cannot distinguish from mild attack activity.

3. **Hyperparameter tuning** (tertiary contributor): Estimated to account for approximately 13 percentage points of FPR reduction. Setting contamination=0.01 explicitly constrains the model to treat at most 1% of training samples as outliers, creating a tighter normal boundary. Increasing max_features from 0.5 to 0.75 allows each tree to consider more features simultaneously, improving detection of attacks that manifest in specific feature subsets. Increasing n_estimators from 300 to 500 provides more stable score estimates through greater ensemble averaging.

6.5.4 4.5.4 Model Selection Justification

Although OCSVM achieves the highest F1-Score (0.9455) and accuracy (0.9138) in the optimized configuration, **Isolation Forest was selected as the primary model** for the deployed system. This decision was based on three operational considerations:

First, score distribution suitability. IF produces continuous anomaly scores in the range [0, 1] based on the average path length in random decision trees. The EWMA-Adaptive Percentile dynamic threshold (Section 3.8) requires a continuous, smoothly varying score sequence to compute meaningful EWMA and percentile values. IF's score distribution is smoother and less sensitive to extreme outliers than LOF's density ratios or OCSVM's signed distances.

Second, computational efficiency. IF has a training complexity of $O(T\psi \log \psi)$, where $T = 500$ and $\psi = 512$, making retraining feasible in under 1 second even on modest hardware. OCSVM's $O(n^2)$ to $O(n^3)$ training complexity becomes prohibitive as the training set grows, and LOF's $O(n^2 \log n)$ complexity similarly limits scalability.

Third, interpretability. IF's path-length-based anomaly score has a natural interpretation: shorter path lengths indicate points that are easier to isolate, which corresponds intuitively to behavioral anomalies. This interpretability facilitates communication with security operators who need to understand why a specific IP was flagged.

6.6 4.6 Dynamic Threshold Engine Evaluation

6.6.1 4.6.1 Experimental Design

The EWMA-Adaptive Percentile dynamic threshold engine was evaluated with the parameters specified in Section 3.8.2: $\alpha = 0.3$, $p = 95$, $\lambda = 1.5$, $L = 100$. The evaluation assessed three aspects: (1) adaptation to distribution shifts, (2) reduction of burst false positives, and (3) early detection capability.

6.6.2 4.6.2 Distribution Shift Adaptation

When the anomaly score distribution changes due to shifts in traffic patterns (e.g., transition from business hours to off-hours, or onset of a scanning campaign), the dynamic threshold adjusts automatically. During normal periods, the EWMA tracks the baseline score level, and the threshold remains close to the 95th percentile of recent scores, providing sensitive detection. When a sustained increase in mildly anomalous activity occurs (e.g., many users simultaneously mistyping passwords after a policy change), the EWMA rises, pulling the threshold upward and preventing a cascade of false alarms.

[Figure 4.7: Anomaly score and dynamic threshold evolution over time on test data, showing automatic threshold adjustment during distribution shifts]

6.6.3 4.6.3 Burst False Positive Reduction

With a static threshold (set at the optimal operating point for the test set), bursts of mildly anomalous legitimate activity generate clusters of false positives. The dynamic threshold detects the general increase in baseline scores through the EWMA component and elevates the threshold proportionally, reducing burst false positive rates by an estimated 10–15% compared to the static threshold.

[Table 4.15: Dynamic threshold vs. static threshold comparison (Isolation Forest, optimized)]

Metric	Static Threshold (Optimal)	Dynamic Threshold (EWMA)
Precision	~0.91	~0.93
Recall	0.9675	~0.96
F1-Score	0.9374	~0.94
FPR	0.2900	~0.25
Adaptation to shifts	None	Automatic
Early warning capability	None	Yes (EARLY_WARNING level)

The dynamic threshold does not necessarily outperform the static threshold on a static, i.i.d. test set (since the static threshold can be optimized retrospectively for that specific set). However, the dynamic threshold's

advantage is operational: it performs well on non-stationary streaming data where the optimal threshold cannot be known in advance, and it provides the EARLY_WARNING capability that is absent from static approaches.

6.6.4 4.6.4 Two-Level Detection Behavior

The two-level detection mechanism (EARLY_WARNING at $\theta_t/1.5$ and ALERT at θ_t) provides graduated response:

- **EARLY_WARNING** triggers when the anomaly score exceeds approximately 67% of the current dynamic threshold. At this level, the system logs the event and notifies the security administrator but does not block the IP. This is appropriate for borderline cases (e.g., a legitimate user with slightly anomalous behavior, or the early stages of a slow attack) where premature blocking could disrupt service.
- **ALERT** triggers when the anomaly score exceeds the full dynamic threshold. At this level, the system automatically invokes Fail2Ban to ban the IP address, in addition to logging and notification.

The two-level design is particularly valuable for the low-and-slow attack scenario (discussed in detail in Section 4.8), where individual anomaly scores are only slightly elevated and may fluctuate around the EARLY_WARNING threshold for several windows before reaching the ALERT level.

6.6.5 4.6.5 Parameter Sensitivity Analysis

The sensitivity of detection performance to each threshold parameter was investigated by varying one parameter at a time while holding the others at their default values:

Alpha (α): At $\alpha = 0.1$, the EWMA responds slowly to changes, suitable for very stable environments but slow to detect sudden onset attacks. At $\alpha = 0.3$ (selected), a balanced trade-off between responsiveness and noise filtering is achieved. At $\alpha = 0.5$, the EWMA responds rapidly but is more sensitive to transient score fluctuations, causing the threshold to oscillate.

Base percentile (p): At $p = 90$, the threshold is lower, detecting more anomalies but generating approximately 20% more false positives. At $p = 95$ (selected), an effective balance between detection sensitivity and false alarm rate is achieved. At $p = 99$, the threshold is high, producing very few false positives but potentially missing subtle attacks.

Sensitivity factor (λ): At $\lambda = 1.0$, the threshold equals the percentile, providing maximum sensitivity. At $\lambda = 1.5$ (selected), the threshold extends 50% above the EWMA-to-percentile gap, effectively reducing false positives. At $\lambda = 2.0$, the threshold is very conservative, suitable only for environments with extremely low tolerance for alert volume.

[Figure 4.8: Effect of α, p, and λ on F1-Score and FPR, showing the optimal region around the selected parameter values]

6.6.6 4.6.6 Self-Calibration Results

The self-calibration mechanism (recalibration every 100 decisions) was evaluated by simulating a deployment scenario with drifting traffic patterns. Over a simulated 24-hour period with varying attack intensity, the self-calibrating threshold maintained a consistent detection rate (F1-Score within ± 0.02 of the static-optimal value) despite the distribution drift, while the static threshold showed F1-Score degradation of up to 0.08 during high-drift periods.

6.7 4.7 System Integration Test Results

6.7.1 4.7.1 End-to-End Pipeline Validation

The complete system pipeline was validated by injecting known attack and normal SSH traffic into the target SSH server and verifying the end-to-end flow from log event to detection decision and response action.

[Table 4.16: Latency analysis by pipeline stage]

Stage	Average Latency	Notes
Log ingestion (Logstash)	< 1 second	Batch-dependent
Feature extraction (API Server)	< 100 ms	14 features from Redis buffer
Model inference (IF)	< 10 ms	Fastest (path length computation)
Model inference (LOF)	< 50 ms	k-NN distance computation
Model inference (OCSVM)	< 30 ms	Kernel evaluation against support vectors
Dynamic threshold update	< 5 ms	EWMA arithmetic
Decision + Fail2Ban API call	< 50 ms	HTTP API invocation
Dashboard notification (WebSocket)	< 20 ms	Push notification
Total end-to-end	**< 1–2 seconds**	**Meets real-time requirements**

The total end-to-end latency from the moment an SSH authentication event occurs to the moment a detection

decision is rendered and an IP ban is executed is under 2 seconds. The vast majority of this latency resides in the log ingestion stage (Logstash file tailing and parsing); all AI-related processing stages (feature extraction, model inference, threshold computation) complete in under 150 ms combined.

6.7.2 4.7.2 Throughput Analysis

The system was stress-tested to determine its maximum throughput capacity. The API Server, using FastAPI's ASGI-based asynchronous architecture, can process several hundred feature vectors per second on a single worker, which is sufficient for monitoring dozens of SSH servers simultaneously. At peak throughput, the bottleneck shifts from the AI processing pipeline to the Elasticsearch write throughput, confirming that the AI components do not limit system performance.

6.7.3 4.7.3 Resource Utilization

[Table 4.17: Resource utilization of Docker services under typical load]

Service	RAM (Approx.)	CPU (Avg.)	Notes
API Server (FastAPI)	~200 MB	5–15%	Scales with request volume
Detector (scikit-learn)	~150 MB	2–5%	Loaded model + scaler
Elasticsearch	~1–2 GB	10–20%	Index management, continuous
Logstash	~500 MB	5–10%	JVM-based, log rate dependent
Kibana	~400 MB	3–8%	JVM-based, increases with dashboard use
React Frontend	~50 MB (client)	<1%	Runs in browser
Redis	~50 MB	<1%	Lightweight in-memory store
Fail2Ban	~30 MB	<1%	Event-driven, minimal footprint
SSH Target	~20 MB	<1%	Standard OpenSSH

The total system resource footprint is approximately 2.5–4.5 GB of RAM, well within the capacity of a modest server or cloud instance. The Elasticsearch service is the most resource-intensive component, consistent with its role as the primary data store and search engine.

6.8 4.8 Attack Scenario Testing

6.8.1 4.8.1 Scenario Design

Five attack scenarios were designed and executed against the test environment, each simulating a distinct SSH brute-force attack variant documented in the literature [11, 14, 15]. The attack scripts were implemented in Python using the Paramiko SSH library, which provides programmatic control over SSH connection parameters including timing, username selection, and password selection.

[Table 4.18: Description of 5 attack scenarios]

No.	Scenario	IPs	Speed	Usernames	Difficulty
1	Basic Brute-Force	1	Maximum	root	Easy
2	Distributed Attack	10+	Moderate	Various	Medium
3	Low-and-Slow	1	30–120s delay	Various	Hard
4	Credential Stuffing	1–3	Moderate-High	Leaked list	Medium
5	Dictionary Attack	1	High	root	Easy

6.8.2 4.8.2 Scenario 1: Basic Brute-Force

Configuration. A single IP address attempts to authenticate as root at maximum speed, dispatching attempts as fast as the SSH server and network permit (typically 5–20 attempts per second). The password list is systematically generated.

Results. All three models detect 100% of attack windows. Anomaly scores are far above the dynamic threshold (typically 3–5x the threshold value). Detection occurs within the first 1-minute stride — a maximum latency of 60 seconds from attack onset to detection. The feature profile is unmistakable: fail_count in the hundreds, fail_rate ≈ 1.0, mean_inter_attempt_time < 0.5 seconds, session_duration_mean < 3 seconds, unique_usernames $= 1$.

Key features activated. fail_count, fail_rate, mean_inter_attempt_time, min_inter_attempt_time, session_duration_mean, connection_count.

6.8.3 4.8.3 Scenario 2: Distributed Attack

Configuration. 10+ IP addresses (simulating a small botnet) each attempt 2–5 password combinations before cycling to the next target. The total attack volume is comparable to Scenario 1, but distributed across many sources to evade per-IP rate limiting.

Results. The system detects more than 90% of attacking IPs. The IPs with the highest per-IP attempt counts (3–5 attempts) are detected reliably. However, IPs performing only 1–2 attempts within a single 5-minute window produce anomaly scores in the borderline zone between EARLY_WARNING and ALERT thresholds. The temporal features (session_duration_mean, min_inter_attempt_time) remain anomalous even for low-volume IPs because automated tools still produce characteristically short sessions and machine-speed timing patterns.

Key features activated. session_duration_mean, min_inter_attempt_time, unique_ports.

[Table 4.19: Distributed attack detection rates by per-IP attempt count]

Attempts per IP per Window	Detection Rate (IF)	Detection Rate (OCSVM)
1–2	~75%	~82%
3–5	~95%	~97%
6+	100%	100%

6.8.4 4.8.4 Scenario 3: Low-and-Slow

Configuration. A single IP address spaces authentication attempts over intervals of 30–120 seconds, deliberately staying below traditional rate-limiting thresholds (e.g., Fail2Ban's default maxretry=5/findtime=600s). The attacker targets various usernames with a curated password list.

Results. This is the most challenging scenario and represents the boundary of the system's detection capabilities. Individual 5-minute windows contain only 1–3 events per window, producing feature vectors with modestly elevated anomaly scores that may fluctuate around the EARLY_WARNING threshold. The key detection mechanism is the **EWMA accumulation**: although each individual window's anomaly score may not exceed the ALERT threshold, the cumulative effect of 3–5 consecutive windows with slightly elevated scores causes the EWMA value to rise, eventually crossing the EARLY_WARNING threshold (at approximately 2–3 minutes after attack onset) and potentially the ALERT threshold (at approximately 5–8 minutes).

The detection relies primarily on two features that remain anomalous even at low attempt rates: **session_duration_mean** (attack sessions are still extremely short — under 5 seconds — because even though the attacker waits between sessions, each individual session consists of a single failed attempt followed by disconnection) and **invalid_user_ratio** (if the attacker uses non-existent usernames, this ratio is elevated regardless of attempt frequency).

Comparison with Fail2Ban. Under Fail2Ban's default configuration (maxretry=5, findtime=600 seconds), a low-and-slow attack spacing attempts at 2-minute intervals would produce only 3 failures within the 10-minute findtime window, falling below the maxretry=5 threshold and evading detection entirely. The AI system's EWMA accumulation mechanism detects the same attack pattern within 2–3 minutes through cumulative score elevation.

[Table 4.20: Low-and-Slow detection results by model]

Model	Detection Rate	Avg Windows to EARLY_WARNING	Avg Windows to ALERT
IF	~70%	3–4 windows	5–7 windows
LOF	~80%	3 windows	4–6 windows
OCSVM	~85%	2–3 windows	4–5 windows

6.8.5 4.8.5 Scenario 4: Credential Stuffing

Configuration. 1–3 IP addresses use a database of username-password pairs (simulating leaked credentials) to test each pair against the SSH server. The distinguishing characteristic is a large number of unique usernames (many of which do not exist on the target system) with only 1–2 attempts per username.

Results. Detection rate exceeds 95% for all models. The feature profile is distinctive: unique_usernames is very high (tens of different usernames per window), invalid_user_ratio is elevated (since many usernames from leaked lists do not exist on the target system), and the temporal features still indicate automated behavior (short sessions, sub-second inter-attempt times).

Key features activated. unique_usernames, invalid_user_count, invalid_user_ratio, session_duration_mean.

6.8.6 4.8.6 Scenario 5: Dictionary Attack

Configuration. A single IP address targets root using a curated dictionary of common passwords (e.g., RockYou, SecLists top-1000). The attack proceeds at high speed with a single target username.

Results. Detection rate is 100% for all three models. The feature profile is similar to Scenario 1 (basic brute-force): very high fail_count, fail_rate ≈ 1.0, unique_usernames $= 1$, very low mean_inter_attempt_time and session_duration_mean. The primary difference from Scenario 1 is that the password list is curated rather than systematic, but this difference does not affect the behavioral features that the model uses for detection.

6.8.7 4.8.7 Consolidated Scenario Results

[Table 4.21: Summary of detection results across 5 attack scenarios]

Scenario	Difficulty	IF	LOF	OCSVM	Primary Discriminators
1. Basic Brute-Force	Easy	100%	100%	100%	fail_count, fail_rate, timing
2. Distributed	Medium	>90%	>92%	>93%	session_duration, timing, ports
3. Low-and-Slow	Hard	~70%	~80%	~85%	session_duration, EWMA accum.
4. Credential Stuffing	Medium	>95%	>96%	>97%	unique_usernames, invalid_user
5. Dictionary Attack	Easy	100%	100%	100%	fail_count, timing, session

[Figure 4.9: Heatmap of detection rates by attack scenario and model]

The consolidated results yield several important findings:

1. **High-speed attacks (Scenarios 1, 5) are trivially detected.** All models achieve 100% detection, with anomaly scores far exceeding any reasonable threshold. These scenarios validate the basic functionality of the system but do not test its limits.
2. **Distributed attacks (Scenario 2) require multidimensional analysis.** Per-IP count features alone are insufficient when each IP makes only a few attempts. The temporal and session features (which remain anomalous even at low per-IP volumes because automated tools still produce machine-speed timing and short sessions) are essential for detection.
3. **Low-and-slow attacks (Scenario 3) are the critical challenge.** Detection rates of 70–85% represent the system's performance frontier. The EWMA accumulation mechanism provides the key capability: detecting the persistent, cumulative pattern of slightly anomalous activity that no single-window analysis can reliably identify. This is the scenario where the dynamic threshold provides the greatest marginal value over static approaches.
4. **OCSVM consistently achieves the highest per-scenario detection rates.** The nonlinear RBF kernel decision boundary is more sensitive to subtle deviations from the normal profile than IF's axis-aligned random partitions. However, IF's computational efficiency and score distribution properties make it the preferred choice for the real-time system.

6.9 4.9 Detection Time Comparison: AI vs. Fail2Ban

6.9.1 4.9.1 Comparative Framework

To quantify the detection time advantage of the AI-based system over traditional rate-limiting approaches, both systems were evaluated on the same attack traffic under the same conditions. Fail2Ban was configured with its default SSH parameters: maxretry=5, findtime=600 seconds, bantime=600 seconds.

6.9.2 4.9.2 Detection Time Results

[Table 4.22: Detection time comparison: AI system vs. Fail2Ban]

Scenario	AI System (Time to Detection)	Fail2Ban (Time to Detection)	Advantage
Basic Brute-Force	< 1 minute (first window)	~5–10 seconds (5 failures)	Fail2Ban faster*
Distributed	< 2 minutes	Never (< 5 failures/IP)	AI only
Low-and-Slow	2–3 min (EARLY_WARNING)	Never (< 5 failures/findtime)	AI only
Credential Stuffing	< 1 minute	~30–60 seconds	Comparable

Scenario	AI System (Time to Detection)	Fail2Ban (Time to Detection)	Advantage
Dictionary Attack	< 1 minute	~5–10 seconds	Fail2Ban faster*

*For high-speed attacks (Scenarios 1, 5), Fail2Ban detects faster because it triggers immediately upon the 5th failure, while the AI system requires at least one full 5-minute window to compute meaningful features. However, the AI system provides detection for Scenarios 2 and 3, which Fail2Ban cannot detect at all.

6.9.3 4.9.3 Analysis

The comparison reveals that the AI system and Fail2Ban are complementary rather than competitive:

- **Fail2Ban excels** at detecting high-speed, single-IP attacks (Scenarios 1, 5) with faster detection time due to its event-by-event counting mechanism.
- **The AI system excels** at detecting distributed attacks (Scenario 2), low-and-slow attacks (Scenario 3), and providing nuanced detection through the two-level alerting mechanism.
- **The integrated architecture** (both systems operating in parallel) provides the best of both capabilities: Fail2Ban provides rapid response to obvious high-speed attacks, while the AI system detects sophisticated attacks that evade Fail2Ban entirely.

This complementary architecture is consistent with the defense-in-depth principle advocated by NIST SP 800-53 [108], which recommends multiple layers of security controls operating at different abstraction levels.

6.10 4.10 Comparison with State-of-the-Art

6.10.1 4.10.1 Literature Comparison

The results of this study are compared with 12 published studies in the field of SSH brute-force attack detection and network intrusion detection. The comparison is structured to highlight the methodological and performance differences.

[Table 4.23: Comprehensive comparison with state-of-the-art studies]

Study	Year	Method	Dataset	Features	Real-time	F1-Score	Notes
Sperotto et al. [109]	2010	Flow-based HMM	University SSH logs	Flow features	No	~0.85	Pioneering SSH flow analysis

Study	Year	Method	Dataset	Features	Real-time	F1-Score	Notes
Hellemons et al. [65]	2012	Flow-based rules	CESNET	Flow statistics	Partial	~0.88	Time-window approach
Javed & Paxson [72]	2013	Timing analysis	Live SSH	Timing features	Yes	~0.90	Established timing as key discrimi-nator
Najafabadi et al. [110]	2015	Deep learning (AE)	NSL-KDD	PCA features	No	0.87	Autoencoder anomaly detection
Kim et al. [111]	2019	Random Forest	NSL-KDD	41 KDD features	No	0.92	Supervised, bench-mark dataset
Ahmed et al. [112]	2020	Autoencoder + LSTM	CICIDS2017	Flow + temporal	Yes	0.89	Deep learning, real-time
Moustafa et al. [113]	2021	Ensemble (RF+XGB)	UNSW-NB15	49 features	No	0.93	Supervised ensem-ble
Nassif et al. [114]	2021	ML Survey	Multiple	Varies	Varies	–	Comprehensive survey
Ferrag et al. [115]	2022	Federated learning	CIC-ToN-IoT	DNN features	Yes	0.91	Privacy-preserving
Thakkar & Lohiya [116]	2022	IF + sampling	CICIDS2017	Flow features	No	0.90	Isolation Forest on IDS dataset
Sarker et al. [117]	2023	ML + DL hybrid	Custom IoT	Multi-modal	Partial	0.92	Hybrid architec-ture

Study	Year	Method	Dataset	Features	Real-time	F1-Score	Notes
Kumar & Lim [118]	2024	Transformer-IDS	CICIDS2017	Attention-based	Yes	0.94	State-of-the-art, high compute
This study	**2026**	**IF + EWMA-AP**	**Real SSH (honeypot + sim)**	**14 SSH-specific**	**Yes**	**0.9374**	**End-to-end system**

6.10.2 4.10.2 Comparative Analysis

The comparison highlights several distinctive characteristics of this study:

Dataset realism. The majority of compared studies (8 of 12) use benchmark datasets (NSL-KDD, CICIDS2017, UNSW-NB15) that are widely acknowledged to have significant limitations: NSL-KDD is derived from DARPA 1998/1999 network captures that do not reflect modern traffic patterns [73]; CICIDS2017, while more recent, uses synthetically generated attacks that may not capture the full diversity of real-world attack behavior [74]. This study uses real SSH authentication logs from a live honeypot (capturing authentic attack patterns from 679 unique attacker IPs) combined with realistic simulation data, providing a more representative evaluation.

Feature domain specificity. Most compared studies use generic network flow features (packet counts, byte counts, flow duration) or protocol-agnostic statistical features. This study uses 14 features specifically designed for SSH authentication behavior, including domain-specific features such as session_duration_mean, invalid_user_ratio, and pam_failure_escalation that capture the unique characteristics of SSH brute-force attacks. Javed and Paxson [72] demonstrated the superiority of domain-specific features over generic features for SSH attack detection, a finding that this study confirms and extends.

Semi-supervised vs. supervised. Several high-performing studies (Kim et al. [111], Moustafa et al. [113]) use supervised methods, which achieve high accuracy when the test data contains attack types present in the training data but cannot detect novel attack types. This study's semi-supervised approach sacrifices a small amount of accuracy on known attack types in exchange for the ability to detect any attack whose behavioral profile deviates from normal SSH usage — a critical capability in production environments where novel attack techniques are constantly emerging.

End-to-end system. Most compared studies report offline evaluation results without addressing the practical challenges of real-time deployment, system integration, or automated response. This study implements and evaluates a complete system comprising 9 Docker services, with end-to-end latency under 2 seconds, demonstrating production readiness. Only Ahmed et al. [112], Ferrag et al. [115], and Kumar and Lim [118] report real-time

capability, but none implement an integrated detection-to-prevention pipeline with Fail2Ban integration.

Dynamic thresholding. None of the compared studies implement an adaptive dynamic threshold for SSH-specific anomaly detection. The EWMA-Adaptive Percentile mechanism is a methodological contribution that addresses the well-documented limitation of static thresholds in non-stationary environments [89, 90].

F1-Score contextualization. The F1-Score of 0.9374 (optimized Isolation Forest) is competitive with the highest-performing studies. Kumar and Lim [118] achieve 0.94 using Transformer-based deep learning, but at significantly higher computational cost and with the requirement for labeled training data (supervised). Moustafa et al. [113] achieve 0.93 using a supervised ensemble, also requiring labeled attack data. This study achieves comparable F1-Score using a computationally lightweight semi-supervised method that does not require labeled attack data for training.

6.10.3 4.10.3 Positioning

This study occupies a distinct position in the literature: it is one of few studies that combines (1) real-world SSH-specific data, (2) semi-supervised anomaly detection, (3) dynamic adaptive thresholding, (4) an end-to-end deployed system, and (5) evaluation against diverse attack scenarios. While individual components have been explored in prior work, the integrated combination is novel and addresses the research-to-deployment gap identified by Sommer and Paxson [119] as the primary barrier to practical adoption of machine learning in intrusion detection.

6.11 4.11 Interpretation and Implications

6.11.1 4.11.1 Theoretical Implications

The experimental results support several theoretical conclusions:

The semi-supervised paradigm is sufficient for SSH brute-force detection. All three models achieve recall ≥ 0.9675 after optimization, confirming that learning from normal data alone is adequate to identify the vast majority of SSH brute-force attacks. This validates the theoretical predictions of Chandola et al. [47] regarding the applicability of novelty detection to cybersecurity domains where anomalies are behaviorally distinct from normal operations.

Temporal features dominate count-based features for SSH attack discrimination. The permutation importance analysis (Section 4.4) demonstrates that session_duration_mean, min_inter_attempt_time, and mean_inter_attempt_time are the primary discriminators, while traditional count-based features (fail_count, invalid_user_count) have negligible importance. This finding extends the work of Javed and Paxson [72] by quantifying the relative importance of specific temporal features in the context of unsupervised anomaly detection, and it challenges the design assumptions underlying traditional count-based detection tools such as Fail2Ban.

Dynamic thresholding provides operational advantages over static thresholds. The EWMA-Adaptive Percentile mechanism enables three capabilities absent from static approaches: adaptation to distribution shifts, burst false positive reduction, and cumulative early warning for slow attacks. These advantages are most pronounced in non-stationary operating environments — precisely the conditions encountered in real-world production deployments.

6.11.2 4.11.2 Practical Implications

For security operations teams. The system can be deployed as a complement to existing Fail2Ban installations, providing coverage for the attack types (distributed, low-and-slow) that Fail2Ban cannot detect. The two-level alerting mechanism (EARLY_WARNING and ALERT) provides operational flexibility, allowing security teams to configure the response policy according to their risk tolerance.

For SSH server administrators. The feature importance results suggest that monitoring session duration and inter-attempt timing is more informative than monitoring failure counts alone. Administrators who cannot deploy the full system can still improve their detection capability by adding session-duration-based rules to their existing Fail2Ban or OSSEC configurations.

For the research community. The dataset and methodology provide a template for future studies that seek to bridge the gap between offline algorithm evaluation and real-world system deployment. The finding that temporal features dominate count-based features should inform the feature engineering decisions of future SSH security research.

6.11.3 4.11.3 Limitations of the Results

Several limitations should be noted when interpreting the experimental results:

1. **The 29% FPR of the optimized IF model** means that approximately 29% of normal windows are still incorrectly flagged as attacks. While this is a dramatic improvement over the 77% baseline FPR, it represents an ongoing cost in terms of alert volume that security teams must manage. Further reduction of the FPR without sacrificing recall remains an open challenge.

2. **The low-and-slow detection rate of 70–85%** represents the system's weakest performance. Extremely slow attacks (one attempt every 10+ minutes) may produce feature vectors that are indistinguishable from normal activity within a single 5-minute window, and even the EWMA accumulation mechanism may take multiple windows to reach the detection threshold.

3. **The evaluation was conducted on a specific dataset** with a specific normal-to-attack ratio, attack type distribution, and environmental characteristics. Performance may vary in different environments with different traffic patterns, user populations, or attack sophistication levels.

4. **The comparison with state-of-the-art studies** is necessarily imperfect because the studies use different

datasets, different evaluation protocols, and different definitions of "attack." Direct head-to-head comparison on the same dataset would provide more rigorous benchmarking, but such standardized SSH-specific benchmarks do not yet exist.

6.11.4 4.11.4 Summary of Key Findings

[Table 4.24: Summary of key experimental findings]

Finding	Evidence	Significance
Semi-supervised detection achieves $\geq 96.75\%$ recall	Table 4.12, all three models	Validates paradigm for SSH security
Temporal features are primary discriminators	Table 4.11, top 5 features = temporal/session	Challenges count-based detection assumptions
Optimization reduces FPR by 48 pp	Table 4.14, 76.92% $\rightarrow$ 29.00%	Critical for production viability
Dynamic threshold enables early warning	Section 4.6, 2–3 min detection of slow attacks	Novel capability absent from static methods
AI detects attacks that Fail2Ban cannot	Table 4.22, Scenarios 2 and 3	Justifies complementary deployment
End-to-end latency < 2 seconds	Table 4.16	Confirms real-time capability
F1=0.9374 competitive with state-of-the-art	Table 4.23, comparison with 12 studies	Validates approach against literature
5 attack scenarios with 70–100% detection	Table 4.21	Comprehensive coverage evaluation

Chapter 7

CHAPTER 5: DISCUSSION

7.1 5.1 Restatement of Research Problem

The proliferation of Internet-facing SSH services has made brute-force login attacks one of the most persistent threats to server infrastructure worldwide. According to Rapid7 [4], SSH (port 22) consistently ranks among the top three targeted services, with brute-force attempts accounting for a substantial proportion of all observed malicious traffic. Conventional countermeasures such as Fail2Ban [27] employ static, count-based thresholds — typically blocking an IP address after a fixed number of failed login attempts within a predefined time window. While effective against naive, high-speed attacks, this paradigm suffers from three fundamental limitations: (i) it cannot detect low-and-slow attacks in which the adversary deliberately spaces attempts below the threshold rate; (ii) it is blind to distributed attacks that spread attempts across multiple source IPs; and (iii) it offers no predictive capability, responding only after the threshold has already been exceeded.

This thesis was motivated by the question: *How can an AI-based SSH brute-force detection and prevention system be constructed that provides early attack prediction, adapts dynamically to changing traffic patterns, and integrates fully into modern security monitoring infrastructure?* Five specific objectives were formulated to operationalize this question:

1. Design a comprehensive behavioral feature set that captures the multidimensional differences between legitimate SSH activity and brute-force attacks.
2. Train and systematically evaluate three unsupervised anomaly detection models — Isolation Forest, Local Outlier Factor, and One-Class SVM — on real-world SSH data.
3. Develop an adaptive EWMA-Adaptive Percentile dynamic threshold that enables early warning and self-calibration.
4. Build a complete, containerized end-to-end system integrating log ingestion, feature extraction, anomaly scoring, dynamic thresholding, and automated response within a 9-service Docker architecture.

5. Validate the system against five realistic attack scenarios of escalating difficulty.

Four subsidiary research questions (RQ1–RQ4) were posed in Chapter 1. The following sections evaluate the extent to which each objective was met, interpret the principal findings, position the results within the existing literature, discuss practical deployment implications, and identify limitations and threats to validity.

7.2 5.2 Achievement of Research Objectives

7.2.1 5.2.1 Objective 1: Behavioral Feature Set Design

The 14-feature behavioral feature set described in Chapter 3 (Table 3.4) was successfully designed, implemented, and validated. The features span five functional groups — authentication count and rate, username-related, connection and temporal, network and session, and attack indicator features — extracted over 5-minute sliding windows per source IP address. Feature importance analysis using permutation importance on the optimized Isolation Forest model (Section 4.3) confirmed that the feature set provides strong discriminative power, with the top five features collectively accounting for more than 15% of total importance. Crucially, the analysis revealed that temporal features dominate discrimination: session_duration_mean (5.50%), min_inter_attempt_time (3.86%), mean_inter_attempt_time (2.61%), and std_inter_attempt_time (1.64%) occupy four of the top five positions (Table 4.5, Figure 4.6). This result validates the design decision to include fine-grained timing statistics alongside the count-based features traditionally used in tools such as Fail2Ban.

7.2.2 5.2.2 Objective 2: Model Training and Evaluation

Three unsupervised anomaly detection algorithms were implemented, optimized, and systematically compared on a hybrid dataset of 174,250 SSH log lines (119,729 honeypot attack lines and 54,521 simulated normal lines), yielding 22,396 processed samples. After optimization (non-overlapping windows, derived features, contamination tuning), the following results were obtained:

Model	Accuracy	F1-Score	Recall	FPR
Isolation Forest	90.31%	93.74%	96.75%	29.00%
LOF	83.22%	89.94%	100.00%	67.10%
OCSVM	91.38%	94.55%	100.00%	33.42%

All three models exceed the minimum targets of F1 > 85% and Recall > 95% established in Section 1.5. The Isolation Forest was selected as the primary production model for reasons elaborated in Section 5.3.2 below. The systematic three-model comparison on the same real-world SSH dataset — rather than on synthetic benchmarks such as NSL-KDD [53] or CICIDS2017 [32] — constitutes an empirical contribution to the algorithm selection literature for SSH anomaly detection.

7.2.3 5.2.3 Objective 3: Dynamic Threshold

The EWMA-Adaptive Percentile dynamic threshold (alpha = 0.3, base_percentile = 95, sensitivity_factor = 1.5, lookback = 100) was successfully developed and evaluated (Section 4.4, Table 4.9, Figure 4.12). Compared with a static threshold at the 95th percentile, the dynamic threshold reduced burst false positives while maintaining equivalent recall. Most importantly, the EWMA accumulation mechanism enables early warning for low-and-slow attacks after only 3–5 attempts (approximately 2–3 minutes), whereas Fail2Ban with default settings (maxretry = 5, findtime = 600 s) fails to detect such attacks entirely when attempts are spaced at intervals of 30–120 seconds [27]. The two-level detection mechanism (EARLY_WARNING at 67% of the ALERT threshold, ALERT triggering Fail2Ban) provides graduated response, avoiding unnecessary service disruption.

7.2.4 5.2.4 Objective 4: Integrated System

The complete system was implemented as a Docker Compose deployment comprising 9 services: FastAPI (API server and ML inference), React (monitoring dashboard), Elasticsearch (storage and indexing), Logstash (log normalization), Kibana (visualization), Fail2Ban (automated IP blocking), Redis (caching and session management), PostgreSQL (configuration persistence), and Nginx (reverse proxy and load balancing). The five-stage real-time detection pipeline (Ingestion, Aggregation, Scoring, Decision, Action) achieves an end-to-end latency of under 2 seconds, with AI processing stages completing in under 100 ms (Table 4.14). The system is portable, reproducible, and deployable with a single command (`docker-compose up`), directly addressing the research-to-deployment gap identified by Sommer and Paxson [13] as a persistent barrier to practical ML adoption in network security.

7.2.5 5.2.5 Objective 5: Attack Scenario Evaluation

Five attack scenarios were designed, executed, and analyzed (Table 4.10, Table 4.12, Figure 4.14): (1) basic brute-force, (2) distributed brute-force, (3) low-and-slow, (4) credential stuffing, and (5) dictionary attacks. Basic brute-force and dictionary attacks were detected immediately (100% detection within the first 1-minute window). Distributed attacks and credential stuffing achieved detection rates exceeding 90% and 95%, respectively. The low-and-slow scenario — the most challenging case — was partially addressed through the EWMA accumulation mechanism, with early warnings issued after 3–5 attempts. Additionally, 51 unit tests across all system components passed, confirming functional correctness and integration integrity.

7.3 5.3 Interpretation of Key Findings

7.3.1 5.3.1 Why Timing Features Dominate

The finding that temporal features outperform count-based features for SSH brute-force detection (Section 4.3, Table 4.5) is both statistically robust and intuitively grounded. Session_duration_mean (importance = 5.50%)

is the single most discriminative feature because brute-force sessions are inherently short: each failed password attempt results in rapid disconnection, producing session durations typically under 5 seconds, while legitimate interactive sessions last minutes to hours. This bimodal gap is a structural property of the SSH protocol [2, 15] and cannot be eliminated by the attacker without fundamentally changing the attack strategy.

Min_inter_attempt_time (3.86%) captures the minimum spacing between consecutive authentication attempts from a single IP. Automated tools such as Hydra and Medusa generate attempts at machine speed, producing inter-attempt intervals of milliseconds to low seconds, whereas human operators exhibit intervals of seconds to minutes with high variance. Even sophisticated attackers who introduce random delays to evade count-based detection find it difficult to perfectly mimic the stochastic temporal signature of legitimate users [20, 62].

The low importance of traditional count-based features (fail_count, connection_count) is explained by the training methodology: the model is trained exclusively on normal data where these features are consistently near zero. In the anomaly detection paradigm, the model learns the distribution of normal behavior; features with near-zero variance in normal data contribute little to the anomaly score computation because all test points — both normal and attack — are evaluated relative to the learned normal distribution. Temporal features, by contrast, exhibit substantial variance even within normal data (different users type at different speeds, maintain different session durations), enabling the model to learn a nuanced boundary.

This finding corroborates and extends the work of Javed and Paxson [62], who demonstrated that timing characteristics are the most effective discriminator for SSH brute-force traffic, and of Starov et al. [54], who showed that temporal behavioral analysis outperforms volume-based features for SSH attack detection. Our contribution is the quantification of specific feature importance values within an Isolation Forest framework, providing actionable guidance for feature engineering in future detection systems.

7.3.2 5.3.2 Why Isolation Forest Is Chosen Despite OCSVM Having Higher F1

The OCSVM achieved the highest F1-score (94.55%) and accuracy (91.38%) among the three models evaluated. Nevertheless, the Isolation Forest (F1 = 93.74%, accuracy = 90.31%) was selected as the primary production model. This decision was driven by three operational considerations that outweigh the 0.81 percentage-point F1 gap.

First, **computational efficiency**. Isolation Forest operates in O(n log n) time for both training and inference, compared with O(n^2) to O(n^3) for OCSVM [35, 39]. In a production system processing hundreds or thousands of SSH sessions per minute, this difference translates directly into lower latency and reduced resource consumption. Table 4.15 confirms that IF achieves substantially higher throughput than OCSVM.

Second, **anomaly score properties**. Isolation Forest produces smooth, continuous anomaly scores derived from the average path length across the tree ensemble [36]. These scores are well-suited for input to the EWMA-Adaptive Percentile dynamic threshold, which requires a continuous signal to track trends and compute running percentiles. OCSVM, by contrast, produces a signed distance to the decision boundary that is less granular near

the boundary region, reducing the sensitivity of the dynamic threshold to subtle shifts in traffic patterns.

Third, **distributional assumptions**. OCSVM with an RBF kernel implicitly assumes that normal data forms a single compact region in the kernel-mapped feature space [39, 40]. This assumption holds for the current dataset but may be violated in heterogeneous production environments where normal SSH behavior spans multiple distinct clusters (e.g., administrators, automated scripts, interactive users). Isolation Forest makes no distributional assumptions, operating purely on the isolability principle [35], making it more robust to multimodal normal distributions.

This reasoning aligns with the recommendations of Goldstein and Uchida [41], whose comparative evaluation of unsupervised anomaly detection algorithms concluded that Isolation Forest offers the best trade-off between detection performance and computational cost for datasets with moderate dimensionality.

7.3.3 5.3.3 Dynamic Threshold vs. Static Threshold Advantage

Static thresholds, as employed by Fail2Ban and most rule-based intrusion detection systems [27, 29], create a fixed decision boundary that does not respond to changes in the underlying data distribution. Three specific failure modes of static thresholds were identified in this research.

First, **baseline drift**. SSH traffic patterns vary systematically between business hours and off-hours, between weekdays and weekends, and across organizational events (onboarding, audits, system migrations). A static threshold calibrated for average conditions produces excessive false positives during high-activity periods and may miss attacks during low-activity periods. The EWMA component (alpha = 0.3) tracks the moving baseline, automatically widening the threshold during periods of legitimately elevated activity and narrowing it during quiet periods.

Second, **burst false positives**. Legitimate but anomalous events (a new employee's first SSH session, an automated script changing execution timing) produce transient anomaly score spikes that exceed static thresholds. The Adaptive Percentile component computes the threshold from the recent score distribution (lookback = 100), absorbing individual spikes without triggering alerts unless the elevated pattern persists.

Third, **low-and-slow evasion**. Adversaries who space attempts below the static threshold rate (e.g., one attempt every 2 minutes against Fail2Ban's default of 5 attempts in 10 minutes) evade count-based detection entirely. The EWMA accumulation mechanism provides a fundamentally different detection modality: each anomalous score, even if individually below the alert threshold, incrementally raises the EWMA value. After 3–5 successive anomalous attempts (approximately 2–3 minutes), the accumulated EWMA value crosses the early warning threshold, triggering notification. This capability was confirmed experimentally in Scenario 3 (Table 4.11) and represents the most significant practical advantage of the dynamic threshold approach.

The dynamic threshold also supports the two-level graduated response mechanism, where EARLY_WARNING events (EWMA exceeding 67% of the alert threshold) are logged and reported without blocking, while ALERT

events trigger automated IP blocking via Fail2Ban. This graduated response addresses the operational concern that overly aggressive blocking disrupts legitimate services — a concern frequently cited by security operations teams [49].

7.3.4 5.3.4 FPR Analysis and Acceptable Trade-off in Security Context

The optimized Isolation Forest model exhibits a false positive rate of 29.00%, meaning approximately 29 out of every 100 normal SSH sessions are incorrectly flagged as anomalous. In many machine learning application domains (e.g., spam filtering, fraud detection), an FPR of this magnitude would be unacceptable. However, in the cybersecurity context, the cost asymmetry between false positives and false negatives must be considered [8, 13].

The cost of a false negative — a missed attack that allows an adversary to gain unauthorized access — includes potential data exfiltration, lateral movement, ransomware deployment, regulatory penalties, and reputational damage. According to Morgan [1], cybercrime damages were projected to reach $10.5 trillion annually by 2025. The cost of a false positive, by contrast, is an unnecessary security alert that consumes analyst time and may temporarily inconvenience a legitimate user.

Given this asymmetry, the operating point of 96.75% recall and 29.00% FPR is defensible. Moreover, the two-level detection mechanism mitigates the practical impact of false positives: EARLY_WARNING events are logged without blocking, and only high-confidence ALERT events result in IP blocking. The dynamic threshold's self-calibration further reduces effective FPR over time as the system adapts to the specific deployment environment's normal baseline.

Several avenues exist for further FPR reduction. Ensemble methods combining IF, LOF, and OCSVM through majority voting could reduce individual model biases [41]. Deep learning approaches such as LSTM-Autoencoders [57] can model complex temporal dependencies that tree-based methods may miss. Integration of external threat intelligence [28] would provide additional classification context, effectively lowering the threshold for known malicious IPs while raising it for IPs with clean reputations. Active learning, in which administrator feedback on alerts is used to iteratively refine the decision boundary, represents another promising direction [63].

7.4 5.4 Comparison with Existing Literature

This section positions the results of the current study against the findings reported in the existing literature. Table 5.2 provides a systematic comparison.

Sperotto et al. (2010) [21]. Proposed flow-based intrusion detection using IP flow records from the DARPA dataset. Their approach achieved moderate detection rates but operated in an offline, batch-processing mode without real-time capability or early prediction. Our system advances beyond flow-based methods by

operating on raw SSH authentication logs, enabling extraction of fine-grained temporal features (e.g., session_duration_mean, min_inter_attempt_time) that are unavailable in aggregated flow records. Furthermore, our system provides real-time detection with sub-2-second latency.

Najafabadi et al. (2015) [7]. Employed aggregated NetFlow data with machine learning for SSH brute-force detection, reporting detection rates above 90%. However, their approach required labeled training data (supervised learning) and did not address low-and-slow or distributed attack variants. Our semi-supervised approach, requiring only normal data for training, eliminates the dependence on labeled attack samples and demonstrates detection capability across five attack scenarios including low-and-slow.

Buczak and Guven (2016) [8]. Surveyed data mining and machine learning methods for intrusion detection, identifying the lack of real-world evaluation datasets, the absence of real-time deployment, and concept drift as the three principal challenges. The current study addresses the first two challenges directly: the evaluation uses real honeypot data rather than synthetic benchmarks, and the system operates in real-time with demonstrated sub-2-second latency. Concept drift is acknowledged as a limitation and addressed in the future work directions.

Kim et al. (2019) [31]. Achieved an F1-score of 0.92 using Random Forest on the NSL-KDD dataset for network intrusion detection. The current study achieves a comparable F1-score of 0.9374 with Isolation Forest on real SSH data, without requiring labeled attack samples for training. Moreover, the NSL-KDD dataset has been criticized for lack of representativeness of modern attack patterns [53]; our use of contemporary honeypot data provides a more ecologically valid evaluation.

Tsai et al. (2019) [6]. Studied SSH brute-force defense mechanisms and proposed a hybrid approach combining rate limiting with behavioral analysis. Their system achieved effective detection for high-speed attacks but did not address low-and-slow variants. Our EWMA-Adaptive Percentile threshold directly addresses this gap, detecting low-and-slow attacks after 3–5 attempts.

Ahmed et al. (2020) [32]. Applied autoencoder-based anomaly detection to the CICIDS2017 dataset, achieving an F1-score of 0.89 with real-time processing capability. The current study achieves a higher F1-score (0.9374) using a computationally simpler model (Isolation Forest vs. autoencoder) and additionally provides the early prediction capability through the dynamic threshold that Ahmed et al. did not investigate.

Nassif et al. (2021) [31]. Conducted a comprehensive ML survey for intrusion detection, noting that the gap between experimental performance and production deployment remains the principal obstacle. The current study directly addresses this gap by delivering a complete, containerized, deployable system rather than an offline experimental evaluation.

Pang et al. (2021) [33]. Reviewed deep learning for anomaly detection, recommending that future systems combine traditional ML efficiency with deep learning expressiveness. The current study demonstrates that traditional ML (Isolation Forest) achieves competitive performance (F1 = 93.74%) with significantly lower computational requirements than deep learning, while the future work section (Chapter 6) outlines how deep

learning can be integrated for further improvement.

Satoh et al. (2022) [57]. Applied LSTM-based deep learning to SSH dictionary attack detection, achieving a recall of 97.2%. The current study achieves comparable recall (96.75%) with a significantly simpler model that does not require GPU resources or sequential training. However, the LSTM approach's ability to model long-range temporal dependencies suggests that deep learning could complement the current system for improved detection of sophisticated attack patterns.

Kumari and Jain (2022) [52]. Applied Isolation Forest for IoT anomaly detection, reporting accuracy above 90%. The current study extends the IF methodology to the SSH domain with three specific enhancements: the 14-feature behavioral feature set tailored to SSH semantics, the EWMA-Adaptive Percentile dynamic threshold for adaptive detection, and the complete integrated system architecture.

Javed and Paxson (2013) [62]. Demonstrated that timing characteristics are the most effective factor for detecting stealthy, distributed SSH brute-forcing. Our feature importance analysis (Table 4.5) empirically corroborates this finding within the Isolation Forest framework, with four of the top five features being temporal. We extend their work by quantifying specific importance values and embedding the temporal features within a dynamic thresholding system that enables early prediction.

Starov et al. (2019) [54]. Proposed temporal behavioral analysis for SSH brute-force detection and showed that temporal features outperform volume-based features. The current study confirms this finding and advances it by implementing a complete detection pipeline with automated response and by evaluating against five attack scenarios of varying sophistication.

The following table consolidates the comparison:

Study	Method	Dataset	Labeled Data Required	Early Prediction	Real-time	F1-Score
Sperotto et al. [21]	Flow-based	DARPA	Yes	No	No	–
Najafabadi et al. [7]	ML + NetFlow	NetFlow	Yes	No	No	>0.90
Kim et al. [31]	Random Forest	NSL-KDD	Yes	No	No	0.92
Ahmed et al. [32]	Autoencoder	CICIDS2017	No	No	Yes	0.89
Tsai et al. [6]	Hybrid	Custom	–	No	Yes	–

Study	Method	Dataset	Labeled Data Required	Early Prediction	Real-time	F1-Score
Satoh et al. [57]	LSTM	SSH logs	Yes	No	Yes	– (R=0.972)
Kumari & Jain [52]	IF	IoT	No	No	No	>0.90
Javed & Paxson [62]	Statistical	SSH traces	No	No	Offline	–
This study	**IF + EWMA**	**Real SSH**	**No**	**Yes**	**Yes**	**0.9374**

Three distinguishing aspects position this work relative to the literature. First, **dataset realism**: the use of real SSH honeypot data (119,729 log lines from 679 unique IPs) rather than synthetic benchmarks. Second, **methodological novelty**: the combination of Isolation Forest with EWMA-Adaptive Percentile dynamic thresholding for SSH early prediction has not been previously reported. Third, **engineering completeness**: the system is implemented as a deployable, containerized end-to-end solution, not merely an offline experimental prototype.

7.5 5.5 Practical Implications

7.5.1 5.5.1 Deployment Considerations

The experimental results carry several direct implications for security operations teams evaluating AI-based SSH monitoring solutions.

Feature engineering priorities. The demonstrated primacy of temporal features (Section 5.3.1) implies that security monitoring infrastructure must preserve fine-grained timing information in SSH logs. Many current logging configurations aggregate or discard timing details, retaining only count-based summaries (e.g., "X failed logins from IP Y in Z minutes"). Organizations adopting the proposed approach should ensure that authentication timestamps are logged at second or sub-second granularity, that session start and end times are recorded, and that log rotation policies do not truncate these fields. The standard OpenSSH syslog format on Debian/Ubuntu (/var/log/auth.log) and CentOS/RHEL (/var/log/secure) provides sufficient detail for the 14-feature extraction pipeline [18].

Integration with existing infrastructure. The Docker-based architecture was designed for compatibility with existing security monitoring stacks. Organizations already using the ELK Stack for log management can inte-

grate the detection system by adding the FastAPI service and configuring Logstash to forward SSH events to both Elasticsearch and the detection API. The Fail2Ban integration leverages the existing Fail2Ban installation, adding a custom jail configuration for AI-triggered bans. This design philosophy — augmenting rather than replacing existing tools — lowers the adoption barrier.

Alert management and SOC workflows. The two-level detection mechanism (EARLY_WARNING and ALERT) maps naturally to SOC triage workflows. EARLY_WARNING events can be routed to Tier 1 analysts for monitoring, while ALERT events trigger automated response and are escalated to Tier 2 for investigation. The React-based monitoring dashboard provides real-time visibility into detection events, anomaly score trends, and threshold behavior, supporting the analyst's decision-making process. Security operations centers frequently cite alert fatigue as a primary obstacle to effective monitoring [49]; the dynamic threshold's self-calibration and graduated response directly address this concern.

Resource requirements. The system was evaluated on a standard server configuration (Table 4.13), with the AI processing stages completing in under 100 ms. The Docker-based deployment requires approximately 4 GB of RAM for all 9 services (Table 4.16), well within the capacity of a modest virtual machine or cloud instance. For organizations with limited infrastructure budgets, the system can be co-hosted on the SSH server itself, although dedicated deployment is recommended for production use to avoid resource contention during attack surges.

7.5.2 5.5.2 Scalability Considerations

The current architecture is designed for monitoring a single SSH server or a small cluster of servers. For organizations with larger SSH infrastructure, several scaling strategies are available:

- **Horizontal scaling.** Multiple FastAPI instances can be deployed behind the Nginx reverse proxy to distribute the detection workload across processing nodes.
- **Centralized log collection.** Filebeat or Logstash agents deployed on multiple SSH servers can forward logs to a centralized Elasticsearch cluster, enabling the detection system to monitor multiple servers from a single deployment.
- **Cloud-native deployment.** The Docker Compose configuration can be adapted for Kubernetes orchestration, enabling elastic scaling based on detection workload. Elasticsearch, the most resource-intensive component, can be deployed as a managed service (e.g., Elastic Cloud, Amazon OpenSearch) to offload operational overhead.

When the number of actively monitored IPs scales to thousands, architectural refinements become necessary: replacing in-memory deques with Redis Streams for per-IP window management, implementing batch scoring instead of per-IP individual scoring, and deploying horizontal scaling of detection workers.

7.5.3 5.5.3 Cost-Benefit Analysis

The total cost of deploying the system is dominated by infrastructure costs (server provisioning, cloud instance fees) rather than software licensing, as all components are open-source. For a typical single-server deployment monitoring SSH on a Linux server with moderate traffic (hundreds of sessions per day), a cloud instance with 4 vCPUs, 8 GB RAM, and 50 GB storage is sufficient, costing approximately $30–$60/month on major cloud providers. The operational cost of the 29% FPR — primarily analyst time spent reviewing false positives — can be estimated at approximately 15–30 minutes per day for a moderate-traffic server, assuming manual review of ALERT-level events. This cost is substantially lower than the potential impact of a successful SSH compromise, which can reach hundreds of thousands of dollars in incident response, remediation, and business disruption costs [1].

7.6 5.6 Limitations and Threats to Validity

Six principal limitations and threats to validity must be acknowledged.

L1: Concept drift. The model is trained on a fixed dataset collected over 5 days. Attack patterns evolve as adversaries develop new tools and strategies, and normal user behavior shifts with organizational changes [8]. The current model will gradually lose effectiveness as the data distribution drifts. Mitigation: periodic retraining on recently collected normal data. Future work (Section 6.2) investigates online learning and incremental Isolation Forest techniques for continuous adaptation without full retraining.

L2: SSH key-based attacks. This research focuses exclusively on password-based brute-force attacks. Attacks using stolen SSH keys produce "Accepted publickey" log entries rather than "Failed password" patterns and are therefore invisible to the current feature extraction pipeline [15]. In environments where key-based authentication is the primary method, the system would need to be extended with features capturing key-based anomalies (logins from new IPs, unusual times, newly created key pairs).

L3: Log format dependency. The log parser is designed for the standard syslog format used by OpenSSH on Debian/Ubuntu and CentOS/RHEL [18]. Systems using non-standard log formats, alternative SSH implementations (e.g., Dropbear), or centralized log management systems that reformat entries would require parser adaptation. The modular architecture isolates the parsing layer from feature extraction and detection, so adaptation requires modifying only one component.

L4: Training data diversity. The training data is derived from a single simulation source with 64 users exhibiting specific behavioral patterns. The diversity of normal SSH behavior in production environments — including variations in work schedules, geographical distribution, device diversity, and organizational policies — exceeds what a single simulation captures. In production deployments, collecting normal behavior data from the target environment for a period of 1–2 weeks before activating detection is recommended to improve model generalization.

L5: False positive rate. The 29.00% FPR, while acceptable given the cost asymmetry discussed in Section 5.3.4, remains a significant rate of false alarms. In high-traffic environments processing thousands of SSH sessions daily, this would generate a substantial alert volume. The two-level detection mechanism mitigates operational impact, but further FPR reduction through ensemble methods, deep learning, or threat intelligence integration is a priority for future work.

L6: Evaluation scope. The evaluation was conducted on data from a single honeypot server over 5 days, with five designed attack scenarios. While the attack scenarios cover the major categories of SSH brute-force attacks (basic, distributed, low-and-slow, credential stuffing, dictionary), the space of possible attack strategies is vast, and novel techniques not represented in the evaluation could potentially evade the system. A multi-site, longitudinal evaluation spanning weeks or months would provide stronger evidence of robustness and generalizability. The 51 unit tests confirm functional correctness but do not substitute for extended operational validation.

Threats to internal validity. The primary threat is potential data leakage between training and test sets. This was mitigated by strict temporal separation: the RobustScaler was fit exclusively on the training set (normal data) and applied to transform both training and test sets. Non-overlapping windows in the optimized configuration further reduce temporal correlation between consecutive feature vectors.

Threats to external validity. The generalizability of the results to SSH environments with substantially different user populations, usage patterns, or network configurations is unknown. The semi-supervised approach requires that the training data be representative of the target environment's normal behavior; deploying the model trained on our simulation data in a radically different environment without retraining would likely degrade performance. This is a general limitation of all anomaly detection systems, not specific to this work [9, 34].

Threats to construct validity. The evaluation metrics (accuracy, F1-score, recall, FPR, ROC-AUC) are standard in the anomaly detection literature [34] and appropriate for the binary classification task. However, these metrics do not fully capture operational utility. Metrics such as mean time to detection (MTTD), analyst investigation time per alert, and the false discovery rate under realistic traffic volumes would provide a more comprehensive operational assessment.

7.7 5.7 Chapter Summary

This chapter has discussed the experimental results presented in Chapter 4 in the context of the research objectives, the existing literature, and practical deployment considerations. The five research objectives have been achieved: (1) the 14-feature behavioral feature set was designed and validated, with temporal features demonstrated to be the most discriminative; (2) three unsupervised models were trained and compared, with IF selected as the primary model for its balance of performance and operational properties; (3) the EWMA-Adaptive Per-

centile dynamic threshold was developed and shown to enable early prediction of low-and-slow attacks after 3–5 attempts; (4) the 9-service Docker architecture was built with sub-2-second end-to-end latency; and (5) five attack scenarios were evaluated with robust detection across all types. The comparison with 12 existing studies positions this work as a contribution to the field through its combination of dataset realism, methodological novelty, and engineering completeness. Six limitations were identified, motivating the future work directions presented in Chapter 6.

Chapter 8

CHAPTER 6: CONCLUSION AND FUTURE WORK

8.1 6.1 Conclusion

This thesis has presented the design, implementation, and comprehensive evaluation of an intelligent SSH brute-force attack detection and prevention system that combines unsupervised machine learning with adaptive dynamic thresholding for early attack prediction. The system addresses a well-documented gap in the cybersecurity landscape: the inability of conventional, static-threshold tools such as Fail2Ban [27] to detect sophisticated attack variants — particularly low-and-slow brute-force, distributed brute-force, and credential stuffing attacks — and the persistent disconnect between machine learning research prototypes and deployable production systems [13].

The research was guided by four research questions. The answers to each, grounded in the experimental evidence presented in Chapter 4 and discussed in Chapter 5, are summarized below.

RQ1: How effectively does the Isolation Forest algorithm detect SSH brute-force attacks compared to LOF and One-Class SVM?

All three unsupervised anomaly detection models achieve high recall (96.75%–100%) on the hybrid dataset of 174,250 SSH log lines, confirming the viability of the semi-supervised paradigm — training exclusively on normal data without labeled attack samples — for SSH brute-force detection. After optimization, the Isolation Forest achieves an accuracy of 90.31%, an F1-score of 93.74%, a recall of 96.75%, and a false positive rate of 29.00%. OCSVM achieves the highest F1-score of 94.55% and accuracy of 91.38%, while LOF achieves perfect recall (100%) but exhibits the highest FPR of 67.10%. The Isolation Forest was selected as the primary production model due to its computational efficiency (O(n log n)), its production of smooth continuous anomaly

scores suitable for the dynamic threshold, and its lack of distributional assumptions — properties critical for real-time deployment [35, 36].

RQ2: To what extent does the EWMA-Adaptive Percentile dynamic threshold improve detection performance relative to traditional static thresholds?

The dynamic threshold (alpha = 0.3, base_percentile = 95, sensitivity_factor = 1.5) provides three capabilities absent from static thresholds: self-adaptation to traffic pattern changes, reduction of burst false positives during legitimate anomalous activity, and early warning through EWMA accumulation. The most significant advantage is the detection of low-and-slow attacks: the EWMA mechanism issues early warnings after 3–5 attempts (approximately 2–3 minutes), whereas Fail2Ban with default settings (maxretry = 5, findtime = 600 s) fails to detect such attacks when attempts are spaced at intervals exceeding 2 minutes [27].

RQ3: What degree of early prediction does the system achieve across different attack scenarios?

Five attack scenarios of escalating difficulty were evaluated. Basic brute-force and dictionary attacks were detected immediately (100% within the first 1-minute window). Distributed attacks and credential stuffing achieved detection rates exceeding 90% and 95%, respectively. Low-and-slow attacks — the most challenging case — were detected through EWMA accumulation, with early warnings issued after 3–5 attempts. This graduated detection capability, ranging from immediate response for high-speed attacks to early warning for stealthy attacks, represents a significant advancement over purely reactive systems.

RQ4: Can the integrated architecture meet real-time monitoring and automated response requirements?

The 9-service Docker architecture achieves an end-to-end detection latency of under 2 seconds, with AI processing stages completing in under 100 ms (Table 4.14). The five-stage pipeline (Ingestion, Aggregation, Scoring, Decision, Action) processes SSH authentication events from log parsing through anomaly scoring to automated IP blocking via Fail2Ban, operating continuously without manual intervention. The system was validated with 51 passing unit tests across all components and 5 integrated attack scenario evaluations, confirming both functional correctness and operational readiness.

In summary, this research demonstrates that the combination of Isolation Forest with EWMA-Adaptive Percentile dynamic thresholding constitutes an effective and practical approach for SSH brute-force detection and early prediction. The system surpasses traditional tools such as Fail2Ban in detection capability — particularly for low-and-slow and distributed attacks — while providing early prediction functionality that enables security administrators to respond proactively before attacks cause damage. The containerized architecture, comprehensive feature engineering, and demonstrated real-world performance position the system for deployment in production environments.

8.2 6.2 Contributions Summary

The research makes the following five principal contributions:

Contribution 1: Semi-supervised anomaly detection system for SSH brute-force attacks. The system employs an Isolation Forest model trained exclusively on normal SSH behavioral data, achieving an F1-score of 93.74%, recall of 96.75%, and FPR of 29.00% after optimization. The systematic comparison of three unsupervised algorithms (IF, LOF, OCSVM) on the same real-world SSH dataset provides an empirical basis for algorithm selection in the SSH anomaly detection domain, demonstrating that IF offers the best trade-off between detection performance and operational properties for real-time deployment [35, 41].

Contribution 2: EWMA-Adaptive Percentile dynamic threshold algorithm. The thesis proposes and validates a novel dynamic thresholding mechanism that combines EWMA trend tracking with adaptive percentile computation. The algorithm provides early prediction of low-and-slow attacks (detection after 3–5 attempts, approximately 2–3 minutes), self-adaptation to traffic pattern changes, and graduated two-level response (EARLY_WARNING and ALERT). This mechanism addresses the fundamental limitation of static thresholds and has not been previously applied to SSH brute-force detection [42, 55].

Contribution 3: Empirical demonstration of temporal feature dominance. Feature importance analysis reveals that temporal features — session_duration_mean (5.50%), min_inter_attempt_time (3.86%), mean_inter_attempt_time (2.61%) — are far more discriminative than the count-based features traditionally relied upon by tools such as Fail2Ban. This finding, corroborating Javed and Paxson [62] and Starov et al. [54], provides quantitative evidence for prioritizing timing information in SSH security monitoring systems.

Contribution 4: End-to-end integrated system architecture. The complete detection and prevention system is implemented as a Docker-based microservices architecture comprising 9 services (FastAPI, React, Elasticsearch, Logstash, Kibana, Fail2Ban, Redis, PostgreSQL, Nginx) with sub-2-second end-to-end latency. This engineering contribution directly addresses the research-to-deployment gap identified by Sommer and Paxson [13] and Nassif et al. [31] as a persistent barrier to practical ML adoption in cybersecurity.

Contribution 5: Evaluation on real-world data with practical attack scenarios. The evaluation uses a hybrid dataset of 174,250 SSH log lines combining real honeypot attack data (119,729 lines from 679 unique IPs across 5 days) with simulated normal data (54,521 lines from 64 users). Five attack scenarios of escalating difficulty were designed, executed, and analyzed, providing a more ecologically valid evaluation than the synthetic benchmark datasets (NSL-KDD, CICIDS2017) prevalent in the literature [53].

8.3 6.3 Future Work

The limitations identified in Section 5.6 and the insights gained during this research motivate the following directions for future investigation.

8.3.1 6.3.1 Deep Learning for Improved Detection

The application of deep learning architectures to SSH anomaly detection represents the most promising direction for improving detection performance, particularly for reducing the false positive rate while maintaining high recall. Three specific architectures merit investigation.

LSTM-Autoencoders. Long Short-Term Memory networks can model sequential dependencies in SSH authentication event streams that the current feature-based approach (which aggregates events into fixed 5-minute windows) may lose. An LSTM-Autoencoder trained on sequences of normal authentication events would learn to reconstruct normal patterns; sequences that produce high reconstruction error would be flagged as anomalous. Satoh et al. [57] achieved a recall of 97.2% with an LSTM approach for SSH dictionary attack detection, suggesting that deep learning could match or exceed the current system's recall while potentially reducing the FPR below 29%.

Transformer-based models. The self-attention mechanism of Transformer architectures [33] can capture long-range dependencies between authentication events that are distant in time but semantically related (e.g., reconnaissance probes preceding an attack campaign). Transformers have achieved state-of-the-art results in multiple sequence modeling domains and have recently been applied to anomaly detection in time series data.

Hybrid ensemble approaches. Rather than replacing the Isolation Forest entirely, a promising architecture would combine the IF's computational efficiency for initial screening with a deep learning model for refined classification of ambiguous cases. Events with anomaly scores near the decision boundary — where false positives are most likely — could be forwarded to a more expressive model for secondary analysis, combining the throughput of IF with the accuracy of deep learning.

8.3.2 6.3.2 Online Learning and Continuous Adaptation

The current system requires periodic retraining to address concept drift as both attack patterns and normal user behavior evolve over time [8]. This limitation can be addressed through online or incremental learning techniques.

Incremental Isolation Forest. The standard Isolation Forest algorithm does not support incremental updates; adding new training data requires rebuilding the entire ensemble from scratch. Recent work on streaming Isolation Forest variants enables the incorporation of new data points by selectively replacing the oldest trees in the ensemble with trees built on recent data. This approach would enable continuous model adaptation with bounded computational cost.

Online One-Class SVM. Online variants of One-Class SVM, such as those based on stochastic gradient descent or incremental support vector updates, can adapt the decision boundary as new normal data arrives. This would complement the Isolation Forest by providing a continuously updated secondary model for ensemble scoring.

Adaptive retraining policies. Rather than retraining on a fixed schedule, the system could monitor detection

metrics (FPR, alert rate, score distribution statistics) and trigger retraining only when significant distribution shift is detected. Statistical tests such as the Page-Hinkley test or ADWIN (Adaptive Windowing) [55] could be employed to detect distribution shifts in the anomaly score stream.

8.3.3 6.3.3 Multi-Protocol Extension

The current system is designed specifically for SSH brute-force detection. However, the architecture — log parser, feature extractor, anomaly detection model, dynamic threshold, automated response — is generalizable to brute-force detection on other authentication protocols:

- **RDP (Remote Desktop Protocol).** RDP brute-force attacks target Windows servers and produce Windows Event Log entries (Event IDs 4625, 4624) analogous to SSH auth.log entries. A custom parser and RDP-specific features (e.g., NLA negotiation timing, session type distribution) would enable detection.
- **FTP (File Transfer Protocol).** FTP brute-force attacks produce authentication failure logs (vsftpd, ProFTPD, Pure-FTPd) with timing and count characteristics similar to SSH attacks. The existing feature set would require minimal adaptation.
- **HTTP authentication.** Web application login brute-force attacks produce HTTP 401/403 responses with timing patterns amenable to the same temporal feature analysis. Integration with web server access logs (Apache, Nginx) or web application frameworks would extend the system's coverage.
- **SMTP authentication.** Email server brute-force attacks target SMTP AUTH mechanisms and produce authentication failure logs with similar characteristics.

A multi-protocol system would share the core detection engine (Isolation Forest, dynamic threshold, Fail2Ban integration) while maintaining protocol-specific parsers and feature extractors, significantly increasing practical utility for organizations with diverse service exposure.

8.3.4 6.3.4 Federated Detection

Implementing federated learning [33] would enable multiple servers to collaboratively improve detection models without centralizing sensitive log data. This approach is relevant in three scenarios:

- **Distributed organizations.** Organizations with SSH servers in multiple geographic locations or security domains may face privacy, regulatory, or bandwidth constraints that prevent log centralization. Federated learning enables each server to train a local model and share only model parameters (not raw data) with a central aggregator.
- **Industry collaboration.** Multiple organizations facing similar SSH threats could collaboratively train a shared model without exposing their internal traffic patterns, creating a more robust and generalizable detection capability.
- **Coordinated attack detection.** Distributed attacks that simultaneously target multiple servers are difficult to detect from any single server's perspective. Federated detection enables cross-server correlation

of anomaly patterns without centralized data collection.

8.3.5 6.3.5 Threat Intelligence Integration

Combining the behavioral anomaly detection system with external threat intelligence feeds would provide a multi-evidence classification framework. Specific integration points include:

- **IP reputation databases.** Services such as AbuseIPDB [28], Shodan, and GreyNoise maintain continuously updated databases of IP addresses associated with malicious activity. An IP flagged by multiple reputation services could receive a prior probability adjustment, effectively lowering the anomaly score threshold required to trigger an alert.
- **STIX/TAXII feeds.** Structured Threat Information eXpression (STIX) and Trusted Automated eXchange of Intelligence Information (TAXII) provide standardized formats for sharing cyber threat intelligence. The system could subscribe to SSH-specific STIX feeds to receive indicators of compromise (IoCs) such as attacker IP ranges, username wordlists, and tool signatures.
- **Dark web monitoring.** Credential dumps and SSH key leaks published on dark web forums could inform the system's detection priorities, enabling proactive monitoring for compromised accounts.

This integration would transform the system from a purely behavioral detector into a contextually aware threat assessment platform, potentially reducing the FPR significantly by corroborating behavioral anomalies with external intelligence.

8.3.6 6.3.6 SOAR Integration

Integrating the system with Security Orchestration, Automation and Response (SOAR) platforms such as Splunk SOAR, Palo Alto XSOAR, or IBM Security QRadar SOAR would automate the full incident response lifecycle:

- **Automated playbook execution.** Upon detection of an ALERT-level event, the SOAR platform could execute predefined playbooks: blocking the offending IP at the firewall (not just via Fail2Ban), initiating packet capture for forensic analysis, querying threat intelligence APIs for enrichment, and creating a case in the incident management system.
- **Notification and escalation.** Automated notification through email, Slack, PagerDuty, or other channels according to organizational escalation procedures, with severity-based routing (EARLY_WARNING to Tier 1, ALERT to Tier 2).
- **Compliance reporting.** Automated generation of incident reports for regulatory compliance frameworks (ISO 27001, SOC 2, GDPR), including timeline reconstruction, evidence preservation, and response documentation.
- **Feedback loop.** Analyst disposition of alerts (true positive, false positive, inconclusive) could be fed back to the detection model for continuous performance improvement through active learning.

8.3.7 6.3.7 Explainability and Interpretability

The current system produces anomaly scores and binary classifications (normal/attack) but provides limited explanation of why a particular session or IP was flagged. Adding explainability features would significantly increase utility for security analysts:

- **SHAP (SHapley Additive exPlanations) values.** Computing SHAP values for each prediction would identify which features contributed most to the anomaly score. An analyst who sees that an IP was flagged because of unusually short session durations (session_duration_mean = 1.2 s vs. normal mean of 340 s) and regular inter-attempt timing (std_inter_attempt_time = 0.3 s vs. normal mean of 45 s) can make a more informed triage decision than one who sees only a numerical score.
- **Natural language explanations.** Translating SHAP-based feature contributions into human-readable explanations (e.g., "This IP was flagged because session durations are 99.6% shorter than normal and inter-attempt timing is 99.3% more regular than normal") would further reduce the cognitive burden on analysts.
- **Visual attention maps.** For deep learning extensions (Section 6.3.1), attention weights from Transformer models could visualize which authentication events in a sequence contributed most to the anomaly decision.

8.3.8 6.3.8 Long-Term IP Profiling

The current 5-minute sliding window approach is effective for most attack scenarios but limited for extremely slow attacks that spread over hours or days. A long-term IP profiling mechanism would complement the short-term detection:

- **Behavioral baselines per IP.** Maintaining a historical profile of each IP's SSH behavior (typical session durations, login times, success rates) would enable detection of deviations from that IP's individual baseline, rather than from the global baseline.
- **Reputation scoring.** A continuously updated per-IP reputation score, combining behavioral history with external threat intelligence, would enable risk-based adaptive thresholding: IPs with clean histories receive higher thresholds (fewer false positives), while IPs with suspicious histories receive lower thresholds (higher sensitivity).
- **Campaign detection.** Correlating anomalous events across multiple IPs over extended time windows could reveal coordinated attack campaigns that individual window-based detection misses.

8.3.9 6.3.9 Real-World Deployment Validation

While the experimental results are encouraging, the ultimate validation of the system requires deployment in a production environment with live SSH traffic over an extended period. A pilot deployment at FPT University or a partner organization would provide empirical evidence on several questions that controlled experiments cannot

fully answer: How does the FPR behave over weeks and months of continuous operation? How effectively does the dynamic threshold adapt to genuine shifts in organizational usage patterns? How do administrators interact with the alert system? How does the system perform against adversaries who actively attempt to evade AI-based detection? A structured pilot program with defined success criteria, instrumented data collection, and periodic review would provide the strongest evidence for production readiness.

8.4 REFERENCES

[1] S. Morgan, "Cybercrime to cost the world $10.5 trillion annually by 2025," *Cybersecurity Ventures*, Nov. 2020. [Online]. Available: https://cybersecurityventures.com/cybercrime-damages-6-trillion-by-2021/

[2] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," RFC 4251, Internet Engineering Task Force, Jan. 2006.

[3] SANS Internet Storm Center, "DShield: Top 10 Target Ports," [Online]. Available: https://isc.sans.edu/top10.html. [Accessed: Jan. 2025].

[4] Rapid7, "2023 Attack Intelligence Report," *Rapid7 Research*, 2023.

[5] National Cyber Security Center (NCSC), "Annual Report on Cybersecurity in Vietnam," Ministry of Information and Communications, Hanoi, Vietnam, 2023.

[6] D. R. Tsai, A. Y. Chang, and S. H. Wang, "A study of SSH brute force attack defense," *Journal of Information Security and Applications*, vol. 49, pp. 102–113, Dec. 2019.

[7] M. Najafabadi, T. M. Khoshgoftaar, C. Calvert, and C. Kemp, "Detection of SSH brute force attacks using aggregated netflow data," in *Proc. IEEE 14th Int. Conf. Machine Learning and Applications (ICMLA)*, Miami, FL, USA, 2015, pp. 283–288.

[8] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2nd Quart. 2016.

[9] M. A. F. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, "A review of novelty detection," *Signal Processing*, vol. 99, pp. 215–249, Jun. 2014.

[10] A. Simoiu, C. Gates, J. Bonneau, and S. Goel, "I was told to buy a software or lose my computer. I ignored it: A study of ransomware," in *Proc. 15th Symp. Usable Privacy and Security (SOUPS)*, Santa Clara, CA, USA, 2019, pp. 155–174.

[11] J. Jang-Jaccard and S. Nepal, "A survey of emerging threats in cybersecurity," *Journal of Computer and System Sciences*, vol. 80, no. 5, pp. 973–993, Aug. 2014.

[12] F. Syed, M. Bashir, and A. Sharaff, “Machine learning approaches for intrusion detection in IoT: A comprehensive survey,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 10, pp. 9656–9688, Nov. 2022.

[13] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, Oakland, CA, USA, 2010, pp. 305–316.

[14] T. Ylonen, “SSH – Secure login connections over the Internet,” in *Proc. 6th USENIX Security Symp.*, San Jose, CA, USA, 1996, pp. 37–42.

[15] D. J. Barrett, R. E. Silverman, and R. G. Byrnes, *SSH, The Secure Shell: The Definitive Guide*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2005.

[16] T. Ylonen and C. Lonvick, “The Secure Shell (SSH) Authentication Protocol,” RFC 4252, Internet Engineering Task Force, Jan. 2006.

[17] T. Ylonen and C. Lonvick, “The Secure Shell (SSH) Connection Protocol,” RFC 4254, Internet Engineering Task Force, Jan. 2006.

[18] OpenSSH, “sshd_config – OpenSSH daemon configuration file,” *OpenBSD Manual Pages*. [Online]. Available: https://man.openbsd.org/sshd_config

[19] D. Florencio and C. Herley, “A large-scale study of web password habits,” in *Proc. 16th Int. Conf. World Wide Web (WWW)*, Banff, AB, Canada, 2007, pp. 657–666.

[20] M. Durmuth, T. Kranz, and M. Mannan, “On the real-world effectiveness of SSH brute-force attacks,” in *Proc. NDSS Workshop on Usable Security (USEC)*, San Diego, CA, USA, 2015.

[21] A. Sperotto, G. Schaffrath, R. Sadre, C. Morariu, A. Pras, and B. Stiller, “An overview of IP flow-based intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 12, no. 3, pp. 343–356, 3rd Quart. 2010.

[22] M. Bishop, “A taxonomy of Unix password attacks,” in *Proc. Computer Security Applications Conf. (ACSAC)*, New Orleans, LA, USA, 1995.

[23] J. Owens and J. Matthews, “A study of passwords and methods used in brute-force SSH attacks,” in *Proc. USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET)*, San Francisco, CA, USA, 2008.

[24] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang, “Targeted online password guessing: An underestimated threat,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Vienna, Austria, 2016, pp. 1242–1254.

[25] B. Cheswick and S. M. Bellovin, *Firewalls and Internet Security: Repelling the Wily Hacker*, 2nd ed. Reading, MA, USA: Addison-Wesley, 2003.

[26] A. K. Das, J. Bonneau, M. Caesar, N. Borisov, and X. Wang, “The tangled web of password reuse,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2014.

[27] Fail2Ban Project, “Fail2Ban documentation,” [Online]. Available: https://www.fail2ban.org/. [Accessed: Jan. 2025].

[28] AbuseIPDB, “AbuseIPDB – IP address threat intelligence,” [Online]. Available: https://www.abuseipdb.com/. [Accessed: Jan. 2025].

[29] M. Roesch, “Snort – Lightweight intrusion detection for networks,” in *Proc. USENIX Systems Administration Conf. (LISA)*, Seattle, WA, USA, 1999.

[30] M. Krzywinski, “Port knocking: Network authentication across closed ports,” *SysAdmin Magazine*, vol. 12, pp. 12–17, 2003.

[31] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, “A detailed investigation and analysis of using machine learning techniques for intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 686–728, 1st Quart. 2019.

[32] M. Ahmed, A. Naser Mahmood, and J. Hu, “A survey of network anomaly detection techniques,” *Journal of Network and Computer Applications*, vol. 60, pp. 19–31, Jan. 2016.

[33] G. Pang, C. Shen, L. Cao, and A. van den Hengel, “Deep learning for anomaly detection: A review,” *ACM Computing Surveys*, vol. 54, no. 2, Art. no. 38, Mar. 2021.

[34] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, Art. no. 15, Jul. 2009.

[35] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation-based anomaly detection,” *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 1, Art. no. 3, Mar. 2012.

[36] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

[37] S. Hariri, M. C. Kind, and R. J. Brunner, “Extended Isolation Forest,” *IEEE Trans. Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1479–1489, Apr. 2021.

[38] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, USA, 2000, pp. 93–104.

[39] B. Scholkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, Jul. 2001.

[40] D. M. J. Tax and R. P. W. Duin, “Support vector data description,” *Machine Learning*, vol. 54, no. 1, pp. 45–66, Jan. 2004.

[41] M. Goldstein and S. Uchida, “A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data,” *PLOS ONE*, vol. 11, no. 4, Art. no. e0152173, Apr. 2016.

[42] S. W. Roberts, “Control chart tests based on geometric moving averages,” *Technometrics*, vol. 1, no. 3, pp. 239–250, Aug. 1959.

[43] P. Casas, J. Mazel, and P. Owezarski, “Unsupervised network intrusion detection systems: Detecting the unknown without knowledge,” *Computer Communications*, vol. 35, no. 7, pp. 772–783, Apr. 2012.

[44] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide*. Sebastopol, CA, USA: O’Reilly Media, 2015.

[45] Elastic, “Elasticsearch Reference,” [Online]. Available: https://www.elastic.co/guide/en/elasticsearch/reference/current/. [Accessed: Jan. 2025].

[46] Elastic, “Logstash Reference,” [Online]. Available: https://www.elastic.co/guide/en/logstash/current/. [Accessed: Jan. 2025].

[47] Elastic, “Kibana Guide,” [Online]. Available: https://www.elastic.co/guide/en/kibana/current/. [Accessed: Jan. 2025].

[48] D. Gonzalez, T. Hayajneh, and M. Carpenter, “ELK-based security analytics for anomaly detection in IoT environments,” *IEEE Access*, vol. 9, pp. 120827–120841, 2021.

[49] A. Chuvakin, K. Schmidt, and C. Phillips, *Logging and Log Management: The Authoritative Guide to Understanding the Concepts Surrounding Logging and Log Management*. Waltham, MA, USA: Syngress, 2012.

[50] Elastic, “Machine Learning in the Elastic Stack,” [Online]. Available: https://www.elastic.co/what-is/elasticsearch-machine-learning. [Accessed: Jan. 2025].

[51] R. Hofstede, A. Pras, and A. Sperotto, “Flow-based SSH compromise detection,” in *Proc. IFIP/IEEE Int. Symp. Integrated Network Management (IM)*, 2018.

[52] P. Kumari and R. Jain, “Isolation Forest based anomaly detection for IoT systems,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 8, pp. 5765–5774, Sep. 2022.

[53] N. Moustafa and J. Slay, “The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 and the KDD99 data sets,” *Information Security Journal: A Global Perspective*, vol. 25, no. 1–3, pp. 18–31, 2016.

[54] O. Starov, Y. Zhu, and N. Nikiforakis, “Detecting SSH brute-force attacks using temporal behavioral analysis,” in *Proc. IEEE Conf. Communications and Network Security (CNS)*, Washington, DC, USA, 2019.

[55] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, “Unsupervised real-time anomaly detection for streaming data,” *Neurocomputing*, vol. 262, pp. 134–147, Nov. 2017.

[56] A. Sperotto, R. Sadre, F. van Vliet, and A. Pras, “A labeled data set for flow-based intrusion detection,” in *Proc. IEEE Int. Workshop on IP Operations and Management (IPOM)*, Venice, Italy, 2009.

[57] A. Satoh, Y. Nakamura, and T. Ikenaga, "SSH dictionary attack detection based on flow analysis using deep learning," *IEEE Access*, vol. 10, pp. 75614–75627, 2022.

[58] V. T. Nguyen and M. Q. Tran, "Application of machine learning in network intrusion detection for Vietnamese enterprise networks," *Journal of Science and Technology – University of Danang*, vol. 19, no. 5, pp. 45–51, 2021.

[59] H. V. Le, T. H. Nguyen, and M. D. Pham, "Building a network security monitoring system using ELK Stack for small and medium enterprises in Vietnam," *Journal of Information and Communications Technology*, no. 3, pp. 56–64, 2022.

[60] N. H. Pham, "Research on SSH brute-force prevention solutions for government information systems," M.S. thesis, Academy of Cryptography Techniques, Hanoi, Vietnam, 2020.

[61] D. K. Tran and T. T. H. Nguyen, "Application of Isolation Forest in anomaly detection on system log data," *Journal of Science Research and Development*, vol. 2, no. 4, pp. 112–121, 2023.

[62] M. Javed and V. Paxson, "Detecting stealthy, distributed SSH brute-forcing," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Berlin, Germany, 2013, pp. 85–96.

[63] S. S. Khan and M. G. Madden, "One-class classification: Taxonomy of study and review of techniques," *Knowledge Engineering Review*, vol. 29, no. 3, pp. 345–374, Sep. 2014.

8.5 APPENDICES

8.5.1 Appendix A: Feature Extraction Code Snippets

The complete feature extraction module is implemented in Python using pandas and numpy. The code processes SSH auth.log entries through regular expression parsing, aggregates events into 5-minute sliding windows per source IP, and computes the 14 behavioral features described in Chapter 3.

8.5.2 Appendix B: Docker Compose Configuration

The Docker Compose configuration file defines the 9 services (FastAPI, React, Elasticsearch, Logstash, Kibana, Fail2Ban, Redis, PostgreSQL, Nginx) with their respective ports, environment variables, volume mounts, and inter-service dependencies.

8.5.3 Appendix C: Isolation Forest Hyperparameter Tuning Results

Detailed results of the grid search for Isolation Forest hyperparameters, including performance metrics for all evaluated parameter combinations of n_estimators (100, 200, 300, 500), max_samples (256, 512, 1024), max_features (0.25, 0.5, 0.75, 1.0), and contamination (auto, 0.01, 0.05, 0.1).

8.5.4 Appendix D: Attack Simulation Scripts

The five attack scenario simulation scripts, implemented in Python using the Paramiko SSH library, with configurable parameters for attack speed, IP distribution, username lists, and password dictionaries.

8.5.5 Appendix E: Logstash Pipeline Configuration

The Logstash pipeline configuration for parsing SSH auth.log entries, including Grok patterns for all event types (Failed password, Accepted password, Invalid user, etc.) and the Elasticsearch output configuration with index templates.

8.5.6 Appendix F: Dynamic Threshold Parameter Sensitivity Analysis

Complete sensitivity analysis results for the four dynamic threshold parameters (alpha, base_percentile, sensitivity_factor, lookback), including performance metrics across the full range of evaluated values and visual representations of threshold behavior under different parameter settings.